

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT HƯNG YÊN

NGUYỄN THỊ QUYÊN

ỨNG DỤNG AI SINH KEYWORD KIỂM THỬ TỰ ĐỘNG
ỨNG DỤNG WEB VỚI ROBOT FRAMEWORK

ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

HƯNG YÊN – 2026

NGUYỄN THỊ QUYÊN

ỨNG DỤNG AI SINH KEYWORD KIỂM THỬ TỰ ĐỘNG ỨNG DỤNG WEB

VỚI ROBOT FRAMEWORK

2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT HƯNG YÊN

NGUYỄN THỊ QUYÊN

ỨNG DỤNG AI SINH KEYWORD KIỂM THỬ TỰ ĐỘNG
ỨNG DỤNG WEB VỚI ROBOT FRAMEWORK

NGÀNH: KỸ THUẬT PHẦN MỀM
CHUYÊN NGÀNH: KIỂM THỬ VÀ ĐẢM BẢO
CHẤT LƯỢNG PHẦN MỀM

ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

NGƯỜI HƯỚNG DẪN
ThS. ĐỖ THỊ THU TRANG

HƯNG YÊN – 2026

NHẬN XÉT

Nhận xét của giảng viên hướng dẫn:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

GIẢNG VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

LỜI CAM ĐOAN

Em xin cam đoan đồ án tốt nghiệp “Ứng dụng AI sinh keyword kiểm thử tự động ứng dụng Web với Robot Framework” là công trình nghiên cứu của bản thân. Những phần sử dụng tài liệu tham khảo trong đồ án đã được nêu rõ trong phần tài liệu tham khảo. Các số liệu, kết quả trình bày trong đồ án là hoàn toàn trung thực, nếu sai em xin chịu hoàn toàn trách nhiệm và chịu mọi kỷ luật của bộ môn và nhà trường đề ra.

Hưng Yên, ngày ... tháng ... năm.....

Sinh viên

MỤC LỤC

MỤC LỤC.....	5
DANH SÁCH CÁC THUẬT NGỮ.....	7
DANH SÁCH BẢNG BIỂU.....	9
DANH SÁCH HÌNH VẼ.....	10
CHƯƠNG 1: MỞ ĐẦU.....	13
1.1 Lý do chọn đề án.....	13
1.2 Mục tiêu của đề án.....	14
1.3 Giới hạn và phạm vi của đề án.....	15
1.4 Nội dung thực hiện.....	16
1.5 Phương pháp tiếp cận.....	17
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	18
2.1 Tổng quan về kiểm thử tự động.....	18
2.2 Kiểm thử tự động theo hướng Keyword-Driven.....	21
2.3 Mô hình ngôn ngữ lớn trong sinh keyword kiểm thử.....	24
2.4 Công nghệ và công cụ sử dụng trong framework.....	28
CHƯƠNG 3: XÂY DỰNG FRAMEWORK TÍCH HỢP AI SINH KEYWORD KIỂM THỬ TỰ ĐỘNG.....	32
3.1 Giới thiệu.....	32
3.2 Môi trường triển khai.....	32
3.3 Kiến trúc tổng thể framework kiểm thử tự động.....	33
3.4 Cơ chế sinh keyword bằng AI.....	36
3.5 Cơ chế sinh luồng kiểm thử bằng AI.....	51
3.6 Cơ chế sinh luồng kiểm thử End-to-End bằng AI.....	57

3.7 Các thành phần hỗ trợ trong framework	67
CHƯƠNG 4: TRIỂN KHAI VÀ THỰC THI KIỂM THỬ TỰ ĐỘNG	77
4.1 Giới thiệu chung	77
4.2 Đặc tả hệ thống phần mềm	78
4.3 Thiết kế các yêu cầu kiểm thử	95
4.4 Xây dựng các trường hợp kiểm thử	99
4.5 Thực thi sinh keyword và luồng kiểm thử	116
4.6 Báo cáo kết quả kiểm thử	128
4.7 Đóng gói và hướng dẫn sử dụng framework	138
KẾT LUẬN	141
1. Kết quả đạt được của đề tài	141
2. Hạn chế của đề tài	141
3. Hướng phát triển của đề tài	142
TÀI LIỆU THAM KHẢO	143

DANH SÁCH CÁC THUẬT NGỮ

Từ viết tắt	Từ đầy đủ	Giải thích
AI	Artificial Intelligence	Trí tuệ nhân tạo, được sử dụng để hỗ trợ sinh keyword và luồng kiểm thử tự động.
API	Application Programming Interface	Giao diện lập trình ứng dụng, cho phép các phần mềm giao tiếp với nhau.
CI/CD	Continuous Integration/Continuous Deployment	Quy trình tích hợp và triển khai liên tục trong phát triển phần mềm.
E2E	End To End	Kiểm thử luồng nghiệp vụ hoàn chỉnh từ đầu đến cuối.
IDE	Integrated Development Environment	Môi trường phát triển tích hợp dùng để viết, chạy và quản lý mã nguồn.
KDT	Keyword-Driven Testing	Phương pháp kiểm thử hướng từ khóa, trong đó các thao tác kiểm thử được biểu diễn bằng keyword.
LLM	Large Language Model	Mô hình ngôn ngữ lớn, có khả năng hiểu và sinh văn bản dựa trên mô tả đầu vào.
NRF	Non-Functional Requirement	Yêu cầu phi chức năng, mô tả các tiêu chí như hiệu năng, bảo mật, độ tin cậy.
POM	Page Object Model	Mô hình quản lý locator và thành phần giao diện theo từng

Từ viết tắt	Từ đầy đủ	Giải thích
		trang nhằm tăng khả năng bảo trì.
QA/QC	Quality Assurance/Quality Control	Đảm bảo chất lượng và kiểm soát chất lượng phần mềm.
REST API	Representational State Transfer Application Programming Interface	Kiểu API được sử dụng để giao tiếp giữa hệ thống và dịch vụ/mô hình AI.
UC	Use Case	Trường hợp sử dụng, mô tả tương tác giữa người dùng và hệ thống.
UI	User Interface	Giao diện người dùng, nơi người dùng tương tác trực tiếp với hệ thống.
UML	Unified Modeling Language	Ngôn ngữ mô hình hóa dùng để biểu diễn hệ thống bằng các sơ đồ trực quan.

DANH SÁCH BẢNG BIỂU

Bảng 2.1 Các công cụ kiểm thử tự động phổ biến	20
Bảng 2.2 So sánh các công cụ LLM	27
Bảng 4.1 Danh sách yêu cầu chức năng.....	78
Bảng 4.2 Đặc tả kiểm tra dữ liệu chức năng Đăng ký	80
Bảng 4.3 Đặc tả kiểm tra dữ liệu chức năng Đăng nhập.....	83
Bảng 4.4 Đặc tả kiểm tra dữ liệu chức năng Tìm kiếm sản phẩm	86
Bảng 4.5 Đặc tả kiểm tra dữ liệu chức năng Đặt hàng	88
Bảng 4.6 Đặc tả kiểm tra dữ liệu chức năng Cập nhật hồ sơ	91
Bảng 4.7 Danh sách yêu cầu phi chức năng.....	93
Bảng 4.8 Các trường hợp kiểm thử chức năng Đăng ký.....	99
Bảng 4.9 Các trường hợp kiểm thử chức năng Đăng nhập	102
Bảng 4.10 Các trường hợp kiểm thử chức năng Tìm kiếm.....	104
Bảng 4.11 Các trường hợp kiểm thử chức năng Đặt hàng.....	106
Bảng 4.12 Các trường hợp kiểm thử chức năng Cập nhật hồ sơ	110
Bảng 4.13 Các trường hợp kiểm thử chức năng Mua hàng hoàn chỉnh	112
Bảng 4.14 Đánh giá kết quả AI hỗ trợ sinh keyword và flow kiểm thử	127

DANH SÁCH HÌNH VẼ

Hình 3.1 Mô hình hoạt động của framework	33
Hình 3.2 Cơ chế quản lý và inject keyword vào framework	45
Hình 4.1 Use case tổng quát các yêu cầu chức năng	78
Hình 4.2 Giao diện Đăng ký tài khoản.....	80
Hình 4.3 Use case phân rã chức năng Đăng ký.....	81
Hình 4.4 Giao diện Đăng nhập.....	83
Hình 4.5 Use case phân rã chức năng Đăng nhập.....	84
Hình 4.6 Giao diện Tìm kiếm sản phẩm	85
Hình 4.7 Use case phân rã chức năng Tìm kiếm.....	86
Hình 4.8 Giao diện Đặt hàng.....	87
Hình 4.9 Use case phân rã chức năng Đặt hàng.....	89
Hình 4.10 Giao diện Cập nhật hồ sơ	90
Hình 4.11 Use case phân rã chức năng Cập nhật hồ sơ	91
Hình 4.12 Thiết kế kiểm thử chức năng Đăng ký	95
Hình 4.13 Thiết kế kiểm thử chức năng Đăng nhập	96
Hình 4.14 Thiết kế kiểm thử chức năng Tìm kiếm	96
Hình 4.15 Thiết kế kiểm thử chức năng Đặt hàng	97
Hình 4.16 Thiết kế kiểm thử chức năng Cập nhật hồ sơ.....	98
Hình 4.17 Kết quả output AI sinh keyword chức năng Đăng nhập	116
Hình 4.18 Quá trình inject keyword vào framework chức năng Đăng nhập	117
Hình 4.19 Kết quả AI sinh flow chức năng Đăng nhập	118
Hình 4.20 Test script chức năng Đăng nhập	118

Hình 4.21 Kết quả output AI sinh keyword chức năng Đăng ký	119
Hình 4.22 Business keyword chức năng Đăng ký	119
Hình 4.23 Kết quả AI sinh flow chức năng Đăng ký.....	119
Hình 4.24 Test script chức năng Đăng ký.....	120
Hình 4.25 Kết quả output AI sinh keyword chức năng Tìm kiếm.....	120
Hình 4.26 Business keyword chức năng Tìm kiếm	121
Hình 4.27 Kết quả AI sinh flow chức năng Tìm kiếm.....	121
Hình 4.28 Test script chức năng Tìm kiếm.....	121
Hình 4.29 Kết quả output AI sinh keyword chức năng Đặt hàng.....	122
Hình 4.30 Business keyword chức năng Đặt hàng	122
Hình 4.31 Kết quả AI sinh flow chức năng Đặt hàng.....	123
Hình 4.32 Test script chức năng Đặt hàng.....	123
Hình 4.33 Kết quả AI sinh flow E2E chức năng Đặt hàng	124
Hình 4.34 Test script chức năng Đặt hàng hoàn chỉnh	124
Hình 4.35 Kết quả output AI sinh keyword chức năng Cập nhật hồ sơ	125
Hình 4.36 Business keyword chức năng Cập nhật hồ sơ	125
Hình 4.37 Kết quả AI sinh flow chức năng Cập nhật hồ sơ	125
Hình 4.38 Quá trình sinh flow E2E chức năng Cập nhật hồ sơ	126
Hình 4.39 Kết quả AI sinh flow E2E chức năng Cập nhật hồ sơ	126
Hình 4.40 Test script chức năng Cập nhật hồ sơ	126
Hình 4.41 Quá trình thực thi kiểm thử chức năng Đăng nhập.....	128
Hình 4.42 Log ghi lại quá trình thực thi chức năng Đăng nhập	129
Hình 4.43 Báo cáo Allure kết quả thực thi chức năng Đăng nhập	129
Hình 4.44 Log ghi lại quá trình thực thi chức năng Đăng ký	130

Hình 4.45 Báo cáo Allure kết quả thực thi chức năng Đăng ký	130
Hình 4.46 Ảnh chụp màn hình lỗi (test 3).....	131
Hình 4.47 Log ghi lại quá trình thực thi chức năng Tìm kiếm	131
Hình 4.48 Báo cáo Allure kết quả thực thi chức năng Tìm kiếm	132
Hình 4.49 Log ghi lại quá trình thực thi chức năng Đặt hàng	132
Hình 4.50 Báo cáo Allure kết quả thực thi chức năng Đặt hàng	133
Hình 4.51 Ảnh chụp màn hình lỗi chức năng Đặt hàng (test 3)	133
Hình 4.52 Log ghi lại quá trình thực thi chức năng Đặt hàng hoàn chỉnh.....	134
Hình 4.53 Báo cáo Allure kết quả thực thi chức năng Đặt hàng hoàn chỉnh.....	134
Hình 4.54 Ảnh chụp màn hình lỗi chức năng Đặt hàng (E2E test 3).....	135
Hình 4.55 Quá trình thực thi kiểm thử chức năng Cập nhật hồ sơ	135
Hình 4.56 Log ghi lại quá trình thực thi chức năng Cập nhật hồ sơ.....	136
Hình 4.57 Báo cáo Allure kết quả thực thi chức năng Cập nhật hồ sơ.....	136
Hình 4.58 Ảnh màn hình lỗi chức năng Cập nhật hồ sơ (test 5).....	137
Hình 4.59 Ảnh màn hình lỗi chức năng Cập nhật hồ sơ (test 6).....	137
Hình 4.60 Ảnh màn hình lỗi chức năng Cập nhật hồ sơ (test 7).....	138
Hình 4.61 Ảnh màn hình lỗi chức năng Cập nhật hồ sơ (test 8).....	138

CHƯƠNG 1: MỞ ĐẦU

1.1 Lý do chọn đề án

Trong bối cảnh chuyển đổi số diễn ra mạnh mẽ, các ứng dụng Web ngày càng đóng vai trò quan trọng trong hoạt động kinh doanh, đặc biệt là đối với các hệ thống thương mại điện tử. Việc đảm bảo hệ thống vận hành ổn định, chính xác và đáp ứng tốt nhu cầu người dùng trở thành yêu cầu thiết yếu đối với các doanh nghiệp và tổ chức phát triển phần mềm. Vì vậy, kiểm thử phần mềm là một công đoạn quan trọng nhằm đảm bảo chất lượng trước khi hệ thống được đưa vào vận hành thực tế.

Hiện nay, kiểm thử thủ công vẫn được sử dụng phổ biến do dễ tiếp cận và triển khai. Tuy nhiên, phương pháp này tồn tại nhiều hạn chế như tốn nhiều thời gian, chi phí nhân lực cao, khó tái sử dụng và phụ thuộc nhiều vào kinh nghiệm của kiểm thử viên. Khi hệ thống thường xuyên thay đổi hoặc mở rộng chức năng, việc thực hiện kiểm thử thủ công trở nên thiếu hiệu quả và khó đáp ứng yêu cầu phát triển phần mềm hiện đại.

Để khắc phục những hạn chế đó, kiểm thử tự động đã trở thành xu hướng được áp dụng rộng rãi trong quá trình phát triển phần mềm. Trong số các công cụ hỗ trợ kiểm thử tự động, Robot Framework là một framework mã nguồn mở phổ biến, hỗ trợ kiểm thử ứng dụng Web thông qua SeleniumLibrary và cho phép xây dựng kịch bản kiểm thử theo hướng Keyword-Driven. Phương pháp này giúp các test case có cấu trúc rõ ràng, dễ đọc, dễ bảo trì và tăng khả năng tái sử dụng.

Bên cạnh đó, sự phát triển của trí tuệ nhân tạo (AI), đặc biệt là các mô hình ngôn ngữ lớn (LLM), đã mở ra khả năng hỗ trợ tự động hóa nhiều công đoạn trong kiểm thử phần mềm. AI có thể hỗ trợ sinh keyword kiểm thử, gợi ý luồng thực thi và hỗ trợ xây dựng test script từ mô tả chức năng bằng ngôn ngữ tự nhiên. Việc kết hợp AI với mô hình Keyword-Driven giúp giảm công sức xây dựng test script thủ công, đồng thời nâng cao tốc độ và hiệu quả trong quá trình kiểm thử.

Xuất phát từ nhu cầu thực tiễn đó, đề tài “Ứng dụng AI sinh keyword kiểm thử tự động ứng dụng Web với Robot Framework” được lựa chọn nghiên cứu và thực

hiện. Đề tài tập trung xây dựng hệ thống keyword kiểm thử tự động cho ứng dụng Web bằng Robot Framework, đồng thời ứng dụng AI hỗ trợ sinh keyword và luồng kiểm thử dựa trên mô tả chức năng đầu vào.

Đề tài không chỉ mang ý nghĩa học thuật trong việc tiếp cận các phương pháp kiểm thử hiện đại mà còn có giá trị thực tiễn khi áp dụng kiểm thử tự động cho hệ thống Web thương mại điện tử, góp phần nâng cao chất lượng phần mềm và tối ưu quy trình kiểm thử trong môi trường phát triển phần mềm hiện nay.

1.2 Mục tiêu của đề án

1.2.1 Mục tiêu tổng quát

Xây dựng mô hình ứng dụng AI hỗ trợ sinh keyword kiểm thử tự động cho ứng dụng Web với Robot Framework nhằm nâng cao hiệu quả, khả năng tái sử dụng và giảm công sức xây dựng kịch bản kiểm thử.

1.2.2 Mục tiêu cụ thể

- Xây dựng framework kiểm thử tự động ứng dụng Web theo hướng Keyword-Driven bằng Robot Framework và SeleniumLibrary.
- Áp dụng Page Object Model để quản lý locator, giúp tách biệt phần định vị phần tử giao diện với logic kiểm thử.
- Tổ chức hệ thống keyword theo các tầng Business/UI/Verify nhằm hỗ trợ tái sử dụng và bảo trì.
- Ứng dụng AI thông qua Ollama Local API để hỗ trợ sinh keyword kiểm thử từ mô tả chức năng đầu vào.
- Xây dựng cơ chế xử lý output của AI, kiểm tra trùng lặp keyword và inject keyword vào đúng tầng trong framework.
- Xây dựng cơ chế sinh flow kiểm thử và flow End-to-End bằng AI dựa trên keyword repository hiện có.
- Thực nghiệm trên các chức năng chính của website thương mại điện tử.

- Ghi nhận log, chụp ảnh màn hình khi lỗi và sinh báo cáo kết quả kiểm thử bằng Robot Framework và Allure Report.
- Đánh giá khả năng hỗ trợ của framework trong việc sinh keyword, sinh flow, tái sử dụng keyword và hỗ trợ tester xây dựng automation test.

1.3 Giới hạn và phạm vi của đề án

1.3.1 Đối tượng nghiên cứu

- Đối tượng nghiên cứu: Hệ thống keyword kiểm thử tự động ứng dụng Web theo hướng Keyword-Driven sử dụng Robot Framework và AI hỗ trợ sinh keyword kiểm thử.
- Khách thể nghiên cứu:
 - Kiểm thử viên phần mềm (QA/QC), lập trình viên tham gia phát triển và kiểm thử hệ thống.
 - Người dùng sử dụng website để thực hiện các chức năng mua sắm và đặt hàng.
 - Giảng viên hướng dẫn và những người nghiên cứu trong lĩnh vực kiểm thử phần mềm.

1.3.2 Phạm vi nghiên cứu

- Phạm vi không gian: Triển khai và thử nghiệm framework kiểm thử trên một website thương mại điện tử mẫu, trong môi trường học tập và nghiên cứu tại Trường Đại học SPKT Hưng Yên.
- Phạm vi thời gian: từ tháng 03/2026 đến 05/2026
- Ý nghĩa khoa học và thực tiễn của đề tài:

Đề tài góp phần hệ thống hóa kiến thức về kiểm thử tự động, đặc biệt là phương pháp Keyword-Driven Testing kết hợp Robot Framework và AI. Việc tích hợp AI hỗ trợ sinh keyword kiểm thử và gợi ý luồng thực thi giúp mở rộng hướng tiếp cận mới trong xây dựng framework kiểm thử tự động hiện đại. Đồng thời,

đề tài giúp vận dụng các kiến thức chuyên môn về lập trình, kiểm thử phần mềm và xử lý ngôn ngữ tự nhiên để xây dựng một nền tảng kiểm thử có khả năng tái sử dụng và mở rộng cao.

Framework được xây dựng giúp đơn giản hóa quy trình kiểm thử các chức năng của hệ thống Web, đặc biệt đối với các thao tác lặp đi lặp lại. Việc áp dụng AI hỗ trợ sinh test flow và test case góp phần giảm thời gian xây dựng kịch bản kiểm thử thủ công, nâng cao hiệu quả kiểm thử và hỗ trợ kiểm thử viên trong quá trình phát triển phần mềm. Ngoài ra, framework có thể được điều chỉnh và mở rộng để áp dụng cho các dự án Web tương tự trong tương lai.

1.4 Nội dung thực hiện

- Nghiên cứu cơ sở lý thuyết về kiểm thử phần mềm, kiểm thử tự động và Keyword-Driven Testing.
- Tìm hiểu Robot Framework, SeleniumLibrary, AI/LLM và các công nghệ hỗ trợ liên quan.
- Phân tích các chức năng chính của website để xác định các kịch bản kiểm thử phù hợp.
- Thiết kế kiến trúc framework kiểm thử theo hướng Keyword-Driven kết hợp Page Object Model (POM).
- Xây dựng các keyword dùng chung, keyword nghiệp vụ và test case tự động.
- Xây dựng cơ chế hỗ trợ sinh keyword và luồng kiểm thử bằng AI từ mô tả chức năng đầu vào.
- Thực thi kiểm thử và phân tích kết quả kiểm thử.
- Đánh giá hiệu quả của framework về khả năng mở rộng, tái sử dụng và độ ổn định.
- Hoàn thiện báo cáo và chuẩn bị nội dung bảo vệ đồ án tốt nghiệp.

1.5 Phương pháp tiếp cận

- Nghiên cứu tài liệu liên quan đến kiểm thử phần mềm, Robot Framework, AI/LLM và Keyword-Driven Testing.
- Phân tích và so sánh các mô hình thiết kế framework kiểm thử phổ biến.
- Áp dụng phương pháp học đi đôi với thực hành thông qua việc triển khai framework trên website thương mại điện tử cụ thể.
- Xây dựng và thử nghiệm mô hình hỗ trợ sinh keyword bằng AI dựa trên mô tả chức năng đầu vào.
- Tham khảo ý kiến giảng viên hướng dẫn và những người có kinh nghiệm trong lĩnh vực kiểm thử phần mềm để hoàn thiện framework.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về kiểm thử tự động

2.1.1 Giới thiệu về kiểm thử tự động

Kiểm thử tự động là quá trình sử dụng công cụ phần mềm, script hoặc framework kiểm thử để tự động thực hiện các trường hợp kiểm thử đã được thiết kế sẵn. Thay vì kiểm thử viên phải thao tác thủ công nhiều lần trên giao diện như nhập dữ liệu, nhấn nút, kiểm tra thông báo hoặc đối chiếu kết quả, kiểm thử tự động cho phép các thao tác này được thực hiện thông qua các kịch bản kiểm thử có cấu trúc.

Trong kiểm thử ứng dụng Web, kiểm thử tự động thường được sử dụng để kiểm tra các chức năng có tính lặp lại cao như đăng ký, đăng nhập, tìm kiếm sản phẩm, thêm sản phẩm vào giỏ hàng, đặt hàng và cập nhật thông tin người dùng. Các kịch bản kiểm thử có thể được chạy lại nhiều lần nhằm phát hiện lỗi sớm, giảm công sức kiểm thử thủ công và nâng cao độ ổn định của hệ thống.

So với kiểm thử thủ công, kiểm thử tự động có một số ưu điểm như tăng tốc độ thực thi, giảm sai sót do con người, hỗ trợ kiểm thử hồi quy và dễ dàng lặp lại trong quá trình phát triển phần mềm. Tuy nhiên, kiểm thử tự động cũng yêu cầu quá trình thiết kế framework, xây dựng kịch bản, quản lý dữ liệu kiểm thử và bảo trì khi giao diện hoặc nghiệp vụ hệ thống thay đổi.

Trong phạm vi đồ án này, kiểm thử tự động được triển khai cho ứng dụng Web bằng Robot Framework kết hợp SeleniumLibrary. Các thao tác kiểm thử được tổ chức theo hướng Keyword-Driven nhằm tăng khả năng tái sử dụng, dễ đọc và dễ bảo trì. Bên cạnh đó, AI được sử dụng để hỗ trợ sinh keyword và gợi ý luồng kiểm thử từ mô tả chức năng đầu vào.

2.1.2 Quy trình kiểm thử tự động

Quy trình kiểm thử tự động là tập hợp các bước nhằm xây dựng, thực thi và bảo trì các kịch bản kiểm thử một cách có hệ thống. Tùy theo phạm vi dự án, quy trình có thể được mở rộng hoặc điều chỉnh cho phù hợp. Trong phạm vi kiểm thử ứng

dụng Web theo hướng Keyword-Driven, quy trình kiểm thử tự động có thể được mô tả qua các bước chính sau:

❖ **Xác định mục tiêu và phạm vi kiểm thử**

Đây là bước đầu tiên nhằm xác định các chức năng cần kiểm thử, loại kiểm thử áp dụng và phạm vi kiểm thử cụ thể. Đối với một website thương mại điện tử, các chức năng thường được lựa chọn kiểm thử bao gồm đăng ký, đăng nhập, tìm kiếm sản phẩm, giỏ hàng, đặt hàng và cập nhật hồ sơ người dùng. Việc xác định đúng phạm vi giúp quá trình thiết kế test case và xây dựng keyword tập trung vào các nghiệp vụ quan trọng của hệ thống.

❖ **Phân tích chức năng và thiết kế trường hợp kiểm thử**

Sau khi xác định phạm vi, kiểm thử viên tiến hành phân tích yêu cầu chức năng, luồng nghiệp vụ, dữ liệu đầu vào và kết quả mong đợi. Từ đó, các trường hợp kiểm thử được xây dựng cho cả trường hợp hợp lệ và không hợp lệ. Ví dụ, với chức năng đăng nhập, test case có thể bao gồm đăng nhập thành công, sai mật khẩu, sai email, bỏ trống email hoặc bỏ trống mật khẩu.

❖ **Xây dựng kịch bản kiểm thử tự động**

Ở bước này, các trường hợp kiểm thử được chuyển thành kịch bản tự động. Với hướng tiếp cận Keyword-Driven, các thao tác kiểm thử được biểu diễn thông qua các keyword có ý nghĩa rõ ràng. Các keyword mức thấp thực hiện thao tác trên giao diện như nhập dữ liệu, nhấn nút, chờ phần tử hiển thị; trong khi các keyword mức cao mô tả nghiệp vụ như đăng nhập, tìm kiếm sản phẩm hoặc đặt hàng.

❖ **Chuẩn bị môi trường và dữ liệu kiểm thử**

Môi trường kiểm thử cần được chuẩn bị trước khi thực thi, bao gồm trình duyệt, thư viện kiểm thử, dữ liệu đầu vào và cấu hình cần thiết. Đối với Robot Framework, môi trường cần cài đặt Python, Robot Framework, SeleniumLibrary và trình duyệt tương ứng. Dữ liệu kiểm thử có thể được nhập trực tiếp trong test case hoặc tổ chức thành file dữ liệu riêng tùy theo cách triển khai của framework.

❖ **Thực thi kiểm thử tự động**

Sau khi kịch bản kiểm thử được xây dựng, hệ thống sẽ thực thi các test case thông qua công cụ kiểm thử tự động. Trong quá trình thực thi, framework sẽ mở trình duyệt, thao tác trên giao diện Web, kiểm tra kết quả hiển thị và ghi nhận trạng thái pass/fail của từng test case. Việc thực thi tự động giúp kiểm thử viên tiết kiệm thời gian, đặc biệt đối với các chức năng cần kiểm tra lặp lại nhiều lần.

❖ Phân tích kết quả và sinh báo cáo

Kết quả kiểm thử cần được tổng hợp và trình bày rõ ràng để hỗ trợ quá trình đánh giá chất lượng phần mềm. Các thông tin thường bao gồm số lượng test case pass/fail, thời gian thực thi, lỗi phát sinh, log chi tiết và ảnh chụp màn hình khi test thất bại. Trong đồ án này, kết quả kiểm thử được ghi nhận thông qua log, báo cáo mặc định của Robot Framework và báo cáo trực quan bằng Allure Report.

❖ Bảo trì và tối ưu hóa kịch bản kiểm thử

Trong quá trình phát triển phần mềm, giao diện và logic nghiệp vụ có thể thay đổi, dẫn đến việc các locator, keyword hoặc test case cần được cập nhật. Vì vậy, bảo trì kịch bản kiểm thử là một bước quan trọng để đảm bảo framework hoạt động ổn định. Việc áp dụng mô hình Keyword-Driven kết hợp Page Object Model giúp giảm sự phụ thuộc giữa test case và chi tiết giao diện, từ đó nâng cao khả năng bảo trì và mở rộng framework.

2.1.3 Một số công cụ hỗ trợ kiểm thử tự động

Bảng 2.1 Các công cụ kiểm thử tự động phổ biến

Công cụ	Loại kiểm thử phù hợp	Đặc điểm chính
Selenium	UI Testing, Regression Testing	Mã nguồn mở, hỗ trợ đa trình duyệt và đa ngôn ngữ, tích hợp CI/CD
JUnit/PyTest	Unit Testing	Mã nguồn mở, tích hợp sâu vào ngôn ngữ lập trình (Java/Python), nhẹ và nhanh

Công cụ	Loại kiểm thử phù hợp	Đặc điểm chính
Postman	Integration Testing, API Testing	Mã nguồn mở (bản cơ bản), giao diện dễ sử dụng để kiểm thử API
JMeter	Performance Testing	Mã nguồn mở, mô phỏng tải lớn, hỗ trợ phân tích hiệu năng
Katalon Studio	UI Testing, API Testing, Mobile Testing	Công cụ thương mại, giao diện thân thiện, hỗ trợ kiểm thử đa nền tảng
TesTìim	UI Testing, Regression Testing	Công cụ thương mại, sử dụng AI để tự sửa chữa kịch bản kiểm thử
Robot Framework	UI Testing, Integration Testing, Regression Testing	Mã nguồn mở, hỗ trợ kiểm thử theo hướng Keyword-Driven Testing, dễ bảo trì và mở rộng

Trong phạm vi đề tài này, Robot Framework kết hợp với SeleniumLibrary được lựa chọn do khả năng hỗ trợ kiểm thử theo hướng Keyword-Driven, cú pháp rõ ràng, dễ đọc và phù hợp cho việc xây dựng framework kiểm thử tự động cho ứng dụng Web.

2.2 Kiểm thử tự động theo hướng Keyword-Driven

2.2.1 Khái niệm về Keyword-Driven Testing

Keyword-Driven Testing là phương pháp kiểm thử tự động trong đó các thao tác kiểm thử được trừu tượng hóa thành các từ khóa, gọi là keyword. Thay vì viết trực tiếp các câu lệnh kỹ thuật trong test case, kiểm thử viên sử dụng các keyword có ý nghĩa rõ ràng để mô tả hành động kiểm thử hoặc điều kiện cần xác minh. Cách tiếp cận này giúp test case trở nên dễ đọc, dễ hiểu và thuận tiện hơn trong quá trình bảo trì.

Trong kiểm thử tự động ứng dụng Web, một test case thường bao gồm nhiều thao tác lặp lại như mở trang, nhập dữ liệu, nhấn nút, chờ phản tử hiển thị và kiểm tra

kết quả. Khi sử dụng Keyword-Driven Testing, các thao tác này được đóng gói thành các keyword riêng biệt.

2.2.2 Keyword trong Robot Framework

Trong Robot Framework, keyword có thể được chia thành hai nhóm chính:

- Library Keywords
 - Là các keyword được cung cấp sẵn bởi thư viện, ví dụ SeleniumLibrary cung cấp các keyword.
 - Ví dụ: Click Element, Input Text, Wait Until Element Is Visible,...
 - Các keyword này chịu trách nhiệm trực tiếp thao tác với giao diện người dùng (UI) và trình duyệt.
- User Keywords
 - Là các keyword do người xây dựng framework tự định nghĩa bằng cách kết hợp nhiều library keyword hoặc nhiều user keyword khác. User keyword thường mang ý nghĩa nghiệp vụ hoặc mô tả một bước kiểm thử ở mức cao hơn.
 - Ví dụ: Login To System, Add Product To Cart, Place Order Successfully,...

2.2.3 Tổ chức keyword theo cấu trúc phân tầng

Trong phạm vi đề tài, framework kiểm thử tự động được tổ chức theo hướng Keyword-Driven với ba tầng keyword chính:

Business Layer là tầng chứa các keyword mô tả hành động hoặc luồng nghiệp vụ của người dùng. Các keyword ở tầng này thể hiện mục tiêu nghiệp vụ như đăng nhập, tìm kiếm sản phẩm, đặt hàng hoặc cập nhật hồ sơ. Business keyword giúp test case dễ đọc hơn vì các bước kiểm thử được biểu diễn gần với ngôn ngữ nghiệp vụ.

UI Action Layer là tầng chứa các keyword thao tác giao diện dùng chung. Các keyword ở tầng này thực hiện các hành động kỹ thuật như nhập dữ liệu vào ô input, nhấn vào phần tử, chọn giá trị, cuộn trang hoặc chờ phần tử hiển thị. Tầng này giúp

tái sử dụng các thao tác giao diện trong nhiều chức năng khác nhau và hạn chế việc lặp lại mã kiểm thử.

Verification Layer là tầng chứa các keyword dùng để xác minh kết quả kiểm thử. Các keyword ở tầng này kiểm tra trạng thái hiển thị của phần tử, nội dung thông báo, nội dung trang hoặc kết quả sau khi người dùng thực hiện một hành động. Việc tách riêng tầng Verification giúp các bước kiểm tra rõ ràng hơn và có thể tái sử dụng trong nhiều test case.

Việc tổ chức keyword theo ba tầng giúp framework có cấu trúc rõ ràng, dễ mở rộng và dễ bảo trì. Khi giao diện hệ thống thay đổi, kiểm thử viên có thể cập nhật keyword ở tầng UI Action hoặc locator tương ứng mà không cần sửa toàn bộ test case. Khi nghiệp vụ thay đổi, các business keyword có thể được điều chỉnh mà vẫn tái sử dụng được các keyword thao tác giao diện và keyword xác minh.

2.2.4 Kiến trúc tổng thể framework kiểm thử tự động hướng Keyword-Driven

Framework kiểm thử tự động hướng Keyword-Driven được xây dựng dựa trên nguyên tắc tách biệt giữa test case, keyword, locator, dữ liệu kiểm thử và báo cáo kết quả. Cách tổ chức này giúp framework có cấu trúc rõ ràng, dễ tái sử dụng và dễ bảo trì khi hệ thống thay đổi.

Trong mô hình Keyword-Driven, test case không trực tiếp chứa toàn bộ thao tác kỹ thuật mà gọi đến các keyword đã được định nghĩa sẵn. Các keyword này có thể là keyword thao tác giao diện, keyword nghiệp vụ hoặc keyword xác minh kết quả. Nhờ đó, test case trở nên ngắn gọn, dễ đọc và gần với luồng nghiệp vụ của người dùng.

Framework trong đề tài được tổ chức theo các thành phần chính sau:

a. Tầng keyword

Tầng này chứa các keyword dùng chung và keyword nghiệp vụ được xây dựng dựa trên Robot Framework và SeleniumLibrary. Các keyword đóng vai trò là thành phần trung tâm trong framework, giúp chuẩn hóa các thao tác kiểm thử và tăng khả năng tái sử dụng giữa các test case.

b. Tầng test case

Tầng Test Case chứa các kịch bản kiểm thử được xây dựng bằng Robot Framework. Các test case đóng vai trò điều phối luồng kiểm thử bằng cách gọi đến các keyword tương ứng. Tầng này không nên chứa quá nhiều chi tiết thao tác giao diện, nhằm đảm bảo test case dễ đọc và dễ bảo trì.

c. Tầng Page Object

Tầng Page Object được sử dụng để quản lý locator của các phần tử giao diện Web. Thay vì viết locator trực tiếp trong test case hoặc keyword, các locator được lưu tập trung theo từng trang hoặc từng chức năng. Cách tổ chức này giúp giảm ảnh hưởng khi giao diện thay đổi, vì kiểm thử viên chỉ cần cập nhật locator tại một vị trí.

d. Tầng Utility và Report

Tầng Utility chứa các module hỗ trợ như ghi log, xử lý file, chụp ảnh màn hình khi lỗi và tích hợp báo cáo. Tầng Report chịu trách nhiệm lưu trữ và trình bày kết quả kiểm thử, bao gồm báo cáo mặc định của Robot Framework, file log và báo cáo Allure Report.

Với kiến trúc trên, framework có thể tách biệt rõ giữa nghiệp vụ kiểm thử, thao tác giao diện, xác minh kết quả và quản lý locator. Điều này giúp tăng khả năng tái sử dụng keyword, giảm trùng lặp mã kiểm thử và hỗ trợ mở rộng framework cho nhiều chức năng khác nhau của ứng dụng Web.

2.3 Mô hình ngôn ngữ lớn trong sinh keyword kiểm thử

Trong framework kiểm thử tự động theo hướng Keyword-Driven, việc xây dựng keyword và thiết kế luồng kiểm thử thường tốn nhiều thời gian, đặc biệt khi hệ thống có nhiều chức năng và nhiều trường hợp kiểm thử khác nhau. Kiểm thử viên cần phân tích yêu cầu, xác định thao tác kiểm thử, xây dựng keyword, tổ chức test case và kiểm tra kết quả mong đợi.

Sự phát triển của trí tuệ nhân tạo, đặc biệt là các mô hình ngôn ngữ lớn, mở ra khả năng hỗ trợ một phần quá trình xây dựng kịch bản kiểm thử tự động. AI có thể

phân tích mô tả chức năng bằng ngôn ngữ tự nhiên, gợi ý danh sách keyword, đề xuất luồng kiểm thử và hỗ trợ sinh test script theo cấu trúc định sẵn.

Trong đề tài này, AI được sử dụng với vai trò hỗ trợ kiểm thử viên trong quá trình sinh keyword và sinh flow kiểm thử. Dựa trên mô tả chức năng đầu vào và các ràng buộc trong prompt, mô hình AI sinh ra danh sách keyword theo các tầng Business, UI Action và Verification. Sau đó, framework tiếp tục xử lý output, kiểm tra trùng lặp và inject keyword vào đúng vị trí trong cấu trúc project.

Cách tiếp cận này giúp giảm công sức ban đầu khi xây dựng keyword và luồng kiểm thử, đồng thời hỗ trợ chuẩn hóa cách đặt tên keyword trong framework. Tuy nhiên, AI không thay thế hoàn toàn kiểm thử viên. Kiểm thử viên vẫn cần kiểm tra, hiệu chỉnh logic, bổ sung locator, dữ liệu kiểm thử và các điều kiện xác minh phù hợp trước khi thực thi kiểm thử trong thực tế.

2.3.1 Tổng quan về trí tuệ nhân tạo

Trí tuệ nhân tạo (Artificial Intelligence – AI) là lĩnh vực nghiên cứu và phát triển các hệ thống máy tính có khả năng mô phỏng các hành vi thông minh của con người như học tập, suy luận, nhận thức và ra quyết định. AI bao gồm nhiều nhánh khác nhau như học máy (Machine Learning), học sâu (Deep Learning), xử lý ngôn ngữ tự nhiên (Natural Language Processing – NLP) và thị giác máy tính (Computer Vision).

Trong lĩnh vực kiểm thử phần mềm, AI có thể được ứng dụng để hỗ trợ phân tích yêu cầu, sinh test case, phát hiện lỗi, gợi ý dữ liệu kiểm thử, phân loại lỗi hoặc tối ưu hóa quá trình kiểm thử. Đối với kiểm thử tự động, AI đặc biệt hữu ích trong các công việc liên quan đến xử lý ngôn ngữ tự nhiên, chẳng hạn như chuyển mô tả chức năng thành keyword hoặc luồng kiểm thử.

Trong phạm vi đề tài, AI được sử dụng để hỗ trợ sinh keyword và flow kiểm thử cho Robot Framework. Việc ứng dụng AI giúp kiểm thử viên giảm một phần công sức thiết kế ban đầu, đồng thời tạo cơ sở để xây dựng framework kiểm thử tự động có khả năng mở rộng tốt hơn.

2.3.2 Mô hình ngôn ngữ lớn (LLM)

Mô hình ngôn ngữ lớn, hay Large Language Model (LLM), là mô hình AI được huấn luyện trên lượng lớn dữ liệu văn bản nhằm có khả năng hiểu, phân tích và sinh ngôn ngữ tự nhiên. Các mô hình này có thể xử lý yêu cầu đầu vào dưới dạng văn bản và tạo ra phản hồi phù hợp với ngữ cảnh.

LLM có một số khả năng phù hợp với bài toán hỗ trợ kiểm thử tự động như:

- Hiểu mô tả chức năng bằng ngôn ngữ tự nhiên.
- Phân tích các hành động chính trong một luồng nghiệp vụ.
- Gợi ý keyword kiểm thử theo cấu trúc định sẵn.
- Sinh flow kiểm thử dựa trên danh sách keyword có sẵn.
- Hỗ trợ chuẩn hóa nội dung đầu ra theo format mong muốn.

Trong đề tài này, LLM được sử dụng để phân tích mô tả chức năng đầu vào và sinh ra các keyword hoặc flow kiểm thử phục vụ Robot Framework. Tuy nhiên, output của LLM vẫn cần được kiểm tra và xử lý lại bởi framework cũng như kiểm thử viên để đảm bảo tính đúng đắn và phù hợp với hệ thống thực tế.

2.3.3 So sánh và lựa chọn công cụ LLM

Hiện nay có nhiều công cụ và nền tảng hỗ trợ sử dụng mô hình ngôn ngữ lớn, có thể chia thành hai nhóm chính: nền tảng AI chạy trên cloud và công cụ chạy mô hình cục bộ. Các nền tảng cloud thường có chất lượng mô hình tốt và dễ sử dụng, nhưng có thể phát sinh chi phí và phụ thuộc vào kết nối internet. Trong khi đó, các công cụ chạy cục bộ phù hợp với môi trường nghiên cứu, giúp dữ liệu không phải gửi ra bên ngoài và dễ kiểm soát hơn.

Trong phạm vi đề tài, một số công cụ được xem xét gồm Ollama, LM Studio và các API LLM cloud. Việc lựa chọn công cụ dựa trên các tiêu chí như hình thức triển khai, khả năng tích hợp, chi phí, tính riêng tư dữ liệu và mức độ phù hợp với môi trường học tập.

Bảng 2.2 So sánh các công cụ LLM

Tiêu chí	Ollama	LM Studio	API LLM Cloud
Hình thức triển khai	Chạy cục bộ	Chạy cục bộ	Cloud
Khả năng tích hợp	Tốt, hỗ trợ REST API	Hạn chế hơn	Tốt
Bảo mật dữ liệu	Cao	Cao	Phụ thuộc dịch vụ
Chi phí	Miễn phí	Miễn phí	Có mất phí
Phù hợp nghiên cứu	Rất phù hợp	Phù hợp	Phù hợp nhưng phụ thuộc chi phí và internet
Khả năng mở rộng	Dễ thay đổi model	Hạn chế	Phụ thuộc nhà cung cấp

Từ bảng so sánh có thể thấy Ollama phù hợp với phạm vi của đề tài nhờ khả năng chạy mô hình AI trực tiếp trên máy cục bộ, hỗ trợ Local API và không yêu cầu gửi dữ liệu kiểm thử ra bên ngoài. Điều này giúp đảm bảo tính riêng tư của dữ liệu, đồng thời phù hợp với môi trường học tập và nghiên cứu.

Ngoài ra, Ollama hỗ trợ nhiều mô hình ngôn ngữ lớn mã nguồn mở và cho phép thay đổi mô hình tương đối linh hoạt. Vì vậy, đề tài lựa chọn Ollama làm nền tảng AI chính để hỗ trợ sinh keyword và flow kiểm thử cho Robot Framework.

2.3.4 Vai trò của AI trong framework kiểm thử

Trong framework kiểm thử tự động của đề tài, AI đóng vai trò là công cụ hỗ trợ, không thay thế hoàn toàn kiểm thử viên. AI được sử dụng để phân tích mô tả chức năng, gợi ý keyword, sinh flow kiểm thử và hỗ trợ chuẩn hóa output theo cấu trúc Robot Framework.

Vai trò chính của AI trong framework gồm:

- Hỗ trợ sinh keyword kiểm thử từ mô tả chức năng.

- Hỗ trợ phân loại keyword theo các tầng Business, UI Action và Verification.
- Hỗ trợ sinh flow kiểm thử dựa trên keyword repository.
- Hỗ trợ sinh flow End-to-End từ mô tả nghiệp vụ tổng quát.
- Giảm thời gian xây dựng kịch bản kiểm thử ban đầu.
- Hỗ trợ kiểm thử viên trong quá trình thiết kế automation test.

Tuy nhiên, AI vẫn có một số giới hạn. Kết quả sinh ra phụ thuộc vào chất lượng prompt, mô tả đầu vào và bộ keyword hiện có trong framework. AI có thể sinh thiếu bước, sai thứ tự nghiệp vụ hoặc tạo ra keyword chưa phù hợp. Vì vậy, kiểm thử viên vẫn giữ vai trò kiểm soát chính trong việc đánh giá output, hiệu chỉnh logic, bổ sung dữ liệu kiểm thử và đảm bảo tính chính xác của test script.

Cách kết hợp giữa AI và kiểm thử viên giúp tận dụng khả năng sinh nội dung nhanh của AI, đồng thời vẫn đảm bảo chất lượng kiểm thử thông qua sự kiểm soát của con người.

2.4 Công nghệ và công cụ sử dụng trong framework

Để xây dựng hệ thống kiểm thử tự động theo hướng Keyword-Driven, đề tài sử dụng các công nghệ và công cụ hỗ trợ như Robot Framework, Ollama, Selenium, Allure Report, GitHub và Logging. Các công cụ này giúp hỗ trợ xây dựng, thực thi, quản lý và theo dõi kết quả kiểm thử một cách hiệu quả.

2.4.1 Robot Framework

Robot Framework là một framework kiểm thử tự động mã nguồn mở, được xây dựng với mục tiêu hỗ trợ kiểm thử theo hướng từ khóa. Framework này cho phép xây dựng test case thông qua các keyword có cú pháp rõ ràng, dễ đọc và gần với ngôn ngữ tự nhiên. Nhờ đó, test case trong Robot Framework có thể dễ dàng được đọc hiểu bởi kiểm thử viên, lập trình viên hoặc những người tham gia vào quá trình phát triển phần mềm.

Một ưu điểm quan trọng của Robot Framework là khả năng mở rộng thông qua các thư viện. Người dùng có thể sử dụng các thư viện có sẵn như SeleniumLibrary,

OperatingSystem, String, DateTime hoặc tự xây dựng thư viện bằng Python để bổ sung các chức năng riêng cho framework. Ngoài ra, Robot Framework còn hỗ trợ tổ chức test case, resource file, keyword và biến theo cấu trúc rõ ràng, phù hợp với việc xây dựng framework kiểm thử tự động có khả năng tái sử dụng.

Trong đề tài này, Robot Framework được sử dụng làm nền tảng chính để xây dựng và thực thi các kịch bản kiểm thử tự động. Các keyword được tổ chức thành các tầng như Business Layer, UI Action Layer và Verification Layer nhằm tăng khả năng bảo trì, tái sử dụng và mở rộng framework.

2.4.2 SeleniumLibrary

SeleniumLibrary là thư viện của Robot Framework được sử dụng để kiểm thử tự động giao diện Web. Thư viện này được xây dựng dựa trên Selenium WebDriver, cho phép Robot Framework điều khiển trình duyệt và thực hiện các thao tác mô phỏng hành vi người dùng trên giao diện Web.

SeleniumLibrary cung cấp nhiều keyword phục vụ kiểm thử giao diện như mở trình duyệt, truy cập trang Web, nhập dữ liệu, nhấn nút, chờ phần tử hiển thị, kiểm tra nội dung văn bản và xác minh trạng thái của phần tử. Các keyword này đóng vai trò là nền tảng để xây dựng các keyword mức cao hơn trong framework.

Trong đề tài, SeleniumLibrary được sử dụng để thực hiện các thao tác kiểm thử trên website thương mại điện tử như đăng ký, đăng nhập, tìm kiếm sản phẩm, thêm sản phẩm vào giỏ hàng, đặt hàng và cập nhật hồ sơ người dùng. Việc kết hợp Robot Framework với SeleniumLibrary giúp framework có thể xây dựng test case theo hướng Keyword-Driven, đồng thời vẫn đảm bảo khả năng tương tác trực tiếp với giao diện Web.

2.4.3 Allure Report

Allure Report là công cụ hỗ trợ sinh báo cáo kiểm thử trực quan. Công cụ này giúp trình bày kết quả kiểm thử theo dạng dễ theo dõi, bao gồm trạng thái test case, thời gian thực thi, thông tin lỗi, log thực thi và ảnh chụp màn hình khi xảy ra lỗi.

Trong kiểm thử tự động, báo cáo kết quả là thành phần quan trọng giúp kiểm thử viên và lập trình viên phân tích nguyên nhân lỗi. So với báo cáo mặc định của Robot Framework, Allure Report có giao diện trực quan hơn, hỗ trợ hiển thị các bước thực thi và thông tin lỗi theo từng test case. Điều này giúp quá trình theo dõi kết quả kiểm thử trở nên thuận tiện hơn.

Trong đề tài này, Allure Report được tích hợp nhằm hỗ trợ hiển thị kết quả kiểm thử, ghi nhận các bước thực thi và đính kèm ảnh chụp màn hình khi test case thất bại. Việc tích hợp Allure giúp tăng khả năng phân tích lỗi và hỗ trợ quá trình đánh giá kết quả thực nghiệm.

2.4.4 GitHub

GitHub là nền tảng quản lý mã nguồn dựa trên Git, hỗ trợ lưu trữ, theo dõi thay đổi và quản lý phiên bản mã nguồn trong quá trình phát triển phần mềm. Với GitHub, người phát triển có thể quản lý lịch sử thay đổi của project, khôi phục phiên bản cũ khi cần thiết và chia sẻ mã nguồn với người khác.

Trong quá trình xây dựng framework kiểm thử tự động, GitHub được sử dụng để lưu trữ mã nguồn, quản lý các file keyword, test case, module tiện ích, prompt và tài liệu hướng dẫn sử dụng. Việc sử dụng GitHub giúp quá trình phát triển framework có tính tổ chức hơn, dễ theo dõi thay đổi và thuận tiện khi bàn giao hoặc mở rộng project trong tương lai.

2.4.5 Logging

Logging là cơ chế ghi lại thông tin trong quá trình thực thi kiểm thử nhằm hỗ trợ theo dõi hoạt động của framework và phân tích lỗi khi test case thất bại. Trong kiểm thử tự động, logging giúp kiểm thử viên biết được test case đã thực hiện đến bước nào, dữ liệu nào đã được sử dụng và lỗi xảy ra tại thời điểm nào.

Trong đề tài, framework sử dụng module logging của Python để ghi nhận thông tin thực thi kiểm thử. Các thông tin log bao gồm thời điểm bắt đầu và kết thúc test case, các bước thao tác chính, trạng thái thực thi và thông báo lỗi khi có lỗi phát sinh. Log được lưu thành file riêng để phục vụ việc kiểm tra và phân tích sau khi chạy test.

Việc sử dụng logging mang lại một số lợi ích như hỗ trợ debug nhanh khi xảy ra lỗi, tăng khả năng truy vết quá trình thực thi và hỗ trợ đối chiếu với kết quả trong báo cáo kiểm thử. Khi kết hợp với screenshot và Allure Report, logging giúp framework có khả năng theo dõi và phân tích lỗi tốt hơn.

3.4.6 Ollama

Ollama là công cụ cho phép chạy các mô hình ngôn ngữ lớn trên máy cục bộ. Thay vì gửi dữ liệu đến các dịch vụ AI trên cloud, Ollama cho phép người dùng tải và chạy mô hình trực tiếp trong môi trường local. Điều này phù hợp với các bài toán cần đảm bảo tính riêng tư dữ liệu hoặc cần thử nghiệm trong môi trường học tập.

Ollama cung cấp Local API cho phép các chương trình bên ngoài giao tiếp với mô hình AI thông qua giao thức HTTP. Framework có thể gửi prompt đến API của Ollama và nhận kết quả phản hồi dưới dạng văn bản. Kết quả này sau đó được framework xử lý để tạo ra keyword, flow hoặc test script tương ứng.

Trong đề tài, Ollama Local API được sử dụng làm cầu nối giữa module Python của framework và mô hình ngôn ngữ lớn. Quy trình tổng quát có thể mô tả như sau:

Mô tả chức năng → Prompt → Ollama Local API → LLM Model → Keyword/Flow → Robot Framework.

Ưu điểm của việc sử dụng Ollama Local API:

- Dữ liệu kiểm thử không cần gửi ra bên ngoài internet.
- Phù hợp với môi trường nghiên cứu và học tập.
- Có thể tích hợp với Python thông qua REST API.
- Có thể thay đổi mô hình LLM khi cần thử nghiệm.
- Giảm phụ thuộc vào các dịch vụ AI cloud.

CHƯƠNG 3: XÂY DỰNG FRAMEWORK TÍCH HỢP AI SINH KEYWORD KIỂM THỬ TỰ ĐỘNG

3.1 Giới thiệu

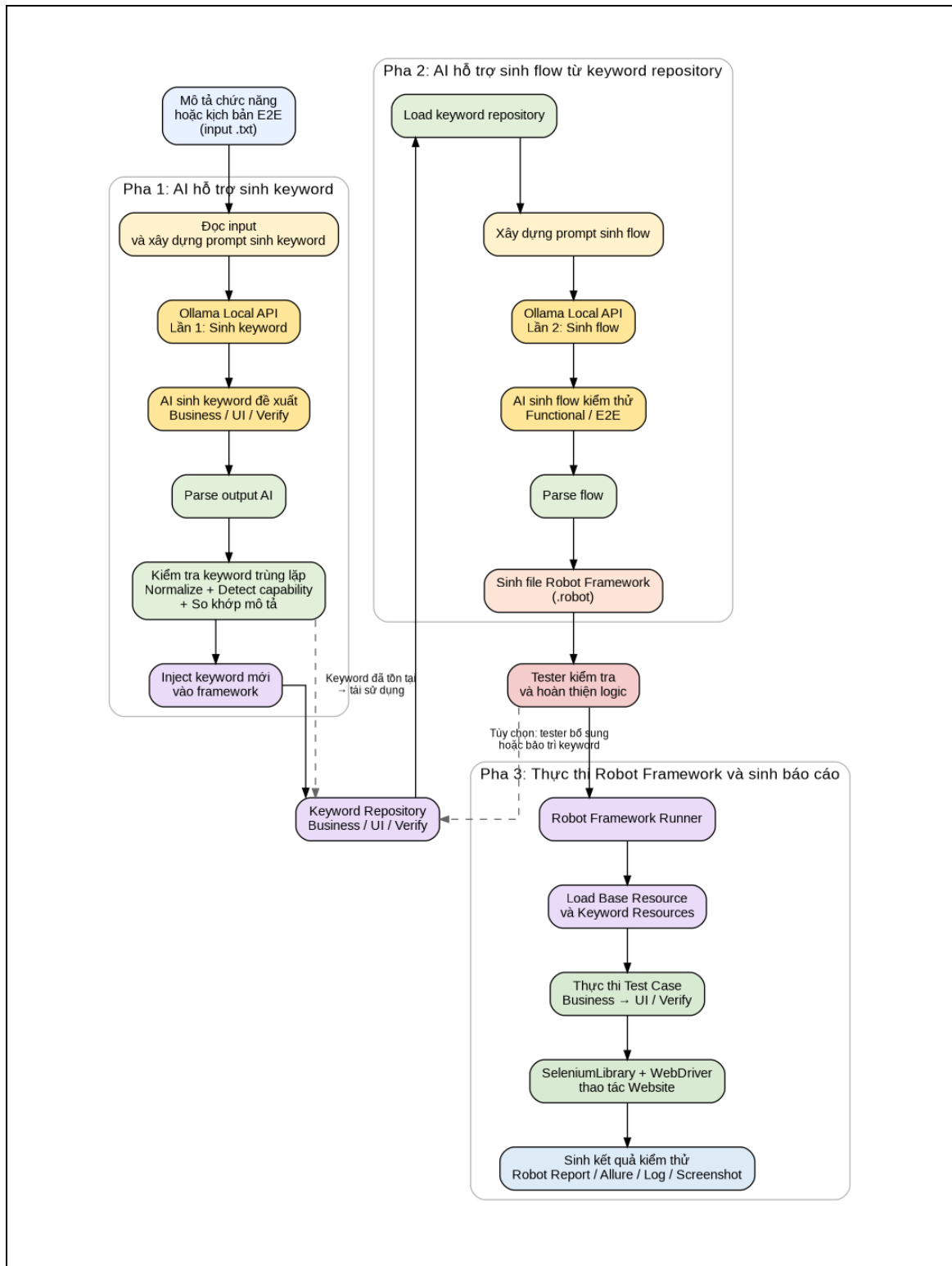
Chương này trình bày quá trình thiết kế và triển khai framework kiểm thử tự động ứng dụng Web theo hướng Keyword-Driven, kết hợp AI hỗ trợ sinh keyword và flow kiểm thử. Nội dung chương tập trung vào kiến trúc tổng thể của framework, cấu trúc thư mục, cơ chế sinh keyword bằng AI, cơ chế inject keyword vào framework, cơ chế sinh flow kiểm thử, sinh flow End-to-End và các thành phần hỗ trợ như quản lý locator, logging, screenshot và báo cáo Allure.

3.2 Môi trường triển khai

- Hệ điều hành: Windows 11
- Ngôn ngữ lập trình: Python
- IDE sử dụng: Visual Studio Code
- Công cụ kiểm thử: Robot Framework, SeleniumLibrary.
- Trình duyệt kiểm thử: Microsoft Edge.
- Công cụ AI: Ollama chạy cục bộ thông qua Local API.
- Công cụ báo cáo: Robot Framework Report/Log và Allure Report.
- Công cụ quản lý mã nguồn: GitHub.

3.3 Kiến trúc tổng thể framework kiểm thử tự động

3.3.1 Mô hình hoạt động của framework



Hình 3.1 Mô hình hoạt động của framework

3.3.2 Cấu trúc thư mục framework

Framework kiểm thử tự động được tổ chức theo mô hình phân tầng nhằm tăng khả năng mở rộng, tái sử dụng và dễ bảo trì. Các thành phần trong framework được phân chia theo từng chức năng riêng biệt như quản lý keyword, quản lý locator, sinh keyword bằng AI, quản lý test case và báo cáo kiểm thử.

Cấu trúc thư mục của framework như sau:

```
DOANTOTNGHIEP_NGUYENQUYEN_10122312
|
|— generate_keywords_use_AI/
|   |— input/           # Chứa mô tả chức năng kiểm thử.
|   |— output/          # Lưu kết quả keyword, flow được AI sinh ra.
|   |— prompt/          # Chứa các template prompt giao tiếp với mô hình AI.
|   |— generate_E2E.py  # Thực hiện sinh flow kiểm thử End-to-End bằng AI.
|   |— generate_keyword.py # Thực hiện sinh keyword, flow từ mô tả chức năng.
|
|
|— keywords/
|   |— business/        # Chứa các business keyword mô tả nghiệp vụ kiểm thử.
|   |— ui/              # Chứa các keyword thao tác giao diện người dùng.
|   |— verify/          # Chứa các keyword xác minh kết quả kiểm thử.
|   |— base_test.robot  # Chứa các keyword dùng chung và cấu hình cơ bản.
|
|
|— pages/               # Quản lý locator của từng trang web dưới dạng các file Python
|   |— home_page.py
```

```
| |— login_page.py
| |— search_page.py
| |— order_page.py
| |— register_page.py
|
|— reports/          # Chứa các kết quả và báo cáo kiểm thử sau khi thực thi.
| |— allure/
| |— logs/
| |— robot/
|
|— tests/           # Chứa các test case Robot Framework thực thi kiểm thử tự động.
| |— E2E/
| |— login_auto_*.robot
| |— order_auto_*.robot
| |— register_auto_*.robot
| |— ...
|
|— screenshots/     # Ảnh chụp màn hình khi test thất bại
|
|— utils/
| |— allure_helper.py    # Hỗ trợ tích hợp và xử lý báo cáo Allure
                           Report.
| |— logger.py           # Hỗ trợ ghi log quá trình thực thi kiểm thử.
```

```
| |— runE2E.py      # Hỗ trợ thực thi các kịch bản kiểm thử End-to-End.  
|— requirements.txt  
|— run.py  
|— README.md
```

3.4 Cơ chế sinh keyword bằng AI

3.4.1 Đầu vào của hệ thống

Hệ thống nhận đầu vào dưới dạng file mô tả chức năng kiểm thử do tester xây dựng. Các file đầu vào được lưu trong thư mục:

```
generate_keywords_use_AI/input/
```

Ví dụ nội dung đầu vào:

Chức năng: Đăng nhập

Người dùng truy cập website.

Người dùng mở form đăng nhập.

Người dùng nhập email và mật khẩu.

Người dùng nhấn nút đăng nhập.

Hệ thống kiểm tra thông tin đăng nhập.

Nếu thông tin hợp lệ, hệ thống hiển thị tài khoản người dùng.

Nếu thông tin không hợp lệ, hệ thống hiển thị thông báo lỗi.

Sau khi nhận đầu vào, hệ thống tiến hành đọc nội dung và xây dựng prompt gửi tới mô hình AI.

Đoạn mã đọc file đầu vào:

```
def read_file(path):  
    with open(path, "r", encoding="utf-8") as f:  
        return f.read()
```

3.4.2 Xây dựng *Prompt Engineering*

Hệ thống sử dụng Prompt Engineering nhằm định hướng mô hình AI sinh keyword đúng cấu trúc Robot Framework và phù hợp với framework hiện có.

Prompt được lưu riêng dưới dạng file template:

```
generate_keywords_use_AI/prompt/generate_keyword_prompt.txt
```

Trong quá trình thực thi, hệ thống sẽ thay thế nội dung mô tả chức năng vào prompt:

```
prompt_template = read_file(PROMPT_FILE)
feature_content = read_file(input_file)
prompt = prompt_template.replace("{}{FEATURE}}",feature_content)
```

Prompt bao gồm:

- Vai trò của AI.
- Quy tắc sinh keyword.
- Kiến trúc 3 layer.
- Định dạng output.
- Quy tắc self-validation.

Cách tiếp cận này giúp chuẩn hóa output và tăng độ chính xác khi sinh keyword.

You are a Senior Automation Architect designing a reusable Keyword-Driven Testing architecture using Robot Framework.

Your task is to generate reusable keywords for the given feature using exactly three layers.

Do not provide explanations. Output tables only.

ARCHITECTURE

Use exactly these layers:

1. Business Layer

2. UI Action Layer

3. Verification Layer

GLOBAL RULES

Do NOT generate:

- test cases
- automation code
- locators
- example values

Robot Framework naming convention:

- Capitalize Each Word
- Words separated by spaces
- No underscores
- No camelCase

Keywords must be reusable, generic, and data-independent.

LAYER DESIGN

BUSINESS

- Represent user actions or business workflows
- May orchestrate UI and Verification keywords
- Must NOT contain UI verbs (Click, Input, Select, Wait)
- Must NOT contain validation logic

BUSINESS KEYWORD PATTERN: <Action> <Business Goal>

UI ACTION

- Generic UI operations only
- Must NOT contain feature-specific words
- Must map to SeleniumLibrary keywords

UI KEYWORD PATTERN: <Action> <Generic UI Target>

Allowed SeleniumLibrary Keywords:

- Input Text
- Click Element
- Wait Until Element Is Visible
- Element Should Be Visible
- Element Should Contain
- Page Should Contain
- Get WebElements
- Scroll Element Into View

Derived From SeleniumLibrary Keyword MUST be chosen from the list above.

Do NOT invent Selenium keywords.

VERIFICATION

- Generic validation keywords
- Validate text, element state, navigation, or page content
- Must map to SeleniumLibrary built-in keywords
- Avoid words derived from the feature description
- Use abstract validation naming

VERIFICATION KEYWORD PATTERN: Verify <Generic Condition>

Good: Verify Element Is Visible, Verify Text Is Present

Bad: Verify Login Success, Verify Search Result

OUTPUT FORMAT (STRICT)

[BUSINESS KEYWORDS]

| Keyword Name | Input Parameters | Required Parameters | Optional Parameters | Description |

|-----|-----|-----|-----|-----|

| ... | Yes/No | parameter names or None | Yes/No | short purpose |

Rules: business actions only, no UI verbs, no validation terms.

[UI ACTION KEYWORDS]

| Keyword Name | Input Parameters | Parameters | Derived From SeleniumLibrary Keyword | Description |

|-----|-----|-----|-----|-----|

| ... | Yes/No | parameter names | Selenium keyword | generic UI action |

Rules: Must be generic UI operations such as Input Text Into Field, Click On Element, eg.

[VERIFICATION KEYWORDS]

| Keyword Name | Input Parameters | Parameters | Derived From SeleniumLibrary Keyword | Description |

|-----|-----|-----|-----|-----|

| ... | Yes/No | parameter names | Selenium keyword | validation purpose |

Rules: generic validation such as Verify Element Is Visible or Verify Text Is Present, eg.

SELF VALIDATION (MANDATORY)

Before output:

- No UI verbs in Business
- No verification logic in Business
- No feature words in UI

- No feature words in Verification
- Selenium keyword must be from allowed list
- All UI and Verification keywords map to real SeleniumLibrary built-in keywords.

If any rule is violated → regenerate the keyword list.

FEATURE DESCRIPTION

{{FEATURE}}”

3.4.3 Giao tiếp Ollama Local API

Sau khi xây dựng prompt, hệ thống gửi request tới Ollama Local API để giao tiếp với mô hình ngôn ngữ lớn (LLM).

Framework sử dụng REST API nội bộ:

<http://localhost:11434/api/generate>

Đoạn mã gửi request:

```
def call_ollama(prompt: str) -> str:
    payload = {
        "model": MODEL,
        "prompt": prompt,
        "stream": False,
        "options": {
            "temperature": 0.1,
            "num_predict": 301,
            "top_p": 0.9
        }
    }
    response = requests.post(OLLAMA_URL, json=payload)
    response.raise_for_status()
    return response.json()["response"]
```

Trong đó:

- temperature giúp giảm mức độ ngẫu nhiên của AI.
- num_predict nhằm giới hạn độ dài output, tránh việc mô hình sinh nội dung quá dài hoặc vượt khỏi format mong muốn.
- top_p giúp kiểm soát xác suất sinh output.

Việc sử dụng Ollama Local API giúp hệ thống:

- Hoạt động cục bộ.
- Không phụ thuộc internet.
- Tăng tính bảo mật dữ liệu kiểm thử.

3.4.4 AI sinh keyword kiểm thử

Sau khi nhận phản hồi từ AI, hệ thống tiến hành:

- Phân tích output.
- Phân loại keyword.
- Kiểm tra keyword trùng lặp công dụng.
- Inject keyword vào framework.

Các keyword được phân loại thành:

- BUSINESS KEYWORDS,
- UI ACTION KEYWORDS,
- VERIFICATION KEYWORDS.

Đoạn mã parse keyword:

```
def parse_keywords(response_text: str):  
    sections = {  
        "BUSINESS": [],  
        "UI": [],  
        "VERIFY": []
```

```

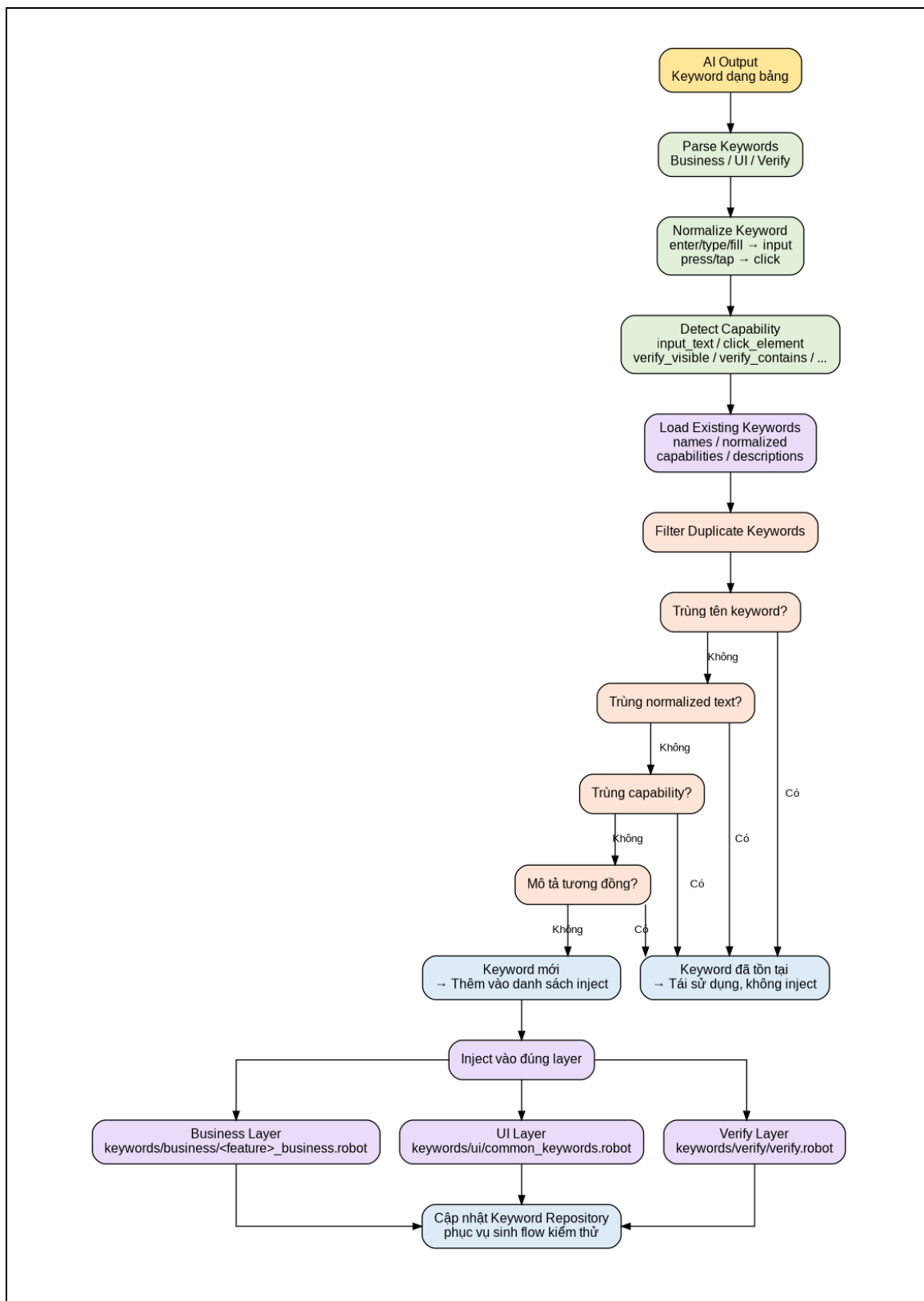
    }
    current_section = None
    for line in response_text.splitlines():
        line = line.strip()
        if "BUSINESS KEYWORDS" in line:
            current_section = "BUSINESS"
            continue
        elif "UI ACTION KEYWORDS" in line:
            current_section = "UI"
            continue
        elif "VERIFICATION KEYWORDS" in line:
            current_section = "VERIFY"
            continue

        # Skip table headers/separators
        if line.startswith("| Keyword Name") or line.startswith("|-----"):
            continue
        # Parse table rows
        if line.startswith("|") and current_section:
            parts = [p.strip() for p in line.split("|")]
            # parts example:
            # ["", 'Enter Text Into Field', 'Yes', 'locator, text', 'Input Text', 'Enter
text...', '']
            if len(parts) >= 6:
                keyword_name = parts[1]
                description = parts[-2]
                if keyword_name and keyword_name != "...":
                    sections[current_section].append({
                        "name": keyword_name,
                        "description": description,
                        "normalized": normalize_keyword(keyword_name),
                        "capability": detect_capability(keyword_name)
                    })

```

```
    })  
    return sections
```

3.4.5 Cơ chế quản lý và inject keyword vào framework



Hình 3.2 Cơ chế quản lý và inject keyword vào framework

Sau khi parse, hệ thống tiếp tục kiểm tra keyword đã tồn tại hay chưa nhằm tránh trùng lặp logic trong framework.

```
def filter_keywords(keywords, existing_names, existing_normalized,
existing_capabilities, existing_descriptions):
    new_keywords = []
    for kw in keywords:
        name = kw["name"]
        normalized = kw["normalized"]
        capability = kw["capability"]
        # duplicate name
        if name in existing_names:
            print(f"Skip duplicate name: {name}")
            continue
        # duplicate normalized text
        if normalized in existing_normalized:
            print(f"Skip semantic duplicate: {name}")
            continue
        # duplicate capability
        if capability in existing_capabilities and capability != "other":
            print(f"Reuse existing capability ({capability}) for: {name}")
            continue
        # duplicate description
        is_duplicate = False
        description = kw["description"]
        # duplicate by description similarity
        for existing_desc in existing_descriptions:
            if is_similar_description(description, existing_desc):
                print(f"Skip semantic duplicate by description: {name}")
                is_duplicate = True
                break
        if is_duplicate:
```

```
        continue
    new_keywords.append(kw)
    existing_names.add(name)
    existing_normalized.add(normalized)
    existing_capabilities.add(capability)
    existing_descriptions.add(description)
return new_keywords
```

Framework không chỉ kiểm tra trùng tên keyword mà còn thực hiện chuẩn hóa tên keyword, xác định nhóm hành động thông qua capability và kiểm tra tương đồng mô tả ở mức đơn giản. Cơ chế này giúp hạn chế việc thêm nhiều keyword có cùng mục đích sử dụng vào framework.

Ví dụ: “Enter Username”, “Input Username” sẽ được xem là cùng chức năng thông qua cơ chế normalize action.

Đoạn mã Detect capability:

```
def detect_capability(keyword_name: str):
    name = keyword_name.lower()
    if "input" in name or "enter" in name or "type" in name or "fill" in name:
        return "input_text"
    if "click" in name or "press" in name:
        return "click_element"
    if "wait" in name and "page" in name:
        return "wait_page"
    if "wait" in name:
        return "wait_element"
    if "open" in name or "navigate" in name or "go to" in name or "view" in name
    or "place" in name:
```

```
        return "open_page"

    if "scroll" in name:

        return "scroll"

    if "select" in name:

        return "select_option"

    if "verify" in name:

        if "visible" in name:

            return "verify_visible"

        if "enabled" in name:

            return "verify_enabled"

        if "disabled" in name:

            return "verify_disabled"

        if "contain" in name:

            return "verify_contains"

        if "present" in name:

            return "verify_present"

        if "not" in name:

            return "verify_not"

        return "verify_generic"

    return "other"
```

Đoạn mã normalize keyword:

```
def normalize_keyword(name: str):
    words = name.lower().split()
    normalized_words = []
    for w in words:
        if w in ACTION_SYNONYMS:
```



```
normalized_words.append(ACTION_SYNONYMS[w])
else:
    normalized_words.append(w)
return " ".join(normalized_words)
```

Đoạn mã similar description:

```
def is_similar_description(desc1: str, desc2: str):
    desc1 = desc1.lower()
    desc2 = desc2.lower()
    keywords = ["login", "search", "add", "delete", "update"]
    for kw in keywords:
        if kw in desc1 and kw in desc2:
            return True
    return False
```

Mô hình AI chỉ chịu trách nhiệm sinh danh sách keyword dưới dạng văn bản theo format quy định. Việc phân tích output, kiểm tra trùng lặp và inject keyword vào framework được xử lý bởi hệ thống Python của framework.

Sau khi kiểm tra, các keyword mới sẽ được inject vào đúng tầng của framework:

- Business keyword.
- UI keyword.
- Verify keyword.

Đoạn mã inject keyword:

```
# ===== APPEND KEYWORD MỚI =====
def append_keywords(file_path: str, new_keywords: list):
    os.makedirs(os.path.dirname(file_path), exist_ok=True)
    with open(file_path, "a", encoding="utf-8") as f:
        for kw in new_keywords:
            f.write(f"\n{kw['name']}\n")
            if kw["description"]:
```

```
f.write(f" [Documentation] {kw['description']}\n")
f.write(" # TODO: Implement\n")
# ===== GHI VÀO FRAMEWORK (CHỈ KEYWORD MỚI) =====
def write_to_framework(parsed_sections: dict, feature_name: str):
    summary = {}
    for section, keywords in parsed_sections.items():
        if section == "BUSINESS":
            file_path = f"keywords/business/{feature_name}_business.robot"
            #nếu chưa có file thì tạo mới
            if not os.path.exists(file_path):
                os.makedirs(os.path.dirname(file_path), exist_ok=True)
                with open(file_path, "w", encoding="utf-8") as f:
                    f.write("*** Settings ***\n\n*** Keywords ***\n")
            else:
                file_path = FILE_MAPPING[section]
                existing_names, existing_normalized, existing_capabilities, existing_descriptions
                = get_existing_keywords(file_path)
                new_keywords = filter_keywords(
                    keywords,
                    existing_names,
                    existing_normalized,
                    existing_capabilities,
                    existing_descriptions
                )
                if new_keywords:
                    append_keywords(file_path, new_keywords)
                summary[section] = {
                    "total_generated": len(keywords),
                    "new_added": len(new_keywords)
                }
    return summary
```

Việc lưu business keyword theo từng chức năng giúp dễ quản lý nghiệp vụ riêng của mỗi module. Trong khi đó, UI keyword và Verification keyword được lưu tập trung để tăng khả năng tái sử dụng giữa nhiều chức năng khác nhau. Cách tổ chức này phù hợp với định hướng Keyword-Driven Testing, đồng thời giúp giảm trùng lặp logic kiểm thử trong framework.

3.5 Cơ chế sinh luồng kiểm thử bằng AI

Sau khi hoàn thành quá trình sinh và inject keyword vào framework, hệ thống tiếp tục thực hiện cơ chế sinh flow kiểm thử tự động dựa trên bộ keyword hiện có trong framework.

Khác với cơ chế sinh keyword, ở giai đoạn này AI không sinh mới keyword mà sử dụng các keyword đã tồn tại trong framework để xây dựng luồng kiểm thử hoàn chỉnh theo mô tả chức năng đầu vào.

Cách tiếp cận này giúp:

- Tăng khả năng tái sử dụng keyword.
- Giảm trùng lặp logic kiểm thử.
- Đảm bảo flow sinh ra phù hợp với kiến trúc framework hiện có.
- Giữ vai trò kiểm soát logic nghiệp vụ cho tester.

3.5.1 Load keyword từ framework

Sau khi inject keyword vào framework, hệ thống tiến hành load toàn bộ keyword hiện có từ các layer:

- Business keywords.
- UI keywords.
- Verify keywords.

Các keyword này được sử dụng làm ngữ cảnh (context) cho AI trong quá trình sinh flow kiểm thử.

Đoạn mã load keyword từ framework:

```
def load_framework_keywords(feature_name):
    sections = {
        "BUSINESS": [],
        "UI": [],
        "VERIFY": []
    }
    files = {
        "BUSINESS": f"keywords/business/{feature_name}_business.robot",
        "UI": "keywords/ui/common_keywords.robot",
        "VERIFY": "keywords/verify/verify.robot"
    }
    for section, file_path in files.items():
        if not os.path.exists(file_path):
            continue
        inside_keyword_block = False
        with open(file_path, "r", encoding="utf-8") as f:
            for line in f:
                line = line.rstrip()
                # detect keyword section
                if line.startswith("*** Keywords"):
                    inside_keyword_block = True
                    continue
                if line.startswith("***"):
                    inside_keyword_block = False
                    continue
                if not inside_keyword_block:
                    continue
                # keyword = dòng không indent
                if line and not line.startswith(" "):
                    sections[section].append({
                        "name": line
```

```
    })  
    return sections
```

Sau khi load, hệ thống tiến hành xây dựng keyword repository để cung cấp cho AI.

Đoạn mã xây dựng keyword context:

```
def build_keyword_context(parsed_sections):  
    lines = []  
    for section, keywords in parsed_sections.items():  
        lines.append(f"\n{section} KEYWORDS:")  
        for kw in keywords:  
            lines.append(f"{kw['name']}")  
    return "\n".join(lines)
```

Keyword repository đóng vai trò là nguồn dữ liệu để AI lựa chọn keyword phù hợp khi sinh flow kiểm thử.

3.5.2 Xây dựng prompt sinh flow kiểm thử

Hệ thống sử dụng prompt riêng cho quá trình sinh flow kiểm thử. Prompt được lưu trong thư mục:

generate_keywords_use_AI/prompt/generate_flow_prompt.txt

Trong quá trình thực thi, hệ thống sẽ:

- Load danh sách keyword từ framework.
- Đọc mô tả chức năng đầu vào.
- Gắn keyword repository vào prompt.

Prompt sinh flow được thiết kế nhằm:

- Yêu cầu AI chỉ sử dụng keyword có sẵn trong framework.
- Không tự sinh thêm keyword mới.
- Xây dựng flow theo đúng cú pháp Robot Framework.

- Sinh luồng nghiệp vụ theo đúng mô tả chức năng đầu vào.

Nội dung Prompt:

```
ROLE: Senior Automation Architect

TASK:

Generate ONE Happy Path Main Business Flow.

RULES:

- Use ONLY the Business and Verification keywords listed below
- Use EXACT keyword names
- Do NOT invent new keywords
- No UI actions (click, input, etc.)
- Use placeholders ${VAR} for inputs
- Keep steps in logical order

AVAILABLE KEYWORDS:

{{KEYWORDS}}

USE CASE:

{{USE_CASE}}

OUTPUT:

[Main Flow Name]

1. <Business Keyword> | ${data}
2. <Business Keyword>
3. <Verification Keyword> | ${expected}
```

Việc sử dụng prompt theo hướng ràng buộc giúp AI sinh flow ổn định hơn và phù hợp với kiến trúc framework.

3.5.3 AI sinh luồng kiểm thử từ keyword repository

Sau khi xây dựng prompt, hệ thống gửi request tới Ollama Local API để AI sinh flow kiểm thử.

Đoạn mã sinh flow:

```
def generate_business_flow(feature_name, parsed_sections, use_case_text):
    prompt_template =
read_file("generate_keywords_use_AI/prompt/generate_flow_prompt.txt")
    keyword_context = build_keyword_context(parsed_sections)
    prompt = prompt_template \
        .replace("{{KEYWORDS}}", keyword_context) \
        .replace("{{USE_CASE}}", use_case_text)
    print(" Generating business flow...")
    result = call_ollama(prompt)
    output_path = os.path.join(
        OUTPUT_DIR, f"{feature_name}_flow.txt"
    )
    with open(output_path, "w", encoding="utf-8") as f:
        f.write(result)
    print(f" Saved flow to: {output_path}")
    return result
```

Kết quả AI trả về là luồng kiểm thử dạng text sử dụng các keyword đã tồn tại trong framework.

Ví dụ output:

1. Login To System
2. Verify Element Text Contains

Cách tiếp cận này giúp:

- AI chỉ đóng vai trò hỗ trợ sinh flow.
- Tester vẫn kiểm soát logic nghiệp vụ kiểm thử.

- Đảm bảo khả năng tái sử dụng keyword trong toàn framework.

Ở giai đoạn sinh flow, AI chỉ được phép lựa chọn keyword đã tồn tại trong framework. Điều này giúp hạn chế tình trạng AI tự tạo keyword mới không có trong project, đồng thời đảm bảo flow sinh ra có thể chuyển đổi thành test script Robot Framework với mức chỉnh sửa thấp hơn.

3.5.4 Sinh test script Robot Framework

Sau khi nhận flow từ AI, hệ thống tiếp tục parse flow và sinh file test script Robot Framework.

Đoạn mã parse flow:

```
def parse_flow(flow_text: str):
    steps = []
    for line in flow_text.splitlines():
        line = line.strip()
        match = re.match(r"^\d+\.\s+(.*)", line)
        if match:
            step = match.group(1).strip()
            # remove markdown
            step = re.sub(r"\*\.?\*\*", r"\1", step)
            # fix Robot syntax
            step = step.replace("|", " ")
            steps.append(step)
    return steps
```

Sau khi parse, hệ thống tự động generate file .robot.

Đoạn mã sinh Robot Framework test script:

```
def generate_robot_test(feature_name, flow_text):
    steps = parse_flow(flow_text)
    output_path = f"tests/{feature_name}_auto.robot"
    os.makedirs("tests", exist_ok=True)
```



```
business_file = f"../keywords/business/{feature_name}_business.robot"
with open(output_path, "w", encoding="utf-8") as f:
    f.write("*** Settings ***\n")
    f.write(f"Resource    {business_file}\n")
    f.write("Resource    ../keywords/ui/common_keywords.robot\n")
    f.write("Resource    ../keywords/verify/verify.robot\n\n")
    f.write("*** Test Cases ***\n")
    f.write(f"{feature_name} Auto Test\n")
    f.write("    [Documentation]    Auto generated from AI flow\n\n")
    for step in steps:
        f.write(f"        {step}\n")
print(f"Robot test generated: {output_path}")
```

Kết quả đầu ra của bước này là file test script Robot Framework được sinh tự động từ flow AI. File này đóng vai trò là bản khởi tạo ban đầu để tester kiểm tra, bổ sung dữ liệu, điều chỉnh logic và hoàn thiện trước khi thực thi chính thức.

3.6 Cơ chế sinh luồng kiểm thử End-to-End bằng AI

3.6.1 Đầu vào sinh luồng kiểm thử End-to-End

Hệ thống hỗ trợ sinh luồng kiểm thử End-to-End (E2E) dựa trên mô tả nghiệp vụ tổng quát do tester cung cấp. Đầu vào được lưu trong file:

generate_keywords_use_AI/input/

Ví dụ nội dung đầu vào:

- Người dùng đăng ký tài khoản mới.
- Đăng nhập hệ thống.
- Tìm kiếm sản phẩm.
- Đặt hàng thành công.

Hệ thống đọc nội dung scenario E2E để làm cơ sở xây dựng prompt gửi tới mô hình AI. Đoạn mã đọc file đầu vào:

```
def read_file(path):
```

```
with open(path, "r", encoding="utf-8") as f:  
    return f.read()
```

Sau khi đọc dữ liệu, hệ thống tiến hành:

- Phân tích scenario.
- Xác định nhóm keyword liên quan.
- Load keyword repository.
- Sinh flow E2E bằng AI.

3.6.2 Xây dựng Prompt Engineering cho luồng kiểm thử E2E

Để định hướng AI sinh đúng luồng kiểm thử End-to-End, hệ thống sử dụng Prompt Engineering kết hợp:

- Scenario nghiệp vụ.
- Bộ keyword hiện có trong framework.
- Các ràng buộc định dạng output.

Prompt được lưu riêng trong file:

```
generate_keywords_use_AI/prompt/generate_E2E_flow_prompt.txt
```

Trong quá trình thực thi, hệ thống thay thế:

- {{SCENARIO}}
- {{KEYWORDS}}

vào prompt template:

```
prompt_template = read_file(E2E_PROMPT)  
prompt = prompt_template \  
    .replace("{{SCENARIO}}", scenario) \  
    .replace("{{KEYWORDS}}", keyword_text)
```

Prompt yêu cầu AI:

- Chỉ sử dụng keyword có sẵn trong framework.
- Sinh flow theo đúng thứ tự nghiệp vụ.
- Không tạo thêm keyword mới.
- Không sinh code Selenium.
- Chỉ trả về danh sách step Robot Framework.

Nội dung Prompt:

You are a senior QA automation engineer.

Your task is to generate an end-to-end test using ONLY available keywords.

=== BUSINESS SCENARIO ===

{{SCENARIO }}

=== AVAILABLE KEYWORDS ===

{{KEYWORDS}}

=== STRICT RULES ===

1. ONLY use BUSINESS and VERIFY keywords
2. DO NOT use UI keywords
3. DO NOT invent new keywords
4. DO NOT explain anything
5. Follow EXACT execution logic:
 - Each action MUST be followed by a verification step
 - Only continue if previous step is verified
6. Maintain STRICT order:
 - login BEFORE search
 - search BEFORE order
 - register BEFORE login (if exists)

7. Remove unnecessary steps

8. Do NOT repeat steps

9. DO NOT use generic names like "step", "action", "process"

OUTPUT FORMAT (STRICT):

Return ONLY a JSON array like below:

```
[  
  "STEP 1",  
  "STEP 2",  
  "STEP 3",  
  ...  
]
```

PARAMETER RULES:

- Parameters MUST be inside the string
- Use SINGLE SPACE between keyword and parameters
- DO NOT use tab
- DO NOT use comma inside string
- DO NOT use comments

STRICT REQUIREMENTS:

- MUST be valid JSON
- MUST use double quotes ""
- MUST NOT have trailing comma
- MUST NOT contain comments
- MUST NOT contain line breaks
- MUST NOT contain any text outside JSON

If output is not valid JSON → it is WRONG.

DO NOT:

- explain anything
- write text before or after
- use markdown ```
- write Robot Framework code”

Cách tiếp cận này giúp:

- Tăng khả năng tái sử dụng keyword repository.
- Giảm sai lệch output của AI.
- Chuẩn hóa flow kiểm thử E2E.

3.6.3 Load keyword repository phục vụ E2E

Trước khi sinh flow E2E, hệ thống tiến hành load keyword repository từ framework.

Các keyword được phân nhóm theo chức năng: Login, Register, Search, Order, Profile.

Đoạn mã load keyword:

```
def load_keywords_grouped():
    base_dir = "keywords/business"
    groups = {
        "login": [],
        "register": [],
        "search": [],
        "order": [],
        "profile": []
    }
    for root, _, files in os.walk(base_dir):
        for file in files:
            if not file.endswith(".robot"):
                continue
```

```
path = os.path.join(root, file)
key = None
name = file.lower()
if "login" in name:
    key = "login"
elif "register" in name:
    key = "register"
elif "search" in name:
    key = "search"
elif "order" in name:
    key = "order"
elif "profile" in name:
    key = "profile"
if not key:
    continue
with open(path, "r", encoding="utf-8") as f:
    for line in f:
        line = line.strip()
        if line and not line.startswith("****") and not line.startswith("#"):
            if not line.startswith("  "):
                groups[key].append(line)

# VERIFY
verify = []
with open("keywords/verify/verify.robot", "r", encoding="utf-8") as f:
    for line in f:
        line = line.strip()
        if line and not line.startswith("****") and not line.startswith("#"):
            if not line.startswith("  "):
                verify.append(line)
return groups, verify
```

Sau khi load keyword, hệ thống tiếp tục lọc keyword phù hợp với scenario E2E nhằm giảm số lượng context gửi tới AI.

Đoạn mã filter keyword theo scenario:

```
def filter_groups_by_scenario(groups, scenario):
    scenario = scenario.lower()
    allowed = []
    if "register" in scenario or "đăng ký" in scenario:
        allowed.append("register")
    if "login" in scenario or "đăng nhập" in scenario:
        allowed.append("login")
    if "search" in scenario or "tìm kiếm" in scenario:
        allowed.append("search")
    if "order" in scenario or "cart" in scenario or "đặt hàng" in scenario:
        allowed.append("order")
    if "profile" in scenario or "hồ sơ" in scenario or "tài khoản" in scenario:
        allowed.append("profile")
    # nếu có order → thường cần login
    if "order" in allowed and "login" not in allowed:
        allowed.append("login")
    print(" Allowed groups:", allowed)
    return {k: v for k, v in groups.items() if k in allowed}
```

Cơ chế này giúp:

- Giảm token gửi tới mô hình AI.
- Tăng độ chính xác khi sinh flow.
- Giới hạn AI chỉ sử dụng keyword phù hợp với nghiệp vụ.

3.6.4 AI sinh luồng kiểm thử End-to-End

Sau khi xây dựng prompt hoàn chỉnh, hệ thống gửi request tới Ollama Local API để mô hình AI sinh flow End-to-End.

Đoạn mã gọi AI:

```
def call_ollama(prompt: str) -> str:
```

```
payload = {
    "model": MODEL,
    "prompt": prompt,
    "stream": False,
    "options": {
        "temperature": 0.1,
        "num_predict": 301,
        "top_p": 0.9
    }
}

response = requests.post(OLLAMA_URL, json=payload)
return response.json()["response"]
```

Sau khi nhận phản hồi từ AI, hệ thống tiến hành:

- Loại bỏ markdown dư thừa.
- Chuẩn hóa dữ liệu.
- Parse danh sách step kiểm thử.

Mặc dù prompt yêu cầu AI trả về JSON array để dễ xử lý, trong thực tế mô hình có thể sinh thêm markdown, số thứ tự hoặc ký tự dư thừa. Vì vậy, framework bổ sung bước làm sạch output nhằm tăng độ ổn định trước khi chuyển danh sách step thành file Robot Framework.

Đoạn mã parse flow:

```
def parse_flow(text: str):
    print("\nRAW AI OUTPUT:\n", text)
    # remove markdown
    text = re.sub(r"```.*?```", "", text, flags=re.DOTALL)
    lines = text.splitlines()
    steps = []
    for line in lines:
        line = line.strip()
```



```
if not line:
    continue
# bỏ prefix 1. / - / ...
line = re.sub(r"^\d+[\.\-])s*", "", line)
line = re.sub(r"^\-s*", "", line)
# bỏ dấu ", ở cuối
line = line.rstrip(",")
# bỏ dấu "
line = line.strip('"')
# loại rác
if any(x in line.lower() for x in [
    "here is",
    "note:",
    "output",
    "example"
]):
    continue
# normalize space
line = " ".join(line.split())
# chỉ giữ dòng hợp lệ
if len(line.split()) >= 2:
    steps.append(line)
print("\nPARSED CLEAN STEPS:", steps)
return steps
```

Kết quả đầu ra là danh sách step Robot Framework dạng:

Register New Account

Login With Valid Account

Search Product

Add Product To Cart

Verify Order Success

Ở giai đoạn này, AI chỉ hỗ trợ sinh flow dựa trên keyword repository hiện có. Tester vẫn cần:

- Kiểm tra lại flow nghiệp vụ.
- Bổ sung logic kiểm thử đặc biệt.
- Điều chỉnh dữ liệu kiểm thử.
- Hoàn thiện các assertion chuyên sâu nếu cần.

3.6.5 Sinh test script E2E Robot Framework

Sau khi parse flow thành công, hệ thống tự động sinh file test script Robot Framework.

Các file được sinh trong thư mục: tests/E2E/

Đoạn mã sinh file Robot Framework:

```
def generate_robot_file(steps):
    os.makedirs("tests/E2E", exist_ok=True)
    filename = build_flow_name(steps)
    if not filename:
        filename = "E2E_test"
    #thêm Timestamp
    Timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"{filename}_{Timestamp}"
    path = f"tests/E2E/{filename}.robot"
    # tránh ghi đè
    counter = 1
    base_path = path
    while os.path.exists(path):
        path = base_path.replace(".robot", f"_{counter}.robot")
        counter += 1
    with open(path, "w", encoding="utf-8") as f:
        f.write("*** Settings ***\n")
```

```
f.write("Resource ../keywords/business/login_business.robot\n")
f.write("Resource ../keywords/business/search_business.robot\n")
f.write("Resource ../keywords/business/order_business.robot\n")
f.write("Resource ../keywords/business/register_business.robot\n")
f.write("Resource ../keywords/verify/verify.robot\n")
f.write("\n*** Test Cases ***\n")
f.write(f"E2E {filename}\n")
f.write(" [Documentation] AI generated E2E\n\n")
for step in steps:
    f.write(f" {step}\n")
print(f"Saved Robot: {path}")
return path
```

3.7 Các thành phần hỗ trợ trong framework

Sau khi hoàn thiện cơ chế hỗ trợ sinh keyword và luồng kiểm thử bằng AI, framework tiếp tục được triển khai theo hướng phân tầng nhằm tăng khả năng tái sử dụng, bảo trì và mở rộng hệ thống kiểm thử.

Framework được xây dựng dựa trên:

- Robot Framework.
- SeleniumLibrary.
- Kiến trúc Keyword-Driven Testing.
- Mô hình Page Object Model (POM).
- Các module hỗ trợ logging, report và screenshot.

3.7.1 Triển khai quản lý locator theo Page Object Model

Framework sử dụng mô hình Page Object Model để quản lý locator của các phần tử giao diện Web. Thay vì viết trực tiếp locator trong test case hoặc keyword, các locator được tách riêng và lưu trong thư mục pages/. Mỗi file trong thư mục này đại diện cho một nhóm trang hoặc một chức năng tương ứng của hệ thống.

Ví dụ, file login_page.py lưu các locator phục vụ chức năng đăng nhập:

```
#LOGIN PAGE
URL = "https://ella.vn/"
LOGIN_URL = "xpath://div[@class='lf']/span[contains(text(),'Đăng nhập')]"
EMAIL_INPUT = "css:input[placeholder='Email']"
PASSWORD_INPUT = "css:input[placeholder='Password']"
LOGIN_BTN = "xpath://button[contains(text(),'Đăng nhập')]"
ERROR_MSG="xpath://div[contains(text(),'Tài khoản hoặc mật khẩu không chính xác')]"
```

Các keyword trong framework sẽ import locator từ các file tương ứng để thực hiện thao tác kiểm thử. Cách tổ chức này giúp tách biệt phần định vị phần tử giao diện khỏi logic kiểm thử, từ đó làm cho test case và keyword dễ đọc, dễ bảo trì hơn.

Khi giao diện Web thay đổi, kiểm thử viên chỉ cần cập nhật locator trong file page tương ứng mà không cần sửa nhiều test case hoặc keyword khác nhau. Ví dụ, nếu locator của nút đăng nhập thay đổi, chỉ cần cập nhật biến LOGIN_BTN trong file login_page.py.

Việc áp dụng Page Object Model trong framework mang lại một số lợi ích:

- Giảm lặp lại locator trong nhiều test case.
- Tăng khả năng bảo trì khi giao diện thay đổi.
- Tách biệt logic kiểm thử và phần định vị giao diện.
- Giúp cấu trúc project rõ ràng hơn.
- Hỗ trợ mở rộng framework khi bổ sung chức năng mới.

Trong phạm vi đề án, các locator được tách theo từng chức năng chính của website như đăng nhập, đăng ký, tìm kiếm, đặt hàng và cập nhật hồ sơ. Đây là cơ sở để các tầng keyword sử dụng lại locator một cách thống nhất trong quá trình thực thi kiểm thử tự động.

3.7.2 Triển khai Base Test Resource

File `base_test.robot` được đặt trong thư mục `keywords/` và đóng vai trò là resource dùng chung cho toàn bộ framework. File này chứa các keyword hỗ trợ thiết lập và kết thúc quá trình kiểm thử như mở trình duyệt, đóng trình duyệt, xử lý khi test case thất bại và chụp ảnh màn hình lỗi.

File triển khai: `keywords/base_test.robot`

Các thư viện được sử dụng trong `base_test.robot` bao gồm:

```
*** Settings ***
Library SeleniumLibrary run_on_failure=None
Library ../utils/logger.py
Library ../utils/allure_helper.py
Library OperatingSystem
Library String
Library DateTime
```

Trong đó:

- SeleniumLibrary hỗ trợ thao tác với trình duyệt và chụp ảnh màn hình.
- `logger.py` hỗ trợ ghi log quá trình thực thi.
- `allure_helper.py` hỗ trợ ghi step và đính kèm ảnh lỗi vào Allure Report.
- OperatingSystem, String, DateTime hỗ trợ xử lý file, chuỗi và thời gian.

a, Khởi tạo trình duyệt và logger

Keyword Open Browser Suite được sử dụng ở bước Suite Setup nhằm khởi tạo logger, mở trình duyệt và chuẩn bị môi trường kiểm thử.

```
Open Browser Suite
[Arguments] ${feature}
Init Logger ${feature}
Open Browser about:blank edge
Maximize Browser Window
```

```
Log Info =====  
Log Info  Browser opened  
Step log  Browser opened
```

Keyword này giúp:

- Khởi tạo file log theo từng chức năng.
- Mở trình duyệt Edge.
- Phóng to cửa sổ trình duyệt.
- Ghi log vào file log và Allure Report.

b, Đóng trình duyệt sau khi kiểm thử

Keyword Close Browser Suite được sử dụng ở bước Suite Teardown để đóng toàn bộ trình duyệt sau khi hoàn tất kiểm thử.

```
Close Browser Suite  
Log Info  Close browser  
Step log  Close browser  
Close All Browsers
```

c, Xử lý khi test case thất bại

Keyword Handle Test Failure được sử dụng trong Test Teardown để xử lý khi test case thất bại. Cơ chế này giúp tự động ghi log, tạo tên file screenshot theo Timestamp, chụp ảnh màn hình lỗi và đính kèm ảnh vào báo cáo Allure.

```
Handle Test Failure  
Log  Test failed: ${TEST NAME}  
Step log  Test failed: ${TEST NAME}  
# 1. Timestamp chuẩn  
${Timestamp}=  Get Current Date  result_format=%Y%m%d_%H%M%S  
# 2. Clean test name (dùng TC cho gọn)  
${clean_test_name}=  Replace String  ${TEST NAME}  ${SPACE}  _  
${clean_test_name}=  Replace String  ${clean_test_name}  :  _
```

```

    ${clean_test_name}=  Replace String  ${clean_test_name}  /  _
    ${clean_test_name}=  Replace String  ${clean_test_name}  \\  _

# 3. Folder
${dir_in}=  Set Variable  ${OUTPUT DIR}${/}screenshots
Create Directory  ${dir_in}

# 4. Folder ngoài project
${dir_out}=  Set Variable  ${CURDIR}${/}..${/}screenshots
Create Directory  ${dir_out}

# 5. Path
${path_in}=  Set Variable  ${dir_in}${/}${clean_test_name}_${Timestamp}.png
${path_out}=  Set
Variable  ${dir_out}${/}${clean_test_name}_${Timestamp}.png

# 6. Chụp vào reports/robot/screenshots
Capture Page Screenshot  ${path_in}

# 7. Copy thêm ra screenshots/ ngoài project
Copy File  ${path_in}  ${path_out}
Log  Screenshot saved at: ${path_in}
Step log  Screenshot saved at: ${path_in}
Attach Screenshot  ${path_in}
```

Cơ chế xử lý lỗi này giúp framework:

- Tự động chụp màn hình tại thời điểm lỗi xảy ra.
- Lưu screenshot trong thư mục report của Robot Framework.
- Sao chép screenshot ra thư mục riêng để dễ theo dõi.
- Đính kèm screenshot vào Allure Report.
- Hỗ trợ tester và developer phân tích nguyên nhân lỗi trực quan hơn.

Việc triển khai base_test.robot giúp tách biệt phần thiết lập và xử lý lỗi khỏi các test case cụ thể, từ đó làm cho test script gọn hơn, dễ bảo trì hơn và nhất quán hơn trong toàn bộ framework.

3.7.3 Triển khai module hỗ trợ và tiện ích

Ngoài các tầng keyword kiểm thử, framework còn xây dựng các module hỗ trợ nhằm tăng khả năng theo dõi, phân tích lỗi và quản lý kết quả kiểm thử tự động.

Framework xây dựng các module hỗ trợ trong thư mục: `utils/`

Bao gồm:

- `logger.py`: hỗ trợ ghi log quá trình thực thi kiểm thử.
- `allure_helper.py`: hỗ trợ tích hợp và hiển thị step trên Allure Report.
- `rune2e.py`: hỗ trợ thực thi các kịch bản kiểm thử End-to-End được AI sinh ra.

a, Module logging kiểm thử

Framework xây dựng module logging nhằm ghi nhận toàn bộ quá trình thực thi kiểm thử. File triển khai: `utils/logger.py`

Module hỗ trợ:

- Ghi log theo từng testcase.
- Lưu log theo ngày thực thi.
- Ghi log vào Robot Framework và Allure Report.
- Hỗ trợ debug khi xảy ra lỗi.

Đoạn mã khởi tạo logger:

```
def Init_Logger(feature_name):
    global LOGGER
    today = datetime.now().strftime("%Y-%m-%d")
    log_dir = os.path.join(BASE_REPORT_DIR, "logs", today)
    os.makedirs(log_dir, exist_ok=True)
    logger = logging.getLogger(feature_name)
    logger.setLevel(logging.INFO)
    log_file = os.path.join(log_dir, f"{feature_name}.log")
    if not logger.handlers:
```



```
file_handler = logging.FileHandler(log_file, encoding="utf-8")
formatter = logging.Formatter(
    "%(asctime)s - %(levelname)s - %(message)s"
)
file_handler.setFormatter(formatter)
logger.addHandler(file_handler)
LOGGER = logger
return logger
```

Framework cũng xây dựng keyword logging dùng trực tiếp trong Robot Framework:

```
def log_info(*args):
    message = " ".join(_safe_to_string(a) for a in args)
    # 1. Ghi file
    LOGGER.info(message)
    # 2. Ghi vào Robot (=> log.html + Allure)
    robot_logger.info(message)
```

Các log này được hiển thị trong:

- File log hệ thống.
- log.html của Robot Framework.
- Allure Report.

b) Module tích hợp Allure Report

Framework tích hợp Allure Report nhằm trực quan hóa kết quả kiểm thử.

File triển khai: `utils/allure_helper.py`

Module hỗ trợ:

- Tạo step hiển thị trên Allure.
- Đính kèm screenshot lỗi.
- Theo dõi flow thực thi testcase.

Đoạn mã tạo step log:

```
def step_log(message):  
    with allure.step(message):  
        pass
```

Đoạn mã đính kèm screenshot:

```
def attach_screenshot(path):  
    with open(path, "rb") as f:  
        allure.attach(  
            f.read(),  
            name="Failure Screenshot",  
            attachment_type=allure.attachment_type.PNG  
        )
```

Việc tích hợp Allure giúp kiểm thử viên dễ dàng theo dõi kết quả automation test và phân tích lỗi trực quan hơn.

c) Module thực thi kiểm thử End-to-End

Framework xây dựng module hỗ trợ thực thi các kịch bản End-to-End được AI sinh ra.

File triển khai: `utils/rune2e.py`

Module sử dụng thư viện `subprocess` để tự động chạy Robot Framework test script.

Đoạn mã thực thi:

```
def run_robot_test(file_path):  
    print(f"\n Running: {file_path}")  
    result = subprocess.run(  
        ["robot", file_path],  
        text=True  
    )  
    return result.returncode == 0
```

3.7.4 Vai trò của AI và kiểm thử viên trong framework

Trong framework kiểm thử tự động được xây dựng trong đề tài, AI được sử dụng như một công cụ hỗ trợ sinh keyword và flow kiểm thử dựa trên mô tả nghiệp vụ đầu vào. Tuy nhiên, AI không thay thế hoàn toàn vai trò của kiểm thử viên mà chỉ hỗ trợ tự động hóa một phần quá trình thiết kế kiểm thử.

❖ Vai trò của AI

Trong hệ thống, AI đảm nhiệm các chức năng hỗ trợ như:

- Phân tích mô tả chức năng đầu vào.
- Sinh danh sách keyword theo cấu trúc Keyword-Driven.
- Hỗ trợ sinh flow kiểm thử tự động.
- Tái sử dụng keyword đã tồn tại trong framework.
- Hỗ trợ chuẩn hóa format test flow Robot Framework.
- Giảm thời gian xây dựng test script thủ công.

AI hoạt động dựa trên:

- Prompt Engineering.
- Keyword repository hiện có trong framework.
- Các ràng buộc do hệ thống định nghĩa.

Mô hình AI chỉ sinh output dưới dạng văn bản theo format yêu cầu, không trực tiếp thao tác với framework hoặc thực hiện kiểm thử tự động.

❖ Vai trò của kiểm thử viên

Kiểm thử viên vẫn giữ vai trò trung tâm trong toàn bộ framework, bao gồm:

- Thiết kế kiến trúc framework kiểm thử tự động.
- Xây dựng và tổ chức keyword repository.
- Thiết kế logic nghiệp vụ kiểm thử.

- Kiểm soát luồng kiểm thử End-to-End.
- Kiểm tra và hiệu chỉnh output do AI sinh ra.
- Bổ sung các assertion và logic kiểm thử đặc biệt.
- Quản lý locator, dữ liệu kiểm thử và cấu trúc framework.
- Bảo trì và mở rộng framework khi hệ thống thay đổi.

Ngoài ra, kiểm thử viên còn chịu trách nhiệm:

- Đánh giá tính đúng đắn của flow kiểm thử.
- Loại bỏ các keyword không phù hợp.
- Điều chỉnh prompt để cải thiện chất lượng output AI.
- Đảm bảo tính ổn định và khả năng tái sử dụng của framework.

Trong framework được xây dựng, AI đóng vai trò hỗ trợ sinh keyword và flow kiểm thử, trong khi kiểm thử viên vẫn là người kiểm soát chính đối với:

- Logic nghiệp vụ.
- Kiến trúc framework.
- Chất lượng kiểm thử.
- Tính chính xác của test script được sinh ra.

Cách tiếp cận này giúp tận dụng khả năng hỗ trợ của AI để giảm thời gian xây dựng kiểm thử, đồng thời vẫn đảm bảo sự kiểm soát và tính chính xác từ phía kiểm thử viên.

CHƯƠNG 4: TRIỂN KHAI VÀ THỰC THI KIỂM THỬ TỰ ĐỘNG

4.1 Giới thiệu chung

Trong đồ án này, hệ thống được lựa chọn để kiểm thử là một ứng dụng Web thương mại điện tử với các chức năng cơ bản như đăng ký tài khoản, đăng nhập, tìm kiếm sản phẩm, xem chi tiết sản phẩm, quản lý giỏ hàng và thanh toán. Hệ thống cho phép người dùng truy cập thông qua trình duyệt Web và thực hiện các thao tác mua sắm trực tuyến.

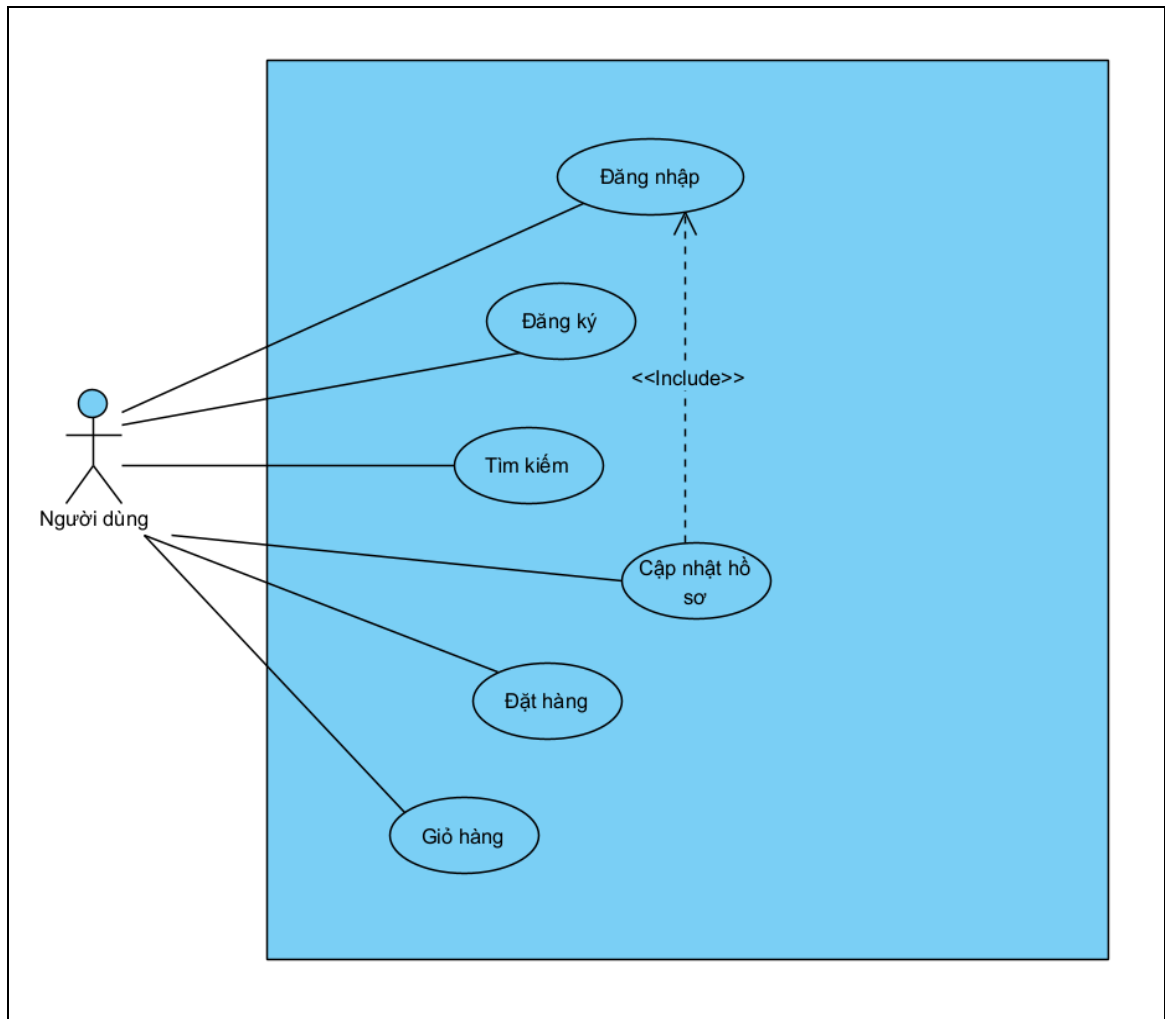
Website được xây dựng theo mô hình client-server, trong đó người dùng tương tác với giao diện Web trên trình duyệt, các yêu cầu được gửi đến máy chủ để xử lý và trả về kết quả hiển thị. Dữ liệu hệ thống bao gồm thông tin người dùng, sản phẩm, giỏ hàng và đơn hàng được lưu trữ và quản lý tập trung.

Việc đặc tả hệ thống website là cơ sở quan trọng để xây dựng các kịch bản kiểm thử và triển khai nền tảng kiểm thử tự động theo hướng Keyword-Driven sử dụng Robot Framework. Các chức năng của hệ thống sẽ được phân tích, từ đó xác định phạm vi kiểm thử và xây dựng các test case phù hợp.

Chương này trình bày các yêu cầu chức năng, yêu cầu phi chức năng, tính toàn vẹn dữ liệu và tính ứng dụng của hệ thống website được lựa chọn để kiểm thử.

4.2 Đặc tả hệ thống phần mềm

4.2.1 Các yêu cầu chức năng



Hình 4.1 Use case tổng quát các yêu cầu chức năng

Bảng 4.1 Danh sách yêu cầu chức năng

STT	Tên chức năng	Mô tả
1	Đăng ký tài khoản	– Người dùng có thể tạo tài khoản mới. – Nhập email, mật khẩu và xác nhận mật khẩu. – Kiểm tra dữ liệu hợp lệ trước khi tạo tài khoản. – Hiển thị thông báo đăng ký thành công hoặc thất bại.

STT	Tên chức năng	Mô tả
2	Đăng nhập	– Người dùng đăng nhập bằng email và mật khẩu. – Hệ thống kiểm tra thông tin xác thực. – Chuyển hướng đến trang chủ sau khi đăng nhập thành công. – Hiển thị thông báo lỗi khi đăng nhập thất bại.
3	Tìm kiếm sản phẩm	– Người dùng nhập từ khóa tìm kiếm. – Hệ thống hiển thị danh sách sản phẩm phù hợp. – Không cho phép tìm kiếm với từ khóa rỗng. – Hiển thị danh sách bất kỳ khi không tìm thấy sản phẩm.
4	Đặt hàng	– Người dùng tiến hành checkout. – Nhập thông tin giao hàng. – Xác nhận đơn hàng. – Hệ thống tạo đơn hàng thành công.
5	Cập nhật hồ sơ	– Người dùng đăng nhập tài khoản, chọn Thông tin tài khoản. – Thay đổi thông tin hồ sơ – Hệ thống ghi nhận và cập nhật hồ sơ.
6	Giỏ hàng	– Người dùng cập nhật số lượng, phân loại sản phẩm. – Xóa sản phẩm khỏi giỏ hàng

Tính toàn vẹn dữ liệu nhằm đảm bảo các dữ liệu nhập vào hệ thống là hợp lệ, đầy đủ và nhất quán. Các quy tắc này là cơ sở để xây dựng các kịch bản kiểm thử tự động theo hướng Data-Driven trong Robot Framework. Hệ thống áp dụng các ràng buộc dữ liệu như sau.

a, Đăng ký tài khoản

Đăng ký
Đóng [x]

Tạo mới một tài khoản cho phép bạn truy cập và sử dụng các dịch vụ trên website này. Nếu bạn đã có tài khoản, bạn hãy [đăng nhập tại đây](#)

Email

Mật khẩu

Xác nhận mật khẩu

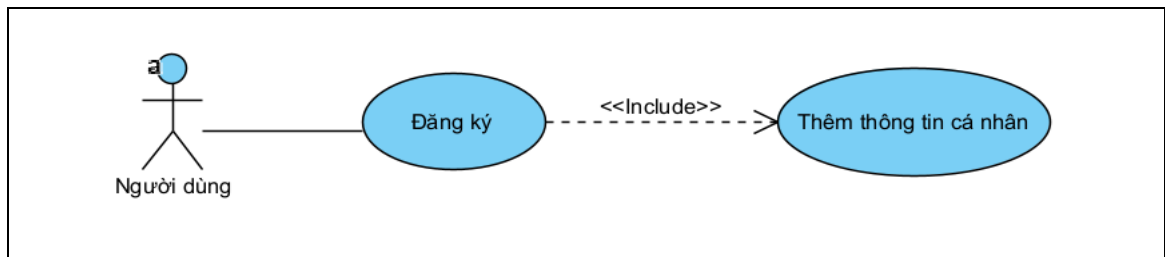
Đăng ký

Hình 4.2 Giao diện Đăng ký tài khoản

Bảng 4.2 Đặc tả kiểm tra dữ liệu chức năng Đăng ký

Tên trường	Bắt buộc	Khoảng hợp lệ	Thông báo lỗi mong đợi
Email	Có	Định dạng {...}@gmail.{...}	- Vui lòng điền vào trường này. - Email đã tồn tại. - Vui lòng nhập phần đứng trước/sau '@'. '@' không hoàn chỉnh. - Email không đúng định dạng.
Mật khẩu	Có	Tối thiểu 6 ký tự	Vui lòng điền vào trường này.

Tên trường	Bắt buộc	Khoảng hợp lệ	Thông báo lỗi mong đợi
			– Mật khẩu tối thiểu phải có 6 ký tự.
Xác nhận mật khẩu	Có	- Tối thiểu 6 ký tự - Trùng khớp với trường Mật khẩu.	– Vui lòng điền vào trường này – Xác nhận mật khẩu tối thiểu phải có 6 ký tự. – Xác nhận mật khẩu không khớp.



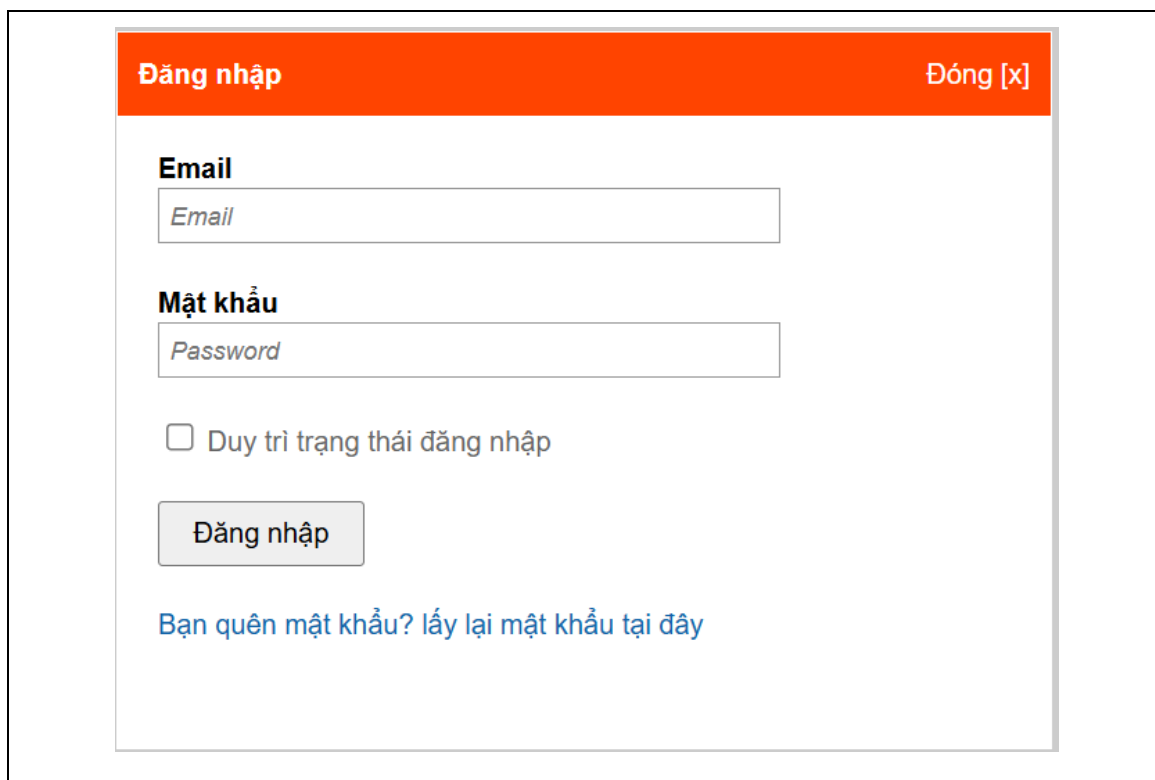
Hình 4.3 Use case phân rã chức năng Đăng ký

UC 1.1 – Đăng ký

Use Case ID	UC 1.1
Use Case Name	Đăng ký tài khoản
UC description	Là người dùng mới, tôi muốn tạo tài khoản để sử dụng tất cả chức năng của trang web.
Actor	Người dùng chưa có tài khoản
Priority	Must have
Trigger	Người dùng muốn tạo tài khoản mới trên hệ thống.
Pre-condition	Thiết bị đã kết nối internet. Người dùng chưa có tài khoản.

Post-condition	• Tài khoản mới được tạo thành công. • Người dùng được chuyển đến trang xác nhận hoặc đăng nhập.
Basic flow	1. Người dùng truy cập trang đăng ký. 2. Hệ thống hiển thị form đăng ký. 3. Người dùng điền đầy đủ thông tin: Email, Mật khẩu, Xác nhận mật khẩu. 4. Người dùng nhấn nút Submit/Đăng ký. 5. Hệ thống xác thực dữ liệu hợp lệ và tạo tài khoản thành công.
Alternative flow	3b. Người dùng nhấn ‘Đăng nhập tại đây’ → điều hướng đến trang đăng nhập (UC 2.1).
Exception flow	5a. Email đã tồn tại → hiển thị TB ‘Email already exists’. 5b. Email sai định dạng → hiển thị TB ‘Email không hợp lệ’. 5c. Mật khẩu quá ngắn → hiển thị TB yêu cầu độ dài tối thiểu. 5d. Bỏ trống trường bắt buộc → hiển thị cảnh báo từng trường.
Business rules	Mỗi email chỉ được đăng ký một tài khoản duy nhất.
Non-Functional requirements	NFR1.1: Quá trình tạo tài khoản hoàn tất trong dưới 5 giây.

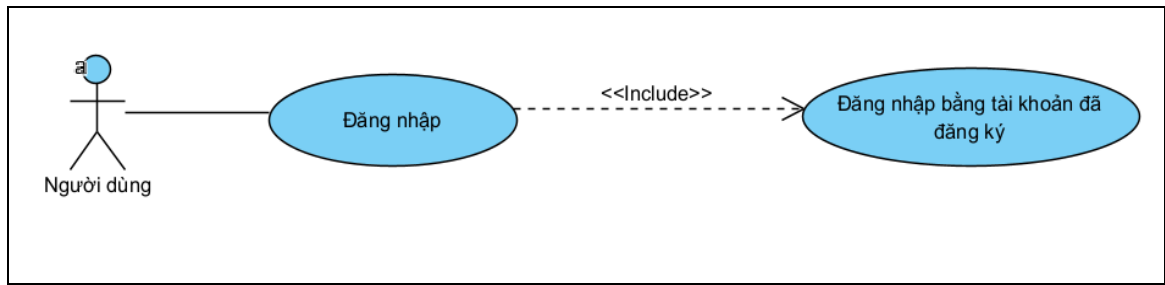
b, Đăng nhập



Hình 4.4 Giao diện Đăng nhập

Bảng 4.3 Đặc tả kiểm tra dữ liệu chức năng Đăng nhập

Tên trường	Bắt buộc	Khoảng hợp lệ	Thông báo lỗi mong đợi
Email	Có	Định dạng {...}@gmail.{...}	- Vui lòng điền vào trường này. - Tài khoản hoặc mật khẩu không chính xác.
Mật khẩu	Có	Tối thiểu 6 ký tự	- Vui lòng điền vào trường này. - Tài khoản hoặc mật khẩu không chính xác.



Hình 4.5 Use case phân rã chức năng Đăng nhập

UC 2.1 – Đăng nhập

Use Case ID	UC 2.1
Use Case Name	Đăng nhập
UC description	Là người dùng, tôi muốn đăng nhập vào hệ thống để sử dụng đầy đủ các tính năng của website.
Actor	Người dùng đã đăng ký
Priority	Must have
Trigger	Người dùng muốn đăng nhập vào hệ thống.
Pre-condition	Thiết bị đã kết nối internet. Người dùng đã có tài khoản đăng ký trước đó.
Post-condition	• Người dùng đăng nhập thành công, chuyển đến trang chủ. • Hệ thống ghi nhận phiên đăng nhập thành công.
Basic flow	1. Người dùng truy cập trang đăng nhập. 2. Hệ thống hiển thị form với các trường Email và Mật khẩu. 3. Người dùng nhập Email và Mật khẩu hợp lệ, nhấn nút Đăng nhập. 4. Hệ thống xác thực thông tin đăng nhập thành công.

	5. Hệ thống chuyển hướng người dùng đến trang chủ.
Alternative flow	3a. Người dùng nhấn ‘Quên mật khẩu?’ → điều hướng đến trang đặt lại mật khẩu. 3b. Người dùng nhấn ‘Sign Up’ → điều hướng đến trang tạo tài khoản (UC 1.1).
Exception flow	4a. Email hoặc mật khẩu sai → hiển thị thông báo lỗi, người dùng nhập lại. 4b. Để trống Email hoặc Mật khẩu → hiển thị cảnh báo ‘Trường này là bắt buộc’.
Business rules	N/A
Non-Functional requirements	NFR2.1: Thời gian xác thực đăng nhập dưới 3 giây. NFR2.2: Sau 5 lần đăng nhập sai liên tiếp, hiển thị cảnh báo bảo mật.

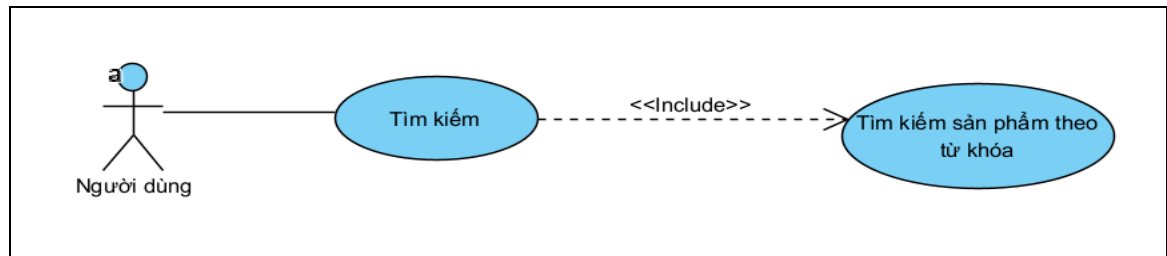
c, Tìm kiếm sản phẩm



Hình 4.6 Giao diện Tìm kiếm sản phẩm

Bảng 4.4 Đặc tả kiểm tra dữ liệu chức năng Tìm kiếm sản phẩm

Tên trường	Bắt buộc	Khoảng hợp lệ	Thông báo lỗi mong đợi
Tìm kiếm sản phẩm	Có	Tối thiểu 1 ký tự	Vui lòng điền vào trường này.



Hình 4.7 Use case phân rã chức năng Tìm kiếm

UC 3.1 – Tìm kiếm sản phẩm

Use Case ID	UC 3.1
Use Case Name	Tìm kiếm sản phẩm
UC description	Là người dùng, tôi muốn tìm kiếm sản phẩm nhanh chóng.
Actor	Người dùng
Priority	Should have
Trigger	Người dùng muốn tìm một sản phẩm cụ thể.
Pre-condition	Thiết bị đã kết nối internet.
Post-condition	- Danh sách kết quả phù hợp với từ khóa được hiển thị. - Người dùng có thể nhấn vào kết quả để đến trang chi tiết.

Basic flow	1. Người dùng nhập từ khóa tìm kiếm và nhấn vào biểu tượng tìm kiếm. 2. Hệ thống tìm kiếm và hiển thị danh sách kết quả liên quan. 3. Người dùng nhấn vào kết quả để đến trang nội dung tương ứng.
Alternative flow	N/A
Exception flow	2a. Bỏ trống ô tìm kiếm → không thực hiện tìm kiếm hoặc hiển thị cảnh báo hoặc trả về danh sách mặc định. 2b. Không có kết quả phù hợp → hiển thị thông báo 'No results found' hoặc hiển thị danh sách sản phẩm mặc định hoặc danh sách sản phẩm gợi ý khi không tìm thấy kết quả phù hợp. 2c. Nhập ký tự đặc biệt/script → hệ thống xử lý an toàn, không lỗi bảo mật, trả về danh sách kết quả mặc định.
Business rules	Tìm kiếm không phân biệt chữ hoa/thường.
Non-Functional requirements	NFR3.1: Kết quả tìm kiếm hiển thị trong ≤ 3 giây.

d, Đặt hàng

THÔNG TIN KHÁCH HÀNG

Họ và tên

Điện thoại *

Email

Địa chỉ

Ghi chú

 **GỬI ĐƠN HÀNG**

HÌNH THỨC THANH TOÁN

☒ Thanh toán khi nhận hàng (COD)

☐ Thanh toán trực tiếp tại cửa hàng

☐ Thanh toán trực tuyến qua ngân hàng

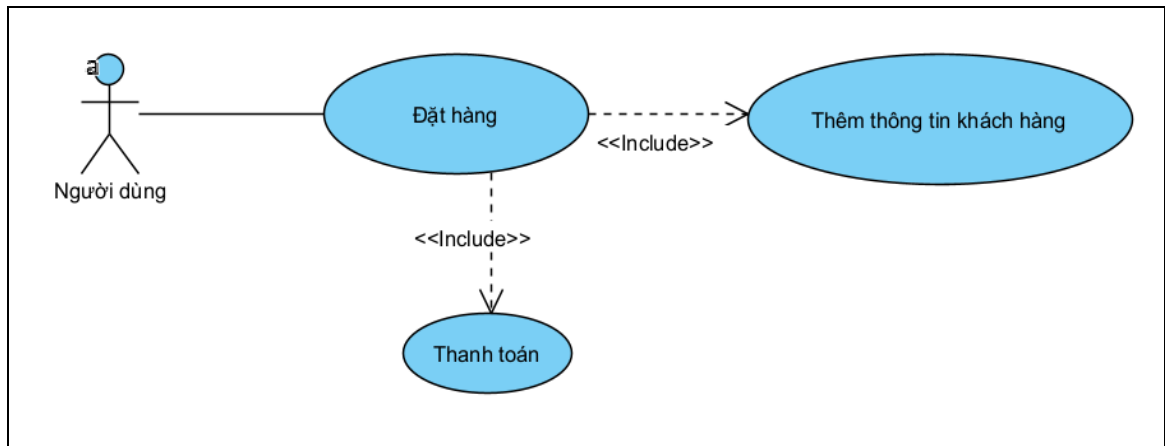
 7 ngày đổi hàng

 Freeship từ 2 sản phẩm

Hình 4.8 Giao diện Đặt hàng

Bảng 4.5 Đặc tả kiểm tra dữ liệu chức năng Đặt hàng

Tên trường	Bắt buộc	Khoảng hợp lệ	Thông báo lỗi mong đợi
Họ và tên	Không	Tối thiểu 0 ký tự Tối đa 255 ký tự	
Điện thoại	Có	Phải là 10 số Bắt đầu bằng số 0	– Vui lòng điền vào trường này. – Vui lòng nhập ít nhất một số điện thoại bạn đang sử dụng.
Email	Không	Định dạng {...}@gmail.{...}	– Vui lòng điền vào trường này. – Email đã tồn tại. – Vui lòng nhập phần đứng trước/sau '@'. '@' không hoàn chỉnh. – Email không đúng định dạng.
Địa chỉ	Có	Không bỏ trống Địa chỉ chính xác	– Vui lòng nhập địa chỉ chính xác. – Vui lòng điền vào trường này
Ghi chú	Không	Tối thiểu 0 ký tự Tối đa 255 ký tự	



Hình 4.9 Use case phân rã chức năng Đặt hàng

UC 4.1 – Đặt hàng

Use Case ID	UC 4.1
Use Case Name	Đặt hàng
UC description	Là người dùng, tôi muốn đặt sản phẩm từ giỏ hàng.
Actor	Người dùng
Priority	Should have
Trigger	Người dùng muốn mua một sản phẩm.
Pre-condition	Thiết bị đã kết nối internet. Sản phẩm tồn tại.
Post-condition	- Đặt hàng thành công hệ thống chuyển hướng đến trang xác nhận đặt hàng. - Hệ thống ghi nhận đơn hàng.

Basic flow	1. Người dùng chọn sản phẩm cần mua và nhấn ‘Mua ngay’ hoặc ‘Thêm vào giỏ hàng’. 2. Người dùng nhập thông tin khách hàng: họ tên, email, số điện thoại, địa chỉ, ghi chú. 3. Người dùng nhấn ‘Gửi đơn hàng’ và kiểm tra kết quả.
Alternative flow	N/A
Exception flow	3a. Bỏ trống trường Số điện thoại, Địa chỉ → Hiển thị cảnh báo hoặc thông báo ‘Vui lòng điền vào trường này.’ 3b. Nhập ký tự đặc biệt, ký tự chữ, ký tự dài hơn 10 số hoặc bé hơn 10 số → Hiển thị thông báo ‘Vui lòng nhập một số điện thoại đang sử dụng.’ 3c. Nhập sai định dạng email → Hiển thị cảnh báo “Vui lòng”
Business rules	Người dùng chỉ có thể đặt hàng nếu số lượng sản phẩm trong kho lớn hơn hoặc bằng số lượng yêu cầu.
Non-Functional requirements	NFR4.1: Kết quả đặt hàng hiển thị trong ≤ 3 giây.

e, Cập nhật hồ sơ

Trang chủ / Tài khoản

THÔNG TIN CHUNG

[Tài khoản](#)

[Thay đổi mật khẩu](#)

[Đăng xuất](#)

Tổng quan về tài khoản

E-mail: ngvanhoan@gmail.com

Mã tài khoản: **67685**

Loại tài khoản: Subscriber

Họ và tên:

Điện thoại:

Địa chỉ:

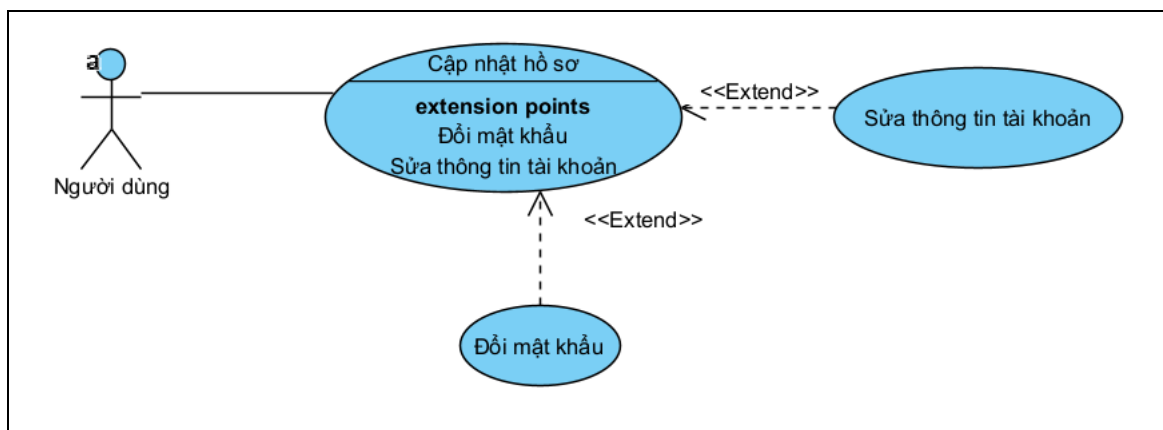
Ngày đăng ký: 08:52:40 20/03/2026

[Cập nhật](#)

Hình 4.10 Giao diện Cập nhật hồ sơ

Bảng 4.6 Đặc tả kiểm tra dữ liệu chức năng Cập nhật hồ sơ

Tên trường	Bắt buộc	Khoảng hợp lệ	Thông báo lỗi mong đợi
Họ và tên	Không	Tối thiểu 0 ký tự Tối đa 255 ký tự	
Điện thoại	Không	Tối thiểu 0 ký tự Tối đa 10 ký tự số Bắt đầu bằng 0	Vui lòng nhập ít nhất một số điện thoại đang sử dụng
Địa chỉ	Không	Tối thiểu 0 ký tự Tối đa 255 ký tự	Địa chỉ không chính xác.



Hình 4.11 Use case phân rã chức năng Cập nhật hồ sơ

UC 5.1 – Cập nhật hồ sơ

Use Case ID	UC 5.1
Use Case Name	Cập nhật hồ sơ
UC description	Là người dùng, tôi muốn cập nhật lại thông tin cá nhân.

Actor	Người dùng
Priority	Should have
Trigger	Người dùng muốn thay đổi thông tin cá nhân.
Pre-condition	Thiết bị đã kết nối internet. Người dùng đã đăng nhập tài khoản.
Post-condition	• Cập nhật thành công. • Hệ thống ghi nhận thay đổi.
Basic flow	1. Người dùng chọn “Tài khoản”. 2. Người dùng nhập thông tin thay đổi: họ tên, số điện thoại, địa chỉ. 3. Người dùng nhấn ‘Cập nhật’ và kiểm tra kết quả.
Alternative flow	N/A
Exception flow	3a. Nhập ký tự đặc biệt, ký tự chữ, ký tự dài hơn 10 số hoặc bé hơn 10 số → Hiển thị thông báo ‘Vui lòng nhập ít nhất một số điện thoại đang sử dụng.’
Business rules	Người dùng chỉ có thể cập nhật thông tin khi đã đăng nhập tài khoản.
Non-Functional requirements	NFR5.1: Kết quả cập nhật hiển thị trong ≤ 3 giây.

4.2.2 Các yêu cầu phi chức năng

Bảng 4.7 Danh sách yêu cầu phi chức năng

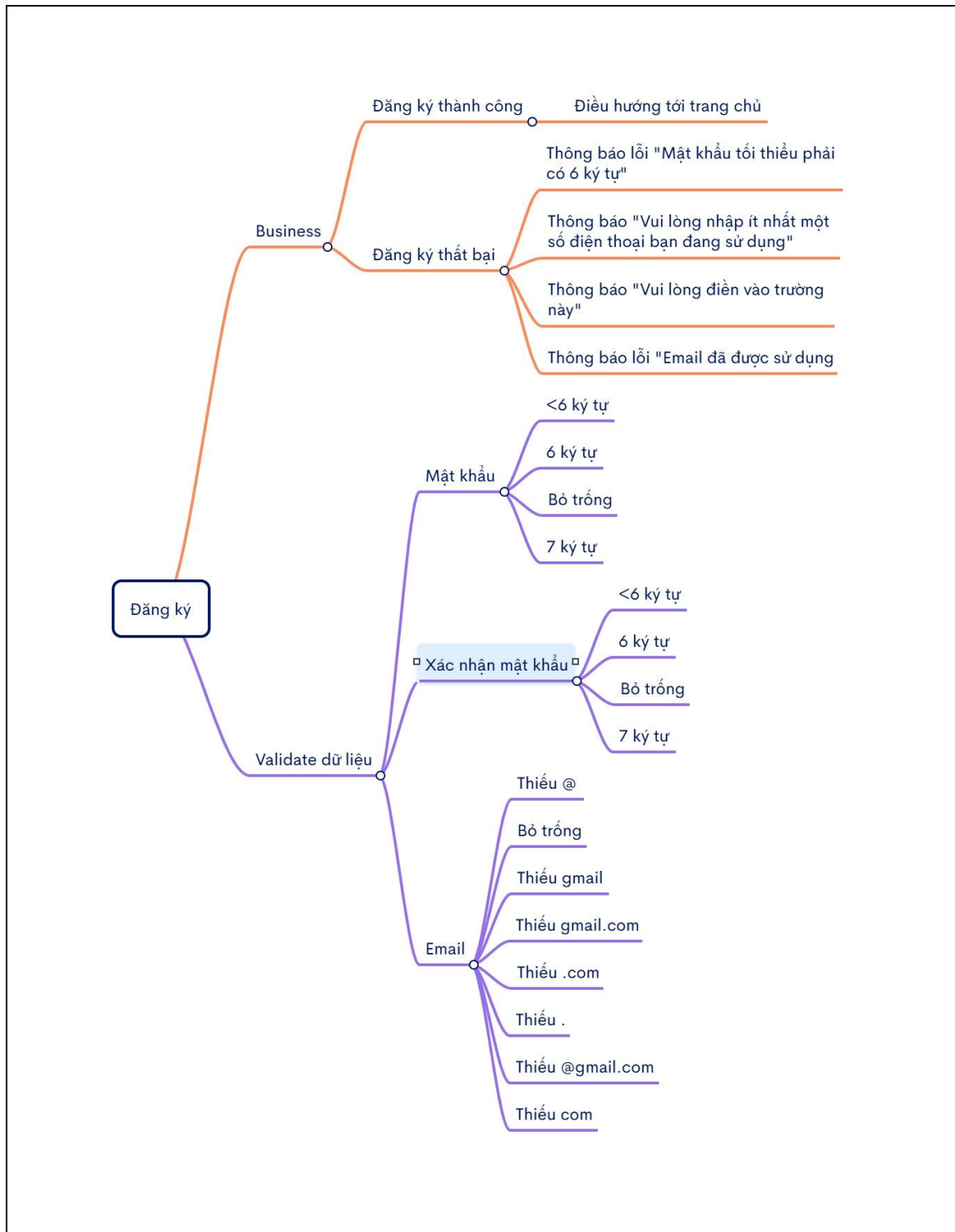
STT	Tên yêu cầu	Mô tả
1	Hiệu năng hệ thống	- Hệ thống phải phản hồi yêu cầu người dùng trong thời gian ngắn. - Thời gian tải trang không quá 3 giây. - Tìm kiếm sản phẩm không quá 2 giây. - Hệ thống hoạt động ổn định khi nhiều người dùng truy cập.
2	Khả năng sử dụng	- Giao diện thân thiện với người dùng. - Dễ dàng thao tác tìm kiếm sản phẩm. - Quy trình mua hàng đơn giản. - Hiển thị thông báo rõ ràng.
3	Khả năng tương thích	- Hỗ trợ các trình duyệt phổ biến: Chrome, Firefox, Edge. - Hỗ trợ nhiều độ phân giải màn hình. - Hoạt động trên hệ điều hành Windows. - Tương thích trên nhiều thiết bị: máy tính, laptop, điện thoại thông minh, máy tính bảng,...
4	Bảo mật	- Thông tin đăng nhập được bảo vệ. - Không hiển thị mật khẩu người dùng. - Kiểm tra dữ liệu đầu vào. - Ngăn chặn truy cập trái phép.

STT	Tên yêu cầu	Mô tả
5	Độ tin cậy	– Luôn hoạt động ổn định, không lỗi hoặc dừng đột ngột trong khi sử dụng. – Thông tin hiển thị chính xác. – Có thể phục vụ lượng người dùng lớn. – Giá tiền, khuyến mãi, số lượng giỏ hàng,... luôn được cập nhật đúng theo hành vi người dùng.

Trong phạm vi đề án, framework tập trung kiểm thử chức năng giao diện Web và một số kiểm tra phản hồi cơ bản. Các yêu cầu như chịu tải lớn hoặc kiểm thử hiệu năng chuyên sâu chưa được triển khai trong phạm vi thực nghiệm.

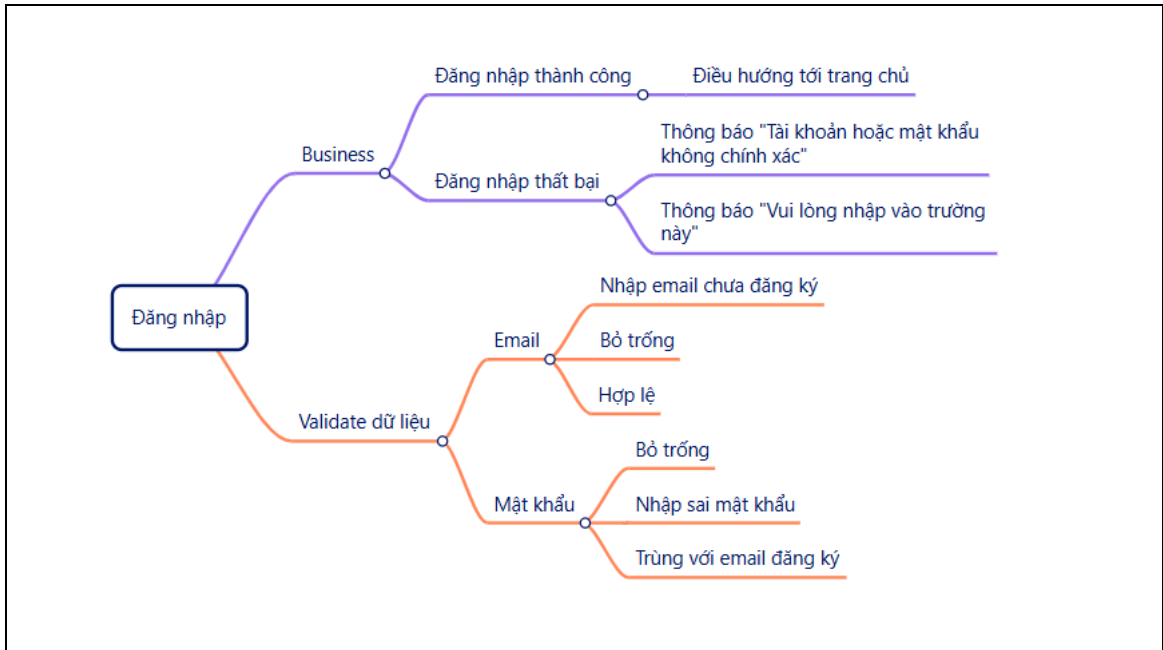
4.3 Thiết kế các yêu cầu kiểm thử

4.3.1 Thiết kế kiểm thử chức năng Đăng ký



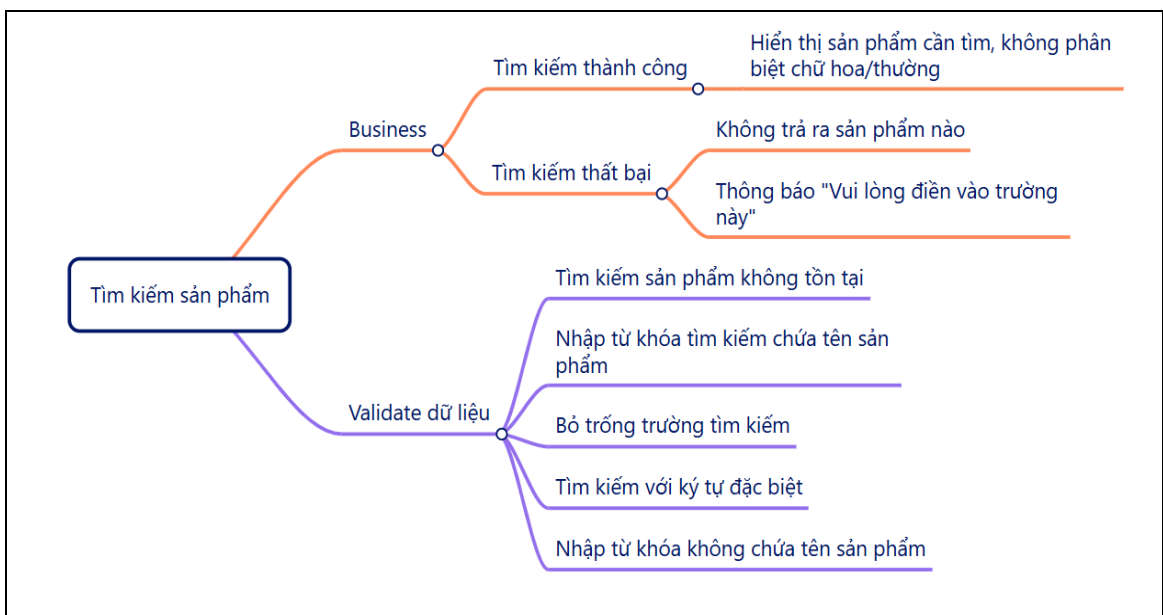
Hình 4.12 Thiết kế kiểm thử chức năng Đăng ký

4.3.2 Thiết kế kiểm thử chức năng Đăng nhập



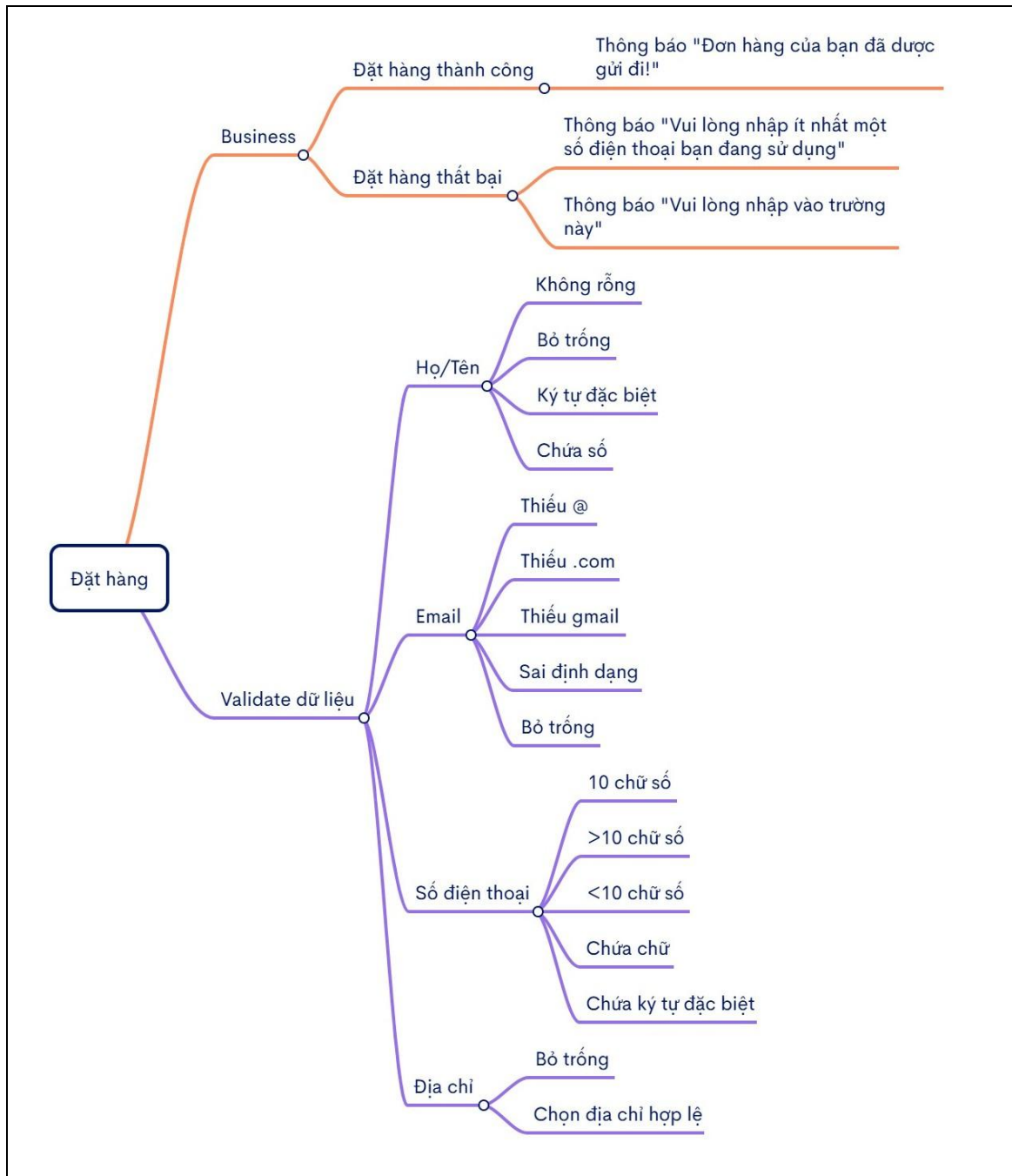
Hình 4.13 Thiết kế kiểm thử chức năng Đăng nhập

4.3.3 Thiết kế kiểm thử chức năng Tìm kiếm



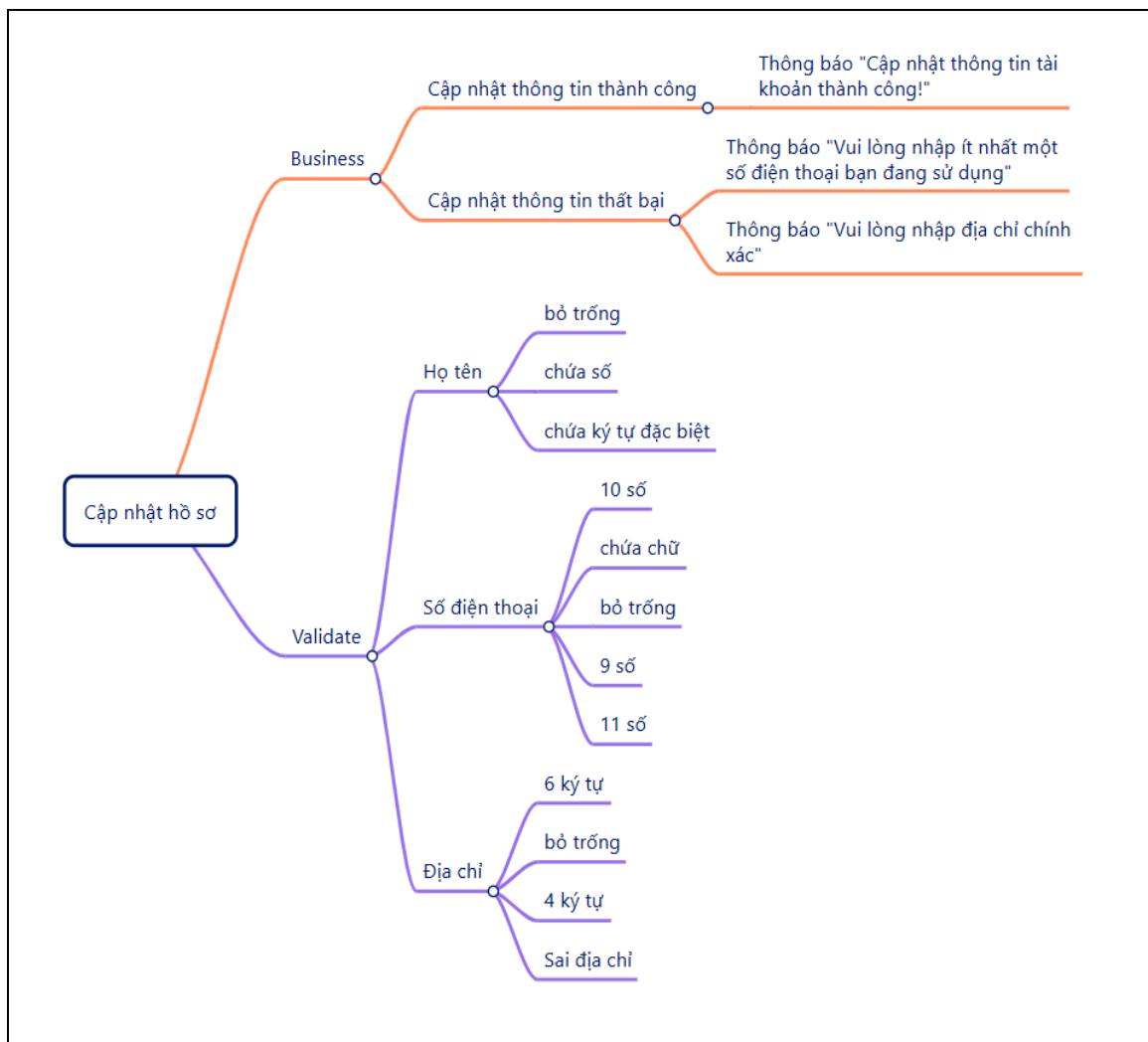
Hình 4.14 Thiết kế kiểm thử chức năng Tìm kiếm

4.3.4 Thiết kế kiểm thử chức năng Đặt hàng



Hình 4.15 Thiết kế kiểm thử chức năng Đặt hàng

4.3.5 Thiết kế kiểm thử chức năng Cập nhật hồ sơ



Hình 4.16 Thiết kế kiểm thử chức năng Cập nhật hồ sơ

4.4 Xây dựng các trường hợp kiểm thử

4.4.1 Chức năng Đăng ký

Bảng 4.8 Các trường hợp kiểm thử chức năng Đăng ký

ID	Test Case Description	Pre -Condition	Test Case Procedure	Expected Output
[DangKy - 01]	Email đã sử dụng	Người dùng chưa có tài khoản để đăng nhập vào hệ thống, đang ở trang đăng ký	B1: Nhập email đã đăng ký trước B2: Nhập mật khẩu và xác nhận mật khẩu B3: Nhấn "Đăng ký"	Thông báo lỗi "Email đã được sử dụng"
[DangKy - 02]	Mật khẩu < 6 ký tự	Người dùng chưa có tài khoản để đăng nhập vào hệ thống, đang ở trang đăng ký	B1: Nhập email, xác nhận mật khẩu hợp lệ B2: Nhập mật khẩu <6 kí tự B3: Nhấn "Đăng ký"	Thông báo lỗi "Mật khẩu tối thiểu phải có 6 ký tự"
[DangKy - 03]	Xác nhận mật khẩu không khớp với mật khẩu	Người dùng chưa có tài khoản để đăng nhập vào hệ thống, đang ở trang đăng ký	B1: Nhập email, mật khẩu B2: Nhập xác nhận mật khẩu khác mật khẩu B3: Nhấn "Đăng ký"	Thông báo lỗi "Xác nhận mật khẩu không khớp với mật khẩu"

ID	Test Case Description	Pre -Condition	Test Case Procedure	Expected Output
[DangKy - 04]	Bỏ trống email	Người dùng chưa có tài khoản để đăng nhập vào hệ thống, đang ở trang đăng ký	B1: Bỏ trống email B2: Nhập mật khẩu, xác nhận mật khẩu hợp lệ B3: Nhấn "Đăng ký"	Thông báo lỗi "Vui lòng điền vào trường này"
[DangKy - 05]	Bỏ trống mật khẩu	Người dùng chưa có tài khoản để đăng nhập vào hệ thống, đang ở trang đăng ký	B1: Nhập email, xác nhận mật khẩu B2: Bỏ trống mật khẩu B3: Nhấn "Đăng ký"	Thông báo lỗi "Vui lòng điền vào trường này"
[DangKy - 06]	Bỏ trống xác nhận mật khẩu	Người dùng chưa có tài khoản để đăng nhập vào hệ thống, đang ở trang đăng ký	B1: Nhập email, mật khẩu B2: Bỏ trống xác nhận mật khẩu B3: Nhấn "Đăng ký"	Thông báo lỗi "Vui lòng điền vào trường này"
[DangKy - 07]	Email thiếu @	Người dùng chưa có tài khoản để đăng nhập vào hệ thống, đang ở trang đăng ký	B1: Nhập email thiếu @ B2: Nhập mật khẩu và xác nhận mật khẩu B3: Nhấn "Đăng ký"	Thông báo "Vui lòng"

ID	Test Case Description	Pre -Condition	Test Case Procedure	Expected Output
[DangKy - 08]	Email thiếu sau @	Người dùng chưa có tài khoản để đăng nhập vào hệ thống, đang ở trang đăng ký	B1: Nhập email thiếu sau @ B2: Nhập mật khẩu và xác nhận mật khẩu B3: Nhấn "Đăng ký"	Thông báo "Vui lòng"
[DangKy - 09]	Email thiếu trước @	Người dùng chưa có tài khoản để đăng nhập vào hệ thống, đang ở trang đăng ký	B1: Nhập email thiếu trước @ B2: Nhập mật khẩu và xác nhận mật khẩu B3: Nhấn "Đăng ký"	Thông báo "Vui lòng"
[DangKy - 10]	Vị trí dấu '.' sai	Người dùng chưa có tài khoản để đăng nhập vào hệ thống, đang ở trang đăng ký	B1: Nhập email sai vị trí dấu '.' B2: Nhập mật khẩu và xác nhận mật khẩu B3: Nhấn "Đăng ký"	Thông báo lỗi "'.' bị sử dụng sai vị trí trong '.' "

4.4.2 Chức năng Đăng nhập

Bảng 4.9 Các trường hợp kiểm thử chức năng Đăng nhập

ID	Test Case Description	Pre -Condition	Test Case Procedure	Expected Output
[DangNhap - 01]	Nhập email, sai mật khẩu	Người dùng đã có tài khoản, chưa đăng nhập và đang ở trang đăng nhập.	B1: Nhập Email hợp lệ B2: Nhập password B3: Nhấn "Đăng nhập"	Thông báo lỗi và yêu cầu nhập lại. "Tài khoản hoặc mật khẩu không chính xác"
[DangNhap - 02]	Email sai, password đúng	Người dùng đã có tài khoản, chưa đăng nhập và đang ở trang đăng nhập.	B1: Nhập Email chưa tồn tại B2: Nhập password B3: Nhấn "Đăng nhập"	Thông báo lỗi và yêu cầu nhập lại. "Tài khoản hoặc mật khẩu không chính xác"
[DangNhap - 03]	Bỏ trống mật khẩu	Người dùng đã có tài khoản, chưa đăng nhập và đang ở trang đăng nhập.	B1: Nhập Email: tuyetmai99@gmail.com B2: Trống password B3: Nhấn "Đăng nhập"	Thông báo lỗi và yêu cầu nhập lại. "Vui lòng điền vào trường này"
[DangNhap - 04]	Bỏ trống email	Người dùng đã có tài khoản, chưa đăng nhập và đang ở trang đăng nhập.	B1: Bỏ trống Email B2: Nhập password B3: Nhấn "Đăng nhập"	Thông báo lỗi và yêu cầu nhập lại. "Vui lòng điền vào trường này"

ID	Test Case Description	Pre -Condition	Test Case Procedure	Expected Output
[DangNhap - 05]	Email, mật khẩu đúng	Người dùng đã có tài khoản, chưa đăng nhập và đang ở trang đăng nhập.	B1: Nhập Email hợp lệ B2: Nhập password hợp lệ B3: Nhấn "Đăng nhập"	Chuyển hướng đến trang chủ Cập nhật tên tài khoản

4.4.3 Chức năng Tìm kiếm

Bảng 4.10 Các trường hợp kiểm thử chức năng Tìm kiếm

ID	Test Case Description	Pre -Condition	Test Case Procedure	Expected Output
[TimKiemSP - 01]	Tìm kiếm sản phẩm theo từ khóa một phần	Người dùng đã truy cập vào trang web	B1: Click vào thanh tìm kiếm B2: Nhập từ khóa 1 phần B3: Nhấn Enter hoặc biểu tượng tìm kiếm	Hệ thống hiện danh sách sản phẩm chứa từ khóa
[TimKiemSP - 02]	Tìm kiếm sản phẩm theo mã sản phẩm	Người dùng đã truy cập vào trang web	B1: Click vào thanh tìm kiếm B2: Nhập: ELA301 B3: Nhấn Enter hoặc biểu tượng tìm kiếm	Hệ thống hiện sản phẩm chứa từ khóa
[TimKiemSP - 03]	Tìm kiếm sản phẩm với tên đầy đủ	Người dùng đã truy cập vào trang web	B1: Click vào thanh tìm kiếm B2: Nhập tên đầy đủ B3: Nhấn Enter hoặc biểu tượng tìm kiếm	Hiện thị sản phẩm có tên chuẩn

ID	Test Case Description	Pre -Condition	Test Case Procedure	Expected Output
[TimKiemSP - 04]	Tìm kiếm với ký tự đặc biệt	Người dùng đã truy cập vào trang web	B1: Click vào thanh tìm kiếm B2: Nhập ký hiệu đặc biệt B3: Nhấn Enter hoặc biểu tượng tìm kiếm	Hệ thống hiện danh sách sản phẩm ngẫu nhiên
[TimKiemSP - 05]	Bỏ trống	Người dùng đã truy cập vào trang web	B1: Click vào thanh tìm kiếm B2: Bỏ trống B3: Nhấn Enter hoặc biểu tượng tìm kiếm	Hệ thống hiện thông báo "Vui lòng điền vào trường này"
[TimKiemSP - 06]	Tìm kiếm với từ khóa không có trong sản phẩm	Người dùng đã truy cập vào trang web	B1: Click vào thanh tìm kiếm B2: Nhập từ không có trong sản phẩm B3: Nhấn Enter hoặc biểu tượng tìm kiếm	Hệ thống không hiện sản phẩm nào

4.4.4 Chức năng Đặt hàng

Bảng 4.11 Các trường hợp kiểm thử chức năng Đặt hàng

ID	Test Case Description	Pre -Condition	Test Case Procedure	Expected Output
[ThanhToan - 01]	Đặt hàng thành công	Người dùng truy cập vào trang web và thêm sản phẩm từ trang chủ vào giỏ hàng	B1: Nhập họ tên, số điện thoại, email, địa chỉ, ghi chú hợp lệ B2: Nhấn "Thanh toán"	Hệ thống thông báo "Đơn hàng của bạn đã được gửi đi"
[ThanhToan - 02]	Sai số điện thoại <10 số	Người dùng truy cập vào trang web và thêm sản phẩm từ trang chủ vào giỏ hàng	B1: Nhập họ tên, email, địa chỉ, ghi chú hợp lệ B2: Nhập số điện thoại <10 số B3: Nhấn "Thanh toán"	Hệ thống thông báo "Vui lòng nhập ít nhất một số điện thoại bạn đang sử dụng"
[ThanhToan - 03]	Sai số điện thoại >10 số	Người dùng truy cập vào trang web và thêm sản phẩm từ	B1: Nhập họ tên, email, địa chỉ, ghi chú hợp lệ B2: Nhập số điện thoại >10 số B3: Nhấn "Thanh toán"	Hệ thống thông báo "Vui lòng nhập ít nhất một số điện thoại bạn đang sử dụng"

ID	Test Case Description	Pre -Condition	Test Case Procedure	Expected Output
		trang chủ vào giỏ hàng		
[ThanhToan - 04]	Sai số điện thoại chứa ký tự đặc biệt	Người dùng truy cập vào trang web và thêm sản phẩm từ trang chủ vào giỏ hàng	B1: Nhập họ tên, email, địa chỉ, ghi chú hợp lệ B2: Nhập số điện thoại chứa ký tự đặc biệt B3: Nhấn "Thanh toán"	Hệ thống thông báo "Vui lòng nhập ít nhất một số điện thoại bạn đang sử dụng"
[ThanhToan - 05]	Sai số điện thoại chứa chữ	Người dùng truy cập vào trang web và thêm sản phẩm từ trang chủ vào giỏ hàng	B1: Nhập họ tên, email, địa chỉ, ghi chú hợp lệ B2: Nhập số điện thoại chứa chữ B3: Nhấn "Thanh toán"	Hệ thống thông báo "Vui lòng nhập ít nhất một số điện thoại bạn đang sử dụng"
[ThanhToan - 06]	Trống số điện thoại	Người dùng truy cập vào trang web và thêm sản phẩm từ trang chủ vào giỏ hàng	B1: Nhập họ tên, email, địa chỉ, ghi chú hợp lệ B2: Bỏ trống số điện thoại B3: Nhấn "Thanh toán"	Hệ thống thông báo "Vui lòng điền vào trường này"

ID	Test Case Description	Pre -Condition	Test Case Procedure	Expected Output
[ThanhToan - 07]	Trống địa chỉ	Người dùng truy cập vào trang web và thêm sản phẩm từ trang chủ vào giỏ hàng	B1: Nhập họ tên, số điện thoại, email, ghi chú hợp lệ B2: Bỏ trống địa chỉ B3: Nhấn "Thanh toán"	Hệ thống thông báo "Vui lòng điền vào trường này"
[ThanhToan - 08]	Email thiếu phần sau @	Người dùng truy cập vào trang web và thêm sản phẩm từ trang chủ vào giỏ hàng	B1: Nhập họ tên, số điện thoại, địa chỉ, ghi chú hợp lệ B2: Nhập email thiếu sau @ B3: Nhấn "Thanh toán"	Hệ thống thông báo "Vui lòng nhập phần đứng sau @"
[ThanhToan - 09]	Email thiếu phần trước @	Người dùng truy cập vào trang web và thêm sản phẩm từ trang chủ vào giỏ hàng	B1: Nhập họ tên, số điện thoại, địa chỉ, ghi chú hợp lệ B2: Nhập email thiếu trước @ B3: Nhấn "Thanh toán"	Hệ thống thông báo "Vui lòng nhập phần đứng trước @"

ID	Test Case Description	Pre -Condition	Test Case Procedure	Expected Output
[ThanhToan - 10]	Email thiếu @	Người dùng truy cập vào trang web và thêm sản phẩm từ trang chủ vào giỏ hàng	B1: Nhập họ tên, số điện thoại, địa chỉ, ghi chú hợp lệ B2: Nhập email thiếu @ B3: Nhấn "Thanh toán"	Hệ thống thông báo "Vui lòng nhập bao gồm '@' trong địa chỉ email"
[ThanhToan - 11]	Vị trí dấu '.' sai	Người dùng truy cập vào trang web và thêm sản phẩm từ trang chủ vào giỏ hàng	B1: Nhập họ tên, số điện thoại, địa chỉ, ghi chú hợp lệ B2: Nhập email sai vị trí dấu '.' B3: Nhấn "Thanh toán"	Hệ thống thông báo "'.' bị sai vị trí trong '.'" "

4.4.5 Chức năng Cập nhật hồ sơ

Bảng 4.12 Các trường hợp kiểm thử chức năng Cập nhật hồ sơ

ID	Test Case Description	Pre -Condition	Test Case Procedure	Expected Output
[ChangeProfile - 01]	cập nhật thành công	Người dùng đã đăng nhập vào trang web và đang ở trong thông tin tài khoản	B1: Nhập Họ và tên, số điện thoại, địa chỉ hợp lệ B2: Nhấn Cập nhật	Hệ thống thông báo "Cập nhật thông tin tài khoản thành công"
[ChangeProfile - 02]	Bỏ trống sdt	Người dùng đã đăng nhập vào trang web và đang ở trong thông tin tài khoản	B1: Nhập Họ và tên, địa chỉ hợp lệ B2: Bỏ trống số điện thoại B3: Nhấn Cập nhật	Hệ thống thông báo "Cập nhật thông tin tài khoản thành công"
[ChangeProfile - 03]	Bỏ trống họ tên	Người dùng đã đăng nhập vào trang web và đang ở trong thông tin tài khoản	B1: Nhập số điện thoại, địa chỉ hợp lệ B2: Bỏ trống Họ và tên B3: Nhấn Cập nhật	Hệ thống thông báo "Cập nhật thông tin tài khoản thành công"

ID	Test Case Description	Pre -Condition	Test Case Procedure	Expected Output
[ChangeProfile - 04]	Bỏ trống địa chỉ	Người dùng đã đăng nhập vào trang web và đang ở trong thông tin tài khoản	B1: Nhập Họ và tên, địa chỉ hợp lệ B2: Bỏ trống địa chỉ hợp lệ B3: Nhấn Cập nhật	Hệ thống thông báo "Cập nhật thông tin tài khoản thành công"
[ChangeProfile - 05]	Số điện thoại chứa ký tự đặc biệt	Người dùng đã đăng nhập vào trang web và đang ở trong thông tin tài khoản	B1: Nhập Họ và tên, địa chỉ hợp lệ B2: Nhập số điện thoại chứa ký tự đặc biệt B3: Nhấn Cập nhật	Hệ thống thông báo "Vui lòng nhập ít nhất một số điện thoại bạn đang sử dụng"
[ChangeProfile - 06]	Số điện thoại < 10 số	Người dùng đã đăng nhập vào trang web và đang ở trong thông tin tài khoản	B1: Nhập Họ và tên, địa chỉ hợp lệ B2: Nhập số điện thoại < 10 số B3: Nhấn Cập nhật	Hệ thống thông báo "Vui lòng nhập ít nhất một số điện thoại bạn đang sử dụng"
[ChangeProfile - 07]	Số điện thoại > 10 số	Người dùng đã đăng nhập vào trang web và đang ở trong thông tin tài khoản	B1: Nhập Họ và tên, địa chỉ hợp lệ B2: Nhập số điện thoại >10 số B3: Nhấn Cập nhật	Hệ thống thông báo "Vui lòng nhập ít nhất một số điện thoại bạn đang sử dụng"

ID	Test Case Description	Pre -Condition	Test Case Procedure	Expected Output
[ChangeProfile - 08]	Số điện thoại chứa chữ	Người dùng đã đăng nhập vào trang web và đang ở trong thông tin tài khoản	B1: Nhập Họ và tên, địa chỉ hợp lệ B2: Nhập số điện thoại chữ chữ B3: Nhấn Cập nhật	Hệ thống thông báo "Vui lòng nhập ít nhất một số điện thoại bạn đang sử dụng"

4.4.6 Chức năng Mua hàng hoàn chỉnh

Bảng 4.13 Các trường hợp kiểm thử chức năng Mua hàng hoàn chỉnh

ID	Test Case Description	Pre -Condition	Test Case Procedure	Expected Output
[E2E_ThanhToan - 01]	Đặt hàng thành công	Người dùng đã đăng nhập tài khoản, tìm kiếm và thêm sản phẩm vào giỏ hàng	B1: Nhập họ tên, số điện thoại, email, địa chỉ, ghi chú hợp lệ B2: Nhấn "Thanh toán"	Hệ thống thông báo "Đơn hàng của bạn đã được gửi đi"
[E2E_ThanhToan - 02]	Sai số điện thoại <10 số	Người dùng đã đăng nhập tài khoản, tìm	B1: Nhập họ tên, email, địa chỉ, ghi chú hợp lệ	

ID	Test Case Description	Pre -Condition	Test Case Procedure	Expected Output
		kiểm và thêm sản phẩm vào giỏ hàng	B2: Nhập số điện thoại <10 số B3: Nhấn "Thanh toán"	Hệ thống thông báo "Vui lòng nhập ít nhất một số điện thoại bạn đang sử dụng"
[E2E_ThanhToan - 03]	Sai số điện thoại >10 số	Người dùng đã đăng nhập tài khoản, tìm kiếm và thêm sản phẩm vào giỏ hàng	B1: Nhập họ tên, email, địa chỉ, ghi chú hợp lệ B2: Nhập số điện thoại >10 số B3: Nhấn "Thanh toán"	Hệ thống thông báo "Vui lòng nhập ít nhất một số điện thoại bạn đang sử dụng"
[E2E_ThanhToan - 04]	Sai số điện thoại chứa ký tự đặc biệt	Người dùng đã đăng nhập tài khoản, tìm kiếm và thêm sản phẩm vào giỏ hàng	B1: Nhập họ tên, email, địa chỉ, ghi chú hợp lệ B2: Nhập số điện thoại chứa ký tự đặc biệt B3: Nhấn "Thanh toán"	Hệ thống thông báo "Vui lòng nhập ít nhất một số điện thoại bạn đang sử dụng"
[E2E_ThanhToan - 05]	Sai số điện thoại chứa chữ	Người dùng đã đăng nhập tài khoản, tìm kiếm và thêm sản phẩm vào giỏ hàng	B1: Nhập họ tên, email, địa chỉ, ghi chú hợp lệ B2: Nhập số điện thoại chứa chữ B3: Nhấn "Thanh toán"	Hệ thống thông báo "Vui lòng

ID	Test Case Description	Pre -Condition	Test Case Procedure	Expected Output
				nhập ít nhất một số điện thoại bạn đang sử dụng"
[E2E_ThanhToan - 06]	Trống SDT	Người dùng đã đăng nhập tài khoản, tìm kiếm và thêm sản phẩm vào giỏ hàng	B1: Nhập họ tên, email, địa chỉ, ghi chú hợp lệ B2: Bỏ trống số điện thoại B3: Nhấn "Thanh toán"	Hệ thống thông báo "Vui lòng điền vào trường này"
[E2E_ThanhToan - 07]	Trống địa chỉ	Người dùng đã đăng nhập tài khoản, tìm kiếm và thêm sản phẩm vào giỏ hàng	B1: Nhập họ tên, số điện thoại, email, ghi chú hợp lệ B2: Bỏ trống địa chỉ B3: Nhấn "Thanh toán"	Hệ thống thông báo "Vui lòng điền vào trường này"
[E2E_ThanhToan - 08]	Email thiếu phần sau @	Người dùng đã đăng nhập tài khoản, tìm kiếm và thêm sản phẩm vào giỏ hàng	B1: Nhập họ tên, số điện thoại, địa chỉ, ghi chú hợp lệ B2: Nhập email thiếu sau @ B3: Nhấn "Thanh toán"	Hệ thống thông báo "Vui lòng nhập phần đứng sau @"
[E2E_ThanhToan - 09]	Email thiếu phần trước @	Người dùng đã đăng nhập tài khoản, tìm	B1: Nhập họ tên, số điện thoại, địa chỉ, ghi chú hợp lệ	

ID	Test Case Description	Pre -Condition	Test Case Procedure	Expected Output
		kiểm và thêm sản phẩm vào giỏ hàng	B2: Nhập email thiếu trước @ B3: Nhấn "Thanh toán"	Hệ thống thông báo "Vui lòng nhập phần đứng trước @"
[E2E_ThanhToan - 10]	Email thiếu @	Người dùng đã đăng nhập tài khoản, tìm kiếm và thêm sản phẩm vào giỏ hàng	B1: Nhập họ tên, số điện thoại, địa chỉ, ghi chú hợp lệ B2: Nhập email thiếu @ B3: Nhấn "Thanh toán"	Hệ thống thông báo "Vui lòng nhập bao gồm '@' trong địa chỉ email"
[E2E_ThanhToan - 11]	vị trí dấu '.' sai	Người dùng đã đăng nhập tài khoản, tìm kiếm và thêm sản phẩm vào giỏ hàng	B1: Nhập họ tên, số điện thoại, địa chỉ, ghi chú hợp lệ B2: Nhập email sai vị trí dấu '.' B3: Nhấn "Thanh toán"	Hệ thống thông báo "'.' bị sai vị trí trong '.'"'

4.5 Thực thi sinh keyword và luồng kiểm thử

4.5.1 Chức năng Đăng nhập

Hệ thống nhận đầu vào dưới dạng file mô tả chức năng kiểm thử login.txt được lưu trong thư mục: generate_keywords_use_AI/input/

Framework thực hiện lệnh:

```
python generate_keywords_use_AI/generate_keyword.py login
```

Sau khi xử lý, AI sinh ra danh sách keyword kiểm thử và lưu tại:

```
generate_keywords_use_AI/output/login_keywords_xxx.txt
```

Kết quả output bao gồm:

- Business keyword.
- UI keyword.
- Verify keyword.

```
Here is the output in the required format:

**BUSINESS KEYWORDS**

| Keyword Name | Input Parameters | Required Parameters | Optional Parameters | Description |
|-----|-----|-----|-----|-----|
| Login User | Yes | username, password | None | Perform login action |
| Logout User | Yes | None | None | Perform logout action |

**UI ACTION KEYWORDS**

| Keyword Name | Input Parameters | Parameters | Derived From SeleniumLibrary Keyword | Description |
|-----|-----|-----|-----|-----|
| Input Username | Yes | username | Input Text | Enter username |
| Input Password | Yes | password | Input Text | Enter password |
| Click Login | Yes | None | Click Element | Click login button |

**VERIFICATION KEYWORDS**

| Keyword Name | Input Parameters | Parameters | Derived From SeleniumLibrary Keyword | Description |
|-----|-----|-----|-----|-----|
| Verify Login Success | Yes | None | Page Should Contain | Validate successful login |
| Verify Error Message | Yes | None | Element Should Contain | Validate error message |

Note: I've followed the rules and guidelines provided to generate reusable keywords for the given feature. The output is in the required format.
```

Hình 4.17 Kết quả output AI sinh keyword chức năng Đăng nhập

Sau khi AI sinh keyword, framework tiến hành:

- Parse output AI.

- Normalize keyword.
- Detect capability.
- Kiểm tra duplicate keyword.
- Inject keyword vào framework.

```
(.venv) PS D:\DoAnTotNghiep_NguyenQuyen_10122312> python generate_keywords_use_AI/generate_keyword.py login
Đang generate keyword cho feature: login ...
Generated keywords for: login
Đã lưu kết quả AI vào: generate_keywords_use_AI/output/login_keywords_20260508_213659.txt
Skip semantic duplicate by description: Login User
Skip duplicate name: Logout User
Reuse existing capability (input_text) for: Input Username
Reuse existing capability (input_text) for: Input Password
Reuse existing capability (click_element) for: Click Login
Reuse existing capability (verify_generic) for: Verify Login Success
Reuse existing capability (verify_generic) for: Verify Error Message
Generating business flow...
Saved flow to: generate_keywords_use_AI/output/login_flow_20260508_214543.txt
Robot test generated: tests/login_auto_20260508_214543.robot

===== SUMMARY =====
BUSINESS: No new keywords.
UI: No new keywords.
VERIFY: No new keywords.

Hoàn tất inject keyword vào framework.
```

Hình 4.18 Quá trình inject keyword vào framework chức năng Đăng nhập

Business keyword được inject vào: keywords/business/login_business.robot

UI keyword được inject vào: keywords/ui/common_keywords.robot

Verify keyword được inject vào: keywords/verify/verify.robot

Sau khi inject keyword, framework tiếp tục:

- Load keyword repository.
- Xây dựng prompt sinh flow.
- Gửi prompt tới AI.
- Sinh flow kiểm thử.

Flow kiểm thử được lưu tại: generate_keywords_use_AI/output/login_flow_xxx.txt

Flow được sinh dựa trên keyword repository hiện có trong framework.

Here is the Happy Path Main Business Flow:

****Login to System****

1. ****Login To System**** | \${username}
2. ****Login To System****
3. ****Verify Current URL Should Be**** | "login-success"
4. ****Verify Page Contains Text**** | "Welcome, \${username}!"

Note: The `\${username}` placeholder represents the actual username input by the user.

Hình 4.19 Kết quả AI sinh flow chức năng Đăng nhập

Framework tự động chuyển flow kiểm thử thành test script Robot Framework.

```
*** Test Cases ***
login Auto Test 1
    [Documentation]    Auto generated from AI flow

    Login To System      ngvanhoan@gmail.com    123456789
    Verify Element Text Contains    ${ERROR_MSG}    Tài khoản hoặc mật khẩu không chính xác

login Auto Test 2
    [Documentation]    Auto generated from AI flow

    Login To System      tuyetmai999    tuyetmai99
    Verify Element Text Contains    ${ERROR_MSG}    Tài khoản hoặc mật khẩu không chính xác

login Auto Test 3
    [Documentation]    Auto generated from AI flow

    Login To System      ngvanhoan@gmail.com    ${EMPTY}
    Verify Required Field Message    Please fill out this field.

login Auto Test 4
    [Documentation]    Auto generated from AI flow

    Login To System      ${EMPTY}    123456789
    Verify Required Field Message    Please fill out this field.
```

Hình 4.20 Test script chức năng Đăng nhập

4.5.2 Chức năng Đăng ký

```
Here is the output:

**BUSINESS KEYWORDS**

| Keyword Name | Input Parameters | Required Parameters | Optional Parameters | Description |
|-----|-----|-----|-----|-----|
| Register User | Yes | username, password | None | Create a new user account. |
| Fill Out Registration Form | Yes | None | None | Enter required information for registration. |

**UI ACTION KEYWORDS**

| Keyword Name | Input Parameters | Parameters | Derived From SeleniumLibrary Keyword | Description |
|-----|-----|-----|-----|-----|
| Input Text Into Field | Yes | text, field_name | Input Text | Enter text into a specific field. |
| Click On Element | Yes | element_name | Click Element | Click on an element. |
| Wait For Registration Form To Load | No | None | Wait Until Element Is Visible | Wait for the registration form to load. |

**VERIFICATION KEYWORDS**

| Keyword Name | Input Parameters | Parameters | Derived From SeleniumLibrary Keyword | Description |
|-----|-----|-----|-----|-----|
| Verify Registration Form Is Displayed | Yes | None | Page Should Contain | Verify that the registration form is displayed. |
| Verify Error Message Is Displayed | Yes | error_message | Element Should Contain | Verify that an error message is displayed. |

Note: I've followed the rules and guidelines provided, ensuring that the keywords are reusable, generic, and data-independent.
```

Hình 4.21 Kết quả output AI sinh keyword chức năng Đăng ký

```
*** Keywords ***

Fill Register Form
    [Documentation]    Fill in the registration form with provided email, password, and re-entered password
    # TODO: Implement
    [Arguments]    ${email}    ${password}    ${re_password}
    Input Text To Element    ${EMAIL_INPUT}    ${email}
    Input Text To Element    ${PASSWORD_INPUT}    ${password}
    Input Text To Element    ${RE_PASSWORD_INPUT}    ${re_password}

Register User
    [Documentation]    Create a new user account.
    # TODO: Implement
    [Arguments]    ${email}    ${password}    ${re_password}
    Open Page    ${URL}
    Click On Element    ${REGISTER_URL}
    Fill Register Form    ${email}    ${password}    ${re_password}
    Click On Element    ${REGISTER_BTN}
```

Hình 4.22 Business keyword chức năng Đăng ký

```
Here is the Happy Path Main Business Flow:

**Register User Successfully**

1. **Register User** | ${data}
2. **Register User**
3. **Verify Page Contains Text** | "Please wait while we process your registration..."
4. **Verify Element Should Be Visible** | "Registration successful"
5. **Verify Current URL Should Be** | "/account-created"

Note: The placeholders `${data}` represent the input data for the Register User business keyword, which is not specified in this example.
```

Hình 4.23 Kết quả AI sinh flow chức năng Đăng ký

```
*** Test Cases ***
register Auto Test 1
  [Documentation]    Auto generated from AI flow

  Register User      ngminhtu@gmail.com  ngminhtu  ngminhtu
  Verify Page Contains Text    Email đã được sử dụng

register Auto Test 2
  [Documentation]    Auto generated from AI flow

  Register User      ngminhtu@gmail.com  ngmi  ngmi
  Verify Element Text Contains    ${ERROR_MSG_REGISTER}    Mật khẩu tối thiểu phải có 6 ký tự

register Auto Test 3
  [Documentation]    Auto generated from AI flow

  Register User      hoanganh@gmail.com  hoanganh  hoanganh1
  Verify Element Text Contains    ${ERROR_MSG_REGISTER}    Xác nhận mật khẩu không khớp với mật khẩu

register Auto Test 4
  [Documentation]    Auto generated from AI flow

  Register User      ${EMPTY}  ngminhtu  ngminhtu
  Verify Required Field Message    Please fill out this field.

register Auto Test 5
  [Documentation]    Auto generated from AI flow

  Register User      ngminhtu@gmail.com  ${EMPTY}  ngminhtu
  Verify Required Field Message    Please fill out this field.
```

Hình 4.24 Test script chức năng Đăng ký

4.5.3 Chức năng Tìm kiếm sản phẩm

```
Here is the output:

**BUSINESS KEYWORDS**

| Keyword Name | Input Parameters | Required Parameters | Optional Parameters | Description |
|-----|-----|-----|-----|-----|
| Search Product | Yes | product_name | | Perform search for a product with given name. |

**UI ACTION KEYWORDS**

| Keyword Name | Input Parameters | Parameters | Derived From SeleniumLibrary Keyword | Description |
|-----|-----|-----|-----|-----|
| Input Search Query | Yes | query_text | Input Text | Enter the search query into the search field. |
| Click Search Button | No | | Click Element | Click the search button to initiate the search. |

**VERIFICATION KEYWORDS**

| Keyword Name | Input Parameters | Parameters | Derived From SeleniumLibrary Keyword | Description |
|-----|-----|-----|-----|-----|
| Verify Product List Is Present | Yes | product_list | Page Should Contain | Verify that the list of products is present on the page. |
| Verify No Results Message Is Present | Yes | message_text | Element Should Be Visible | Verify that the no results message is present an

Note: I've followed the rules and guidelines provided to generate reusable keywords for the given feature using exactly three layers (Busi
```

Hình 4.25 Kết quả output AI sinh keyword chức năng Tìm kiếm


```
*** Keywords ***

Select Product From Result
    [Documentation]    Select a product from the search results based on the given index
    # TODO: Implement
    [Arguments]       ${index}
    ${elements}=      Get WebElements    ${PRODUCT_ITEMS}
    Should Not Be Empty    ${elements}
    Scroll Element Into View    ${elements}[${index}]
    Click On Element          ${elements}[${index}]

Search Product
    [Documentation]    Perform search for a product with given name.
    # TODO: Implement
    [Arguments]       ${product}
    Open Page         ${URL}
    Click On Element  ${SEARCH_INPUT}
    Input Text To Element    ${SEARCH_INPUT}    ${product}
    Click On Element  ${SEARCH_BTN}
```

Hình 4.26 Business keyword chức năng Tìm kiếm

```
Here is the Happy Path Main Business Flow:

**Search Product**

1. **Search Product** | ${searchQuery}
2. **Select Product From Result**
3. **Verify Page Contains Element** | ${expectedProductList}

Note: The `${searchQuery}` placeholder represents the input search query, and `${expectedProductList}` represents the expected list of pro
```

Hình 4.27 Kết quả AI sinh flow chức năng Tìm kiếm

```
*** Test Cases ***

search Auto Test 1
    [Documentation]    Auto generated from AI flow

    Search Product      cao gót 7cm
    Verify Page Contains Element    ${PRODUCTS_NAME}
    Verify Element Text Contains    ${PRODUCTS_NAME}    cao gót 7cm

search Auto Test 2
    [Documentation]    Auto generated from AI flow

    Search Product      ELA301
    Verify Page Contains Element    ${PRODUCTS_NAME}
    Verify Element Text Contains    ${PRODUCTS_NAME}    ELA301

search Auto Test 3
    [Documentation]    Auto generated from AI flow

    Search Product      Giày sandal công sở ELLA 7cm thanh lịch cuốn hút ELA662
    Verify Page Contains Element    ${PRODUCTS_NAME}
    Verify Element Text Contains    ${PRODUCTS_NAME}    Giày sandal công sở ELLA 7cm thanh lịch cuốn hút ELA662

search Auto Test 4
    [Documentation]    Auto generated from AI flow

    Search Product      !@#$
    Verify Page Contains Element    ${PRODUCTS_NAME}

search Auto Test 5
    [Documentation]    Auto generated from AI flow

    Search Product      ${EMPTY}
    Verify Required Field Message    Please fill out this field.
```

Hình 4.28 Test script chức năng Tìm kiếm

4.5.4 Chức năng Đặt hàng

Here is the output:

```

**BUSINESS KEYWORDS**

| Keyword Name | Input Parameters | Required Parameters | Optional Parameters | Description |
|-----|-----|-----|-----|-----|
| Place Order | Yes | Product ID, Quantity | None | Initiate order process |
| Add To Cart | Yes | Product ID | None | Add product to cart |
| View Cart | No | None | None | Display cart contents |

**UI ACTION KEYWORDS**

| Keyword Name | Input Parameters | Parameters | Derived From SeleniumLibrary Keyword | Description |
|-----|-----|-----|-----|-----|
| Click On Product | Yes | Product Element | Click Element | Select product |
| Input Quantity | Yes | Quantity Field | Input Text | Enter quantity |
| Open Cart Page | No | None | Get WebElements | Navigate to cart page |
| Close Cart Page | No | None | Get WebElements | Exit cart page |

**VERIFICATION KEYWORDS**

| Keyword Name | Input Parameters | Parameters | Derived From SeleniumLibrary Keyword | Description |
|-----|-----|-----|-----|-----|
| Verify Product Added | Yes | Product ID | Element Should Contain | Validate product in cart |
| Verify Cart Empty | No | None | Page Should Contain | Check if cart is empty |
    
```

Hình 4.29 Kết quả output AI sinh keyword chức năng Đặt hàng

```

Add To Cart From HomePage
[Documentation]    Add product to cart from home page
Implement with Codex
# TODO: Implement
[Arguments]    ${index}
Open Page    ${URL}
Select Product From Result    ${index}
Wait Until Element Is Visible    ${ADD_TO_CART_BTN}    timeout=10s
Scroll Element Into View    ${ADD_TO_CART_BTN}

Wait Until Keyword Succeeds
...    10s
...    1s    You, 19 hours ago • update ...
...    Click On Element
...    ${ADD_TO_CART_BTN}

Place Order
[Documentation]    Start checkout process
Implement with Codex
# TODO: Implement
[Arguments]    ${name}    ${phone}    ${email}    ${address}    ${note}
Open Page    ${CART_ICON}
Input Text To Element    ${NAME_INPUT}    ${name}
Input Text To Element    ${PHONE_INPUT}    ${phone}
Input Text To Element    ${EMAIL_INPUT_ORDER}    ${email}
Input Text To Element    ${ADDRESS_INPUT}    ${address}
Input Text To Element    ${NOTE_INPUT}    ${note}
Click On Element    ${SUBMIT_ORDER}
    
```

Hình 4.30 Business keyword chức năng Đặt hàng

```
Here is the Happy Path Main Business Flow:

**Order Placement**

1. Add To Cart From HomePage | ${productID}
2. Place Order
3. Verify Page Contains Text | "Order Summary"

Note: The `${data}` and `${expected}` placeholders are used to represent input values and expected results, respectively.
```

Hình 4.31 Kết quả AI sinh flow chức năng Đặt hàng

```
*** Test Cases ***
order Auto Test 1
    [Documentation]    Auto generated from AI flow

    Add To Cart From HomePage    0
    Place Order    Linh    0123456789    linhnguyen@gmail.com    123 Main St, City    Giao giờ hành chính
    Verify Element Text Contains    ${ORDER_SUCCESS_MSG}    ĐƠN HÀNG CỦA BẠN ĐÃ ĐƯỢC GỬI ĐI!
    Verify Current URL Should Be    ${URL_ORDER_SUCCESS}

order Auto Test 2
    [Documentation]    Auto generated from AI flow

    Add To Cart From HomePage    0
    Place Order    Linh    01234    linhnguyen@gmail.com    123 Main St, City    Giao giờ hành chính
    Verify Element Text Contains    ${ORDER_ERROR_MSG}    Vui lòng nhập ít nhất một số điện thoại bạn đang sử dụng

order Auto Test 3
    [Documentation]    Auto generated from AI flow

    Add To Cart From HomePage    0
    Place Order    Linh    012345678999    linhnguyen@gmail.com    123 Main St, City    Giao giờ hành chính
    Verify Element Text Contains    ${ORDER_ERROR_MSG}    Vui lòng nhập ít nhất một số điện thoại bạn đang sử dụng

order Auto Test 4
    [Documentation]    Auto generated from AI flow

    Add To Cart From HomePage    0
    Place Order    Linh    012345679@    linhnguyen@gmail.com    123 Main St, City    Giao giờ hành chính
    Verify Element Text Contains    ${ORDER_ERROR_MSG}    Vui lòng nhập ít nhất một số điện thoại bạn đang sử dụng
```

Hình 4.32 Test script chức năng Đặt hàng

4.5.5 Chức năng Mua hàng hoàn chỉnh

```
Here is the generated end-to-end test flow using only the provided keywords:

[
  "Login To System",
  "Verify Element Should Be Visible: Login page",
  "Search Product",
  "Verify Page Contains Text: Search results",
  "Select Product From Result",
  "Add To Cart",
  "Verify Page Contains Element: Cart contents",
  "Place Order",
  "Verify Required Field Message: Order form",
  "Verify Current URL Should Be: Order confirmation page"
]

Note that I've followed the execution rules and output format guidelines to generate this test flow. Let me know if you have any questions
```

Hình 4.33 Kết quả AI sinh flow E2E chức năng Đặt hàng

```
*** Test Cases ***
E2E login_search_order_20260514_124712_1
  [Documentation]    AI generated E2E

  Login To System    tuyetmai99@gmail.com    tuyetmai99
  Select Product From Result    0
  Verify Element Should Be Visible    ${ACCOUNT_PAGE}
  Search Product    ELA811
  Verify Page Contains Text    ELA811
  Add To Cart From Search Result    0
  Verify Page Contains Element    ${CART_CONTENT}
  Place Order    Tuyet Mai    0123456789    tuyetmai99@gmail    123 Main St, City    Giao giờ hành chính
  Verify Element Text Contains    ${ORDER_SUCCESS_MSG}    ĐƠN HÀNG CỦA BẠN ĐÃ ĐƯỢC GỬI ĐI!
  Verify Current URL Should Be    ${URL_ORDER_SUCCESS}

E2E login_search_order_20260514_124712_2
  [Documentation]    AI generated E2E

  Login To System    tuyetmai99@gmail.com    tuyetmai99
  Select Product From Result    0
  Verify Element Should Be Visible    ${ACCOUNT_PAGE}
  Search Product    ELA811
  Verify Page Contains Text    ELA811
  Add To Cart From Search Result    0
  Verify Page Contains Element    ${CART_CONTENT}
  Place Order    Tuyet Mai    01234    tuyetmai99@gmail    123 Main St, City    Giao giờ hành chính
  Verify Element Text Contains    ${ORDER_ERROR_MSG}    Vui lòng nhập ít nhất một số điện thoại bạn đang sử dụng
```

Hình 4.34 Test script chức năng Đặt hàng hoàn chỉnh

4.5.6 Chức năng Cập nhật hồ sơ hoàn chỉnh

```
Here is the output: You, 6 days ago • ok ...

**BUSINESS KEYWORDS**

| Keyword Name | Input Parameters | Required Parameters | Optional Parameters | Description |
|-----|-----|-----|-----|-----|
| Update Profile | Yes | profile_id, new_info | None | Update user's personal information. |

**UI ACTION KEYWORDS**

| Keyword Name | Input Parameters | Parameters | Derived From SeleniumLibrary Keyword | Description |
|-----|-----|-----|-----|-----|
| Go To Profile Page | No | None | Go To | Navigate to the profile page. |
| Click On Update Button | No | None | Click Element | Click the update button. |
| Input New Information | Yes | new_info | Input Text | Enter new user information. |

**VERIFICATION KEYWORDS**

| Keyword Name | Input Parameters | Parameters | Derived From SeleniumLibrary Keyword | Description |
|-----|-----|-----|-----|-----|
| Verify Profile Updated | No | None | Element Should Contain | Verify that the profile has been updated correctly. |
| Verify Error Message | Yes | error_message | Page Should Contain | Verify that an error message is displayed when data is invalid. |

Note: I've followed the rules and guidelines provided to generate reusable keywords for the given feature.
```

Hình 4.35 Kết quả output AI sinh keyword chức năng Cập nhật hồ sơ

```
Update Profile
[Documentation]    Update user's personal information.
# TODO: Implement
[Arguments]    ${name}    ${phone}    ${address}
Click On Element    ${ACCOUNT_PAGE}
Input Text To Element    ${NAME_INPUT}    ${name}
Input Text To Element    ${PHONE_INPUT}    ${phone}
Input Text To Element    ${ADDRESS_INPUT}    ${address}
Click On Element    ${SAVE_BTN}
```

Hình 4.36 Business keyword chức năng Cập nhật hồ sơ

```
Here is the Happy Path Main Business Flow: You, 6 days ago • ok ...

**Update Profile Successfully**

1. Update Profile | ${newName}, ${newEmail}, ${newPhone}
2. Update Profile
3. Verify Page Contains Text | "Updated successfully"
4. Verify Current URL Should Be | "${expectedURL}"

Note: The placeholders `${newName}`, `${newEmail}`, and `${newPhone}` represent the new values entered by the user for their profile infor
```

Hình 4.37 Kết quả AI sinh flow chức năng Cập nhật hồ sơ

```
RAW AI OUTPUT:
Here is the generated end-to-end test flow using only the provided keywords:

[
  "Login To System",
  "Verify Current URL Should Be",
  "Update Profile",
  "Verify Element Text Contains",
  "Verify Page Contains Text"
]

This flow follows the strict rules and guidelines provided, using only the available business keywords and global verify keywords. It also ensures that each action is followed by a verification step to validate the result of the previous action.

PARSED CLEAN STEPS: ['Login To System', 'Verify Current URL Should Be', 'Update Profile', 'Verify Element Text Contains', 'Verify Page Contains Text']

E2E Steps:
1. Login To System
2. Verify Current URL Should Be
3. Update Profile
4. Verify Element Text Contains
5. Verify Page Contains Text
```

Hình 4.38 Quá trình sinh flow E2E chức năng Cập nhật hồ sơ

```
Here is the generated end-to-end test flow using only the provided keywords: You, 21 hours ago • update ...

[
  "Login To System",
  "Verify Current URL Should Be",
  "Update Profile",
  "Verify Element Text Contains",
  "Verify Page Contains Text"
]

This flow follows the strict rules and guidelines provided, using only the available business keywords and global verify keywords.
```

Hình 4.39 Kết quả AI sinh flow E2E chức năng Cập nhật hồ sơ

```
*** Test Cases ***
E2E login_20260517_213426_1
  [Documentation]    AI generated E2E

  Login To System      ngvanhoan@gmail.com    ngvanhoan
  Select Product From Result    0
  Verify Element Should Be Visible    ${ACCOUNT_PAGE}
  Update Profile      Nguyen Van Hoan    0123456789    123 Nguyen Trai Street
  Verify Element Text Contains    ${SUCCESS_MSG}    Cập nhật thông tin tài khoản thành công!

E2E login_20260517_213426_2
  [Documentation]    AI generated E2E

  Login To System      ngvanhoan@gmail.com    ngvanhoan
  Select Product From Result    0
  Verify Element Should Be Visible    ${ACCOUNT_PAGE}
  Update Profile      ${EMPTY}    0123456789    123 Nguyen Trai Street
  Verify Element Text Contains    ${SUCCESS_MSG}    Cập nhật thông tin tài khoản thành công!

E2E login_20260517_213426_3
  [Documentation]    AI generated E2E

  Login To System      ngvanhoan@gmail.com    ngvanhoan
  Select Product From Result    0
  Verify Element Should Be Visible    ${ACCOUNT_PAGE}
  Update Profile      Nguyen Van Hoan    ${EMPTY}    123 Nguyen Trai Street
  Verify Element Text Contains    ${SUCCESS_MSG}    Cập nhật thông tin tài khoản thành công!
```

Hình 4.40 Test script chức năng Cập nhật hồ sơ

4.5.7 Tổng hợp và đánh giá kết quả thực nghiệm

Bảng 4.14 Đánh giá kết quả AI hỗ trợ sinh keyword và flow kiểm thử

Nội dung đánh giá	Kết quả đạt được
Số chức năng được sử dụng để sinh keyword	5 chức năng: đăng nhập, đăng ký, tìm kiếm, đặt hàng, cập nhật hồ sơ.
Số luồng E2E được sinh bằng AI	2 luồng: mua hàng hoàn chỉnh và cập nhật hồ sơ hoàn chỉnh.
Dữ liệu đầu vào cho AI	File mô tả chức năng hoặc mô tả nghiệp vụ E2E.
Kết quả AI sinh ra	Danh sách keyword, flow kiểm thử và test script Robot Framework.
Cách tổ chức keyword	Phân tách theo Business Layer, UI Action Layer và Verification Layer.
Cơ chế xử lý output AI	Parse output, chuẩn hóa keyword, kiểm tra trùng lặp và inject vào framework.
Khả năng tái sử dụng	UI keyword và Verification keyword được dùng chung cho nhiều chức năng.
Vai trò của kiểm thử viên	Kiểm tra output AI, bổ sung locator, dữ liệu kiểm thử, assertion và logic đặc biệt.
Hạn chế ghi nhận	AI vẫn có thể sinh thiếu bước, sai thứ tự hoặc chưa phù hợp hoàn toàn với nghiệp vụ thực tế.

Kết quả thực nghiệm cho thấy AI có thể hỗ trợ kiểm thử viên trong giai đoạn khởi tạo keyword và flow kiểm thử. Tuy nhiên, output do AI sinh ra chưa thể sử dụng hoàn toàn tự động trong mọi trường hợp. Kiểm thử viên vẫn cần kiểm tra, hiệu chỉnh

và bổ sung logic trước khi đưa test script vào thực thi. Điều này phù hợp với định hướng của đề tài là sử dụng AI như một công cụ hỗ trợ trong framework kiểm thử tự động, thay vì thay thế hoàn toàn vai trò của kiểm thử viên.

Framework đã thực thi được các nhóm chức năng chính của website gồm đăng nhập, đăng ký, tìm kiếm sản phẩm, đặt hàng, mua hàng hoàn chỉnh và cập nhật hồ sơ. Kết quả pass/fail được ghi nhận thông qua báo cáo Robot Framework, log thực thi và Allure Report. Các test case thất bại được framework tự động ghi log và chụp ảnh màn hình nhằm hỗ trợ kiểm thử viên phân tích nguyên nhân lỗi.

4.6 Báo cáo kết quả kiểm thử

4.6.1 Báo cáo kết quả kiểm thử chức năng Đăng nhập

```
(.venv) PS D:\DoAnTotNghiep_NguyenQuyen_10122312> python run.py
===== CHỌN TEST =====
1. Search
2. Login
3. Register
4. Order
5. Profile
Nhập lựa chọn: 2

Running login...

=====
Login Auto 20260510 150439
=====

login Auto Test 1 :: Auto generated from AI flow
DevTools listening on ws://127.0.0.1:55127/devtools/browser/99352884-68db-4a6e-9bce-4c47792807f6
login Auto Test 1 :: Auto generated from AI flow | PASS |
-----

login Auto Test 2 :: Auto generated from AI flow
DevTools listening on ws://127.0.0.1:54838/devtools/browser/20b2c622-ccf6-4087-b858-d9432c97f27c
login Auto Test 2 :: Auto generated from AI flow | PASS |
-----

login Auto Test 3 :: Auto generated from AI flow
DevTools listening on ws://127.0.0.1:57245/devtools/browser/a94e5e6c-1b12-43c5-afad-e22264eba29d
login Auto Test 3 :: Auto generated from AI flow | PASS |
-----

login Auto Test 4 :: Auto generated from AI flow
DevTools listening on ws://127.0.0.1:56564/devtools/browser/3c4a3cb7-9136-4a76-9d9a-cd5056f17abc
login Auto Test 4 :: Auto generated from AI flow | PASS |
-----

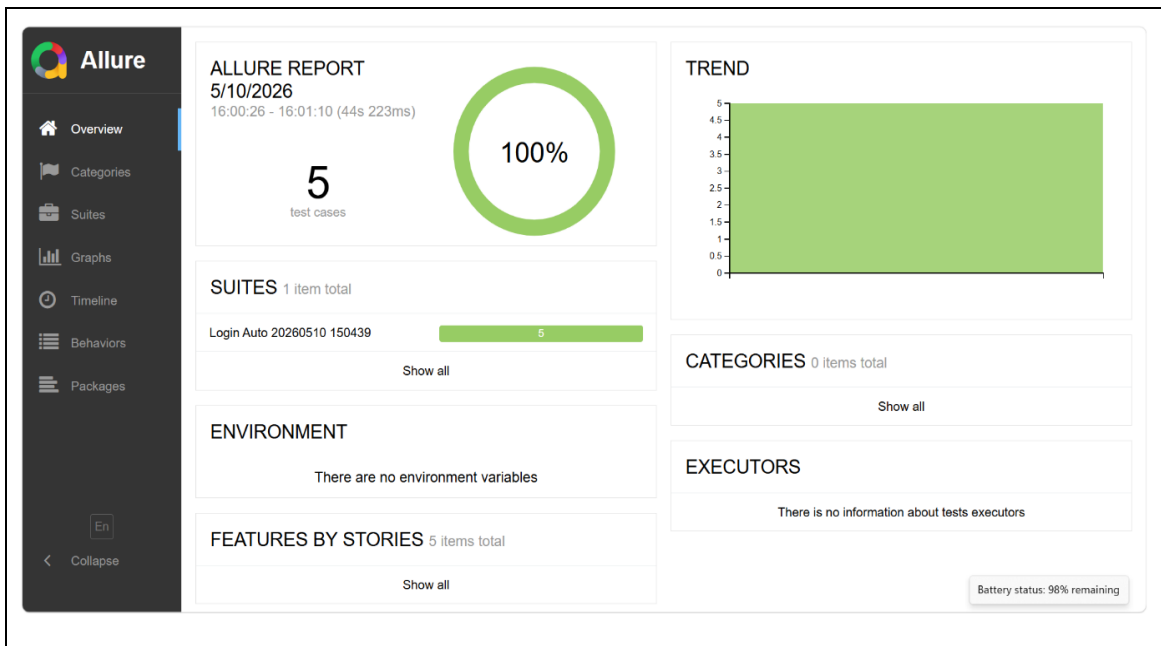
login Auto Test 5 :: Auto generated from AI flow
DevTools listening on ws://127.0.0.1:58984/devtools/browser/235acdc0-544a-47de-b610-db070f1b3a91
..[16400:20568:0510/160107.109:ERROR:chrome/browser/task_manager/providers/fallback_task_provider.cc:126] Every renderer
should have at least one task provided by a primary task provider. If a "Renderer" fallback task is shown, it is a bug. I
f you have repro steps, please file a new bug and tag it as a dependency of crbug.com/40528867.
login Auto Test 5 :: Auto generated from AI flow | PASS |
-----

Login Auto 20260510 150439 | PASS |
```

Hình 4.41 Quá trình thực thi kiểm thử chức năng Đăng nhập


```
2026-05-10 16:00:27,458 - INFO - Browser opened
2026-05-10 16:00:27,461 - INFO - [STEP] Opening page: https://ella.vn/
2026-05-10 16:00:28,988 - INFO - [STEP] Click element xpath://div[@class='lf']/span[contains(text(),'Đăng nhập')]
2026-05-10 16:00:28,990 - INFO - [STEP] Wait until element xpath://div[@class='lf']/span[contains(text(),'Đăng nhập')] is visible with timeout 10s
2026-05-10 16:00:29,146 - INFO - [STEP] Input text "ngvanhoan@gmail.com" to element css:input[placeholder='Email']
2026-05-10 16:00:29,151 - INFO - [STEP] Wait until element css:input[placeholder='Email'] is visible with timeout 10s
2026-05-10 16:00:30,033 - INFO - [STEP] Input text "123456789" to element css:input[placeholder='Password']
2026-05-10 16:00:30,038 - INFO - [STEP] Wait until element css:input[placeholder='Password'] is visible with timeout 10s
2026-05-10 16:00:30,279 - INFO - [STEP] Click element xpath://button[contains(text(),'Đăng nhập')]
2026-05-10 16:00:30,281 - INFO - [STEP] Wait until element xpath://button[contains(text(),'Đăng nhập')] is visible with timeout 10s
2026-05-10 16:00:32,611 - INFO - [STEP] Text: Tài khoản hoặc mật khẩu không chính xác
2026-05-10 16:00:32,613 - INFO - [STEP] Expected: Tài khoản hoặc mật khẩu không chính xác
2026-05-10 16:00:32,621 - INFO - Close browser
2026-05-10 16:00:35,680 - INFO - =====
2026-05-10 16:00:35,681 - INFO - Browser opened
2026-05-10 16:00:35,685 - INFO - [STEP] Opening page: https://ella.vn/
2026-05-10 16:00:37,302 - INFO - [STEP] Click element xpath://div[@class='lf']/span[contains(text(),'Đăng nhập')]
2026-05-10 16:00:37,308 - INFO - [STEP] Wait until element xpath://div[@class='lf']/span[contains(text(),'Đăng nhập')] is visible with timeout 10s
2026-05-10 16:00:37,500 - INFO - [STEP] Input text "tuyetmai999" to element css:input[placeholder='Email']
2026-05-10 16:00:37,507 - INFO - [STEP] Wait until element css:input[placeholder='Email'] is visible with timeout 10s
2026-05-10 16:00:38,396 - INFO - [STEP] Input text "tuyetmai99" to element css:input[placeholder='Password']
2026-05-10 16:00:38,399 - INFO - [STEP] Wait until element css:input[placeholder='Password'] is visible with timeout 10s
2026-05-10 16:00:38,580 - INFO - [STEP] Click element xpath://button[contains(text(),'Đăng nhập')]
2026-05-10 16:00:38,586 - INFO - [STEP] Wait until element xpath://button[contains(text(),'Đăng nhập')] is visible with timeout 10s
2026-05-10 16:00:40,865 - INFO - [STEP] Text: Tài khoản hoặc mật khẩu không chính xác
2026-05-10 16:00:40,867 - INFO - [STEP] Expected: Tài khoản hoặc mật khẩu không chính xác
2026-05-10 16:00:40,874 - INFO - Close browser
2026-05-10 16:00:43,878 - INFO - =====
2026-05-10 16:00:43,879 - INFO - Browser opened
2026-05-10 16:00:43,887 - INFO - [STEP] Opening page: https://ella.vn/
2026-05-10 16:00:45,512 - INFO - [STEP] Click element xpath://div[@class='lf']/span[contains(text(),'Đăng nhập')]
2026-05-10 16:00:45,518 - INFO - [STEP] Wait until element xpath://div[@class='lf']/span[contains(text(),'Đăng nhập')] is visible with timeout 10s
```

Hình 4.42 Log ghi lại quá trình thực thi chức năng Đăng nhập

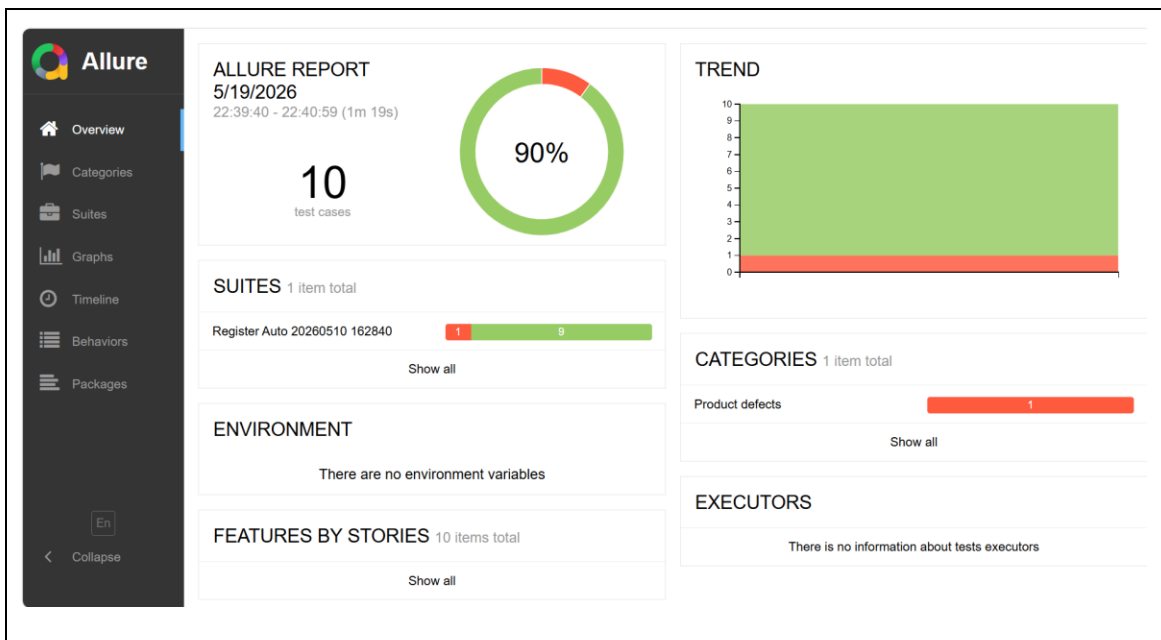


Hình 4.43 Báo cáo Allure kết quả thực thi chức năng Đăng nhập

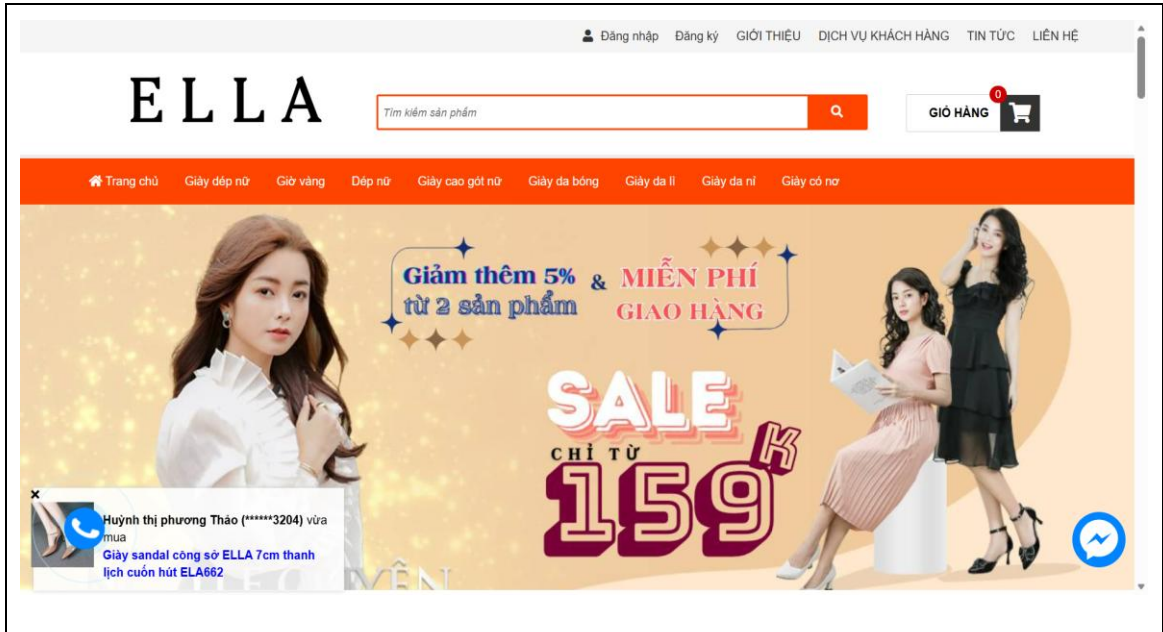
4.6.2 Báo cáo kết quả kiểm thử chức năng Đăng ký

```
2026-05-19 22:24:20,834 - INFO - Browser opened
2026-05-19 22:24:20,840 - INFO - [STEP] Opening page: https://ella.vn/
2026-05-19 22:24:22,406 - INFO - [STEP] Click element xpath://div[@class='lf']/span[contains(text(),'Đăng ký')]
2026-05-19 22:24:22,408 - INFO - [STEP] Wait until element xpath://div[@class='lf']/span[contains(text(),'Đăng ký')] is visible with timeo
2026-05-19 22:24:22,561 - INFO - [STEP] Input text "ngminhtu@gmail.com" to element css:input[placeholder='Email']
2026-05-19 22:24:22,566 - INFO - [STEP] Wait until element css:input[placeholder='Email'] is visible with timeout 10s
2026-05-19 22:24:23,427 - INFO - [STEP] Input text "ngminhtu" to element css:input[placeholder='Password']
2026-05-19 22:24:23,432 - INFO - [STEP] Wait until element css:input[placeholder='Password'] is visible with timeout 10s
2026-05-19 22:24:23,877 - INFO - [STEP] Input text "ngminhtu" to element css:input[placeholder='Re-password']
2026-05-19 22:24:23,685 - INFO - [STEP] Wait until element css:input[placeholder='Re-password'] is visible with timeout 10s
2026-05-19 22:24:23,874 - INFO - [STEP] Click element xpath://button[contains(text(),'Đăng ký')]
2026-05-19 22:24:23,877 - INFO - [STEP] Wait until element xpath://button[contains(text(),'Đăng ký')] is visible with timeout 10s
2026-05-19 22:24:24,720 - INFO - [STEP] Page contains text: Email đã được sử dụng
2026-05-19 22:24:24,727 - INFO - Close browser
2026-05-19 22:24:27,745 - INFO - =====
2026-05-19 22:24:27,746 - INFO - Browser opened
2026-05-19 22:24:27,752 - INFO - [STEP] Opening page: https://ella.vn/
2026-05-19 22:24:29,422 - INFO - [STEP] Click element xpath://div[@class='lf']/span[contains(text(),'Đăng ký')]
2026-05-19 22:24:29,425 - INFO - [STEP] Wait until element xpath://div[@class='lf']/span[contains(text(),'Đăng ký')] is visible with timeo
2026-05-19 22:24:29,606 - INFO - [STEP] Input text "ngminhtu@gmail.com" to element css:input[placeholder='Email']
2026-05-19 22:24:29,609 - INFO - [STEP] Wait until element css:input[placeholder='Email'] is visible with timeout 10s
2026-05-19 22:24:30,531 - INFO - [STEP] Input text "ngmi" to element css:input[placeholder='Password']
2026-05-19 22:24:30,536 - INFO - [STEP] Wait until element css:input[placeholder='Password'] is visible with timeout 10s
2026-05-19 22:24:30,813 - INFO - [STEP] Input text "ngmi" to element css:input[placeholder='Re-password']
2026-05-19 22:24:30,818 - INFO - [STEP] Wait until element css:input[placeholder='Re-password'] is visible with timeout 10s
2026-05-19 22:24:31,064 - INFO - [STEP] Click element xpath://button[contains(text(),'Đăng ký')]
2026-05-19 22:24:31,069 - INFO - [STEP] Wait until element xpath://button[contains(text(),'Đăng ký')] is visible with timeout 10s
2026-05-19 22:24:31,980 - INFO - [STEP] Text: Mật khẩu tối thiểu phải có 6 ký tự
2026-05-19 22:24:31,986 - INFO - [STEP] Expected: Mật khẩu tối thiểu phải có 6 ký tự
2026-05-19 22:24:31,995 - INFO - Close browser
```

Hình 4.44 Log ghi lại quá trình thực thi chức năng Đăng ký



Hình 4.45 Báo cáo Allure kết quả thực thi chức năng Đăng ký

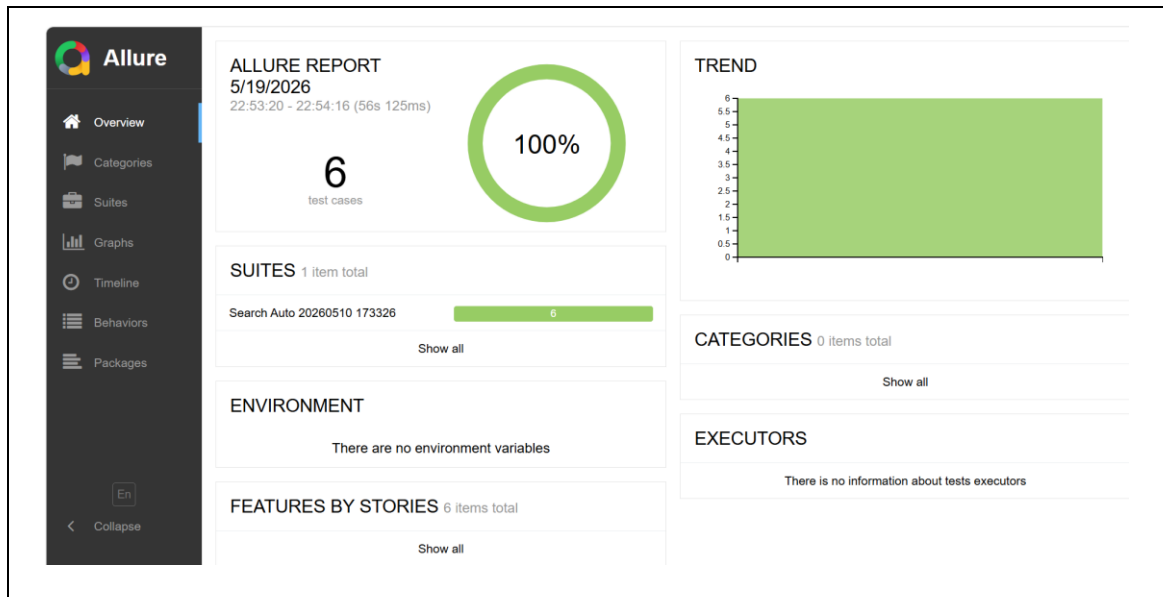


Hình 4.46 Ảnh chụp màn hình lỗi (test 3)

4.6.3 Báo cáo kết quả kiểm thử chức năng Tìm kiếm

```
2026-05-10 17:52:13,145 - INFO - Browser opened
2026-05-10 17:52:13,148 - INFO - [STEP] Opening page: https://ella.vn/
2026-05-10 17:52:14,812 - INFO - [STEP] Click element css:input[name='q']
2026-05-10 17:52:14,814 - INFO - [STEP] Wait until element css:input[name='q'] is visible with timeout 10s
2026-05-10 17:52:14,937 - INFO - [STEP] Input text "cao gót 7cm" to element css:input[name='q']
2026-05-10 17:52:14,940 - INFO - [STEP] Wait until element css:input[name='q'] is visible with timeout 10s
2026-05-10 17:52:15,111 - INFO - [STEP] Click element css:button[type='submit']
2026-05-10 17:52:15,114 - INFO - [STEP] Wait until element css:button[type='submit'] is visible with timeout 10s
2026-05-10 17:52:16,881 - INFO - [STEP] Element located by: xpath:(//div[@class='thread-list-title text-center']) is present on the page.
2026-05-10 17:52:17,021 - INFO - [STEP] Text: Giày cao gót 7cm thiết kế cách điệu với da dập vân cao cấp ELA702
2026-05-10 17:52:17,024 - INFO - [STEP] Expected: cao gót 7cm
2026-05-10 17:52:17,032 - INFO - Close browser
2026-05-10 17:52:20,036 - INFO - =====
2026-05-10 17:52:20,037 - INFO - Browser opened
2026-05-10 17:52:20,043 - INFO - [STEP] Opening page: https://ella.vn/
2026-05-10 17:52:21,630 - INFO - [STEP] Click element css:input[name='q']
2026-05-10 17:52:21,635 - INFO - [STEP] Wait until element css:input[name='q'] is visible with timeout 10s
2026-05-10 17:52:21,762 - INFO - [STEP] Input text "ELA301" to element css:input[name='q']
2026-05-10 17:52:21,766 - INFO - [STEP] Wait until element css:input[name='q'] is visible with timeout 10s
2026-05-10 17:52:21,893 - INFO - [STEP] Click element css:button[type='submit']
2026-05-10 17:52:21,899 - INFO - [STEP] Wait until element css:button[type='submit'] is visible with timeout 10s
2026-05-10 17:52:23,775 - INFO - [STEP] Element located by: xpath:(//div[@class='thread-list-title text-center']) is present on the page.
2026-05-10 17:52:23,862 - INFO - [STEP] Text: Dép cao gót công số ELLA 6cm thiết kế cách điệu với da dập vân cao cấp ELA702
2026-05-10 17:52:23,864 - INFO - [STEP] Expected: ELA301
2026-05-10 17:52:23,872 - INFO - Close browser
2026-05-10 17:52:26,919 - INFO - =====
2026-05-10 17:52:26,921 - INFO - Browser opened
2026-05-10 17:52:26,927 - INFO - [STEP] Opening page: https://ella.vn/
2026-05-10 17:52:28,553 - INFO - [STEP] Click element css:input[name='q']
2026-05-10 17:52:28,558 - INFO - [STEP] Wait until element css:input[name='q'] is visible with timeout 10s
2026-05-10 17:52:28,679 - INFO - [STEP] Input text "Giày sandal công số ELLA 7cm thanh lịch cuốn hút ELA662" to element css:input[name='q']
2026-05-10 17:52:28,683 - INFO - [STEP] Wait until element css:input[name='q'] is visible with timeout 10s
```

Hình 4.47 Log ghi lại quá trình thực thi chức năng Tìm kiếm

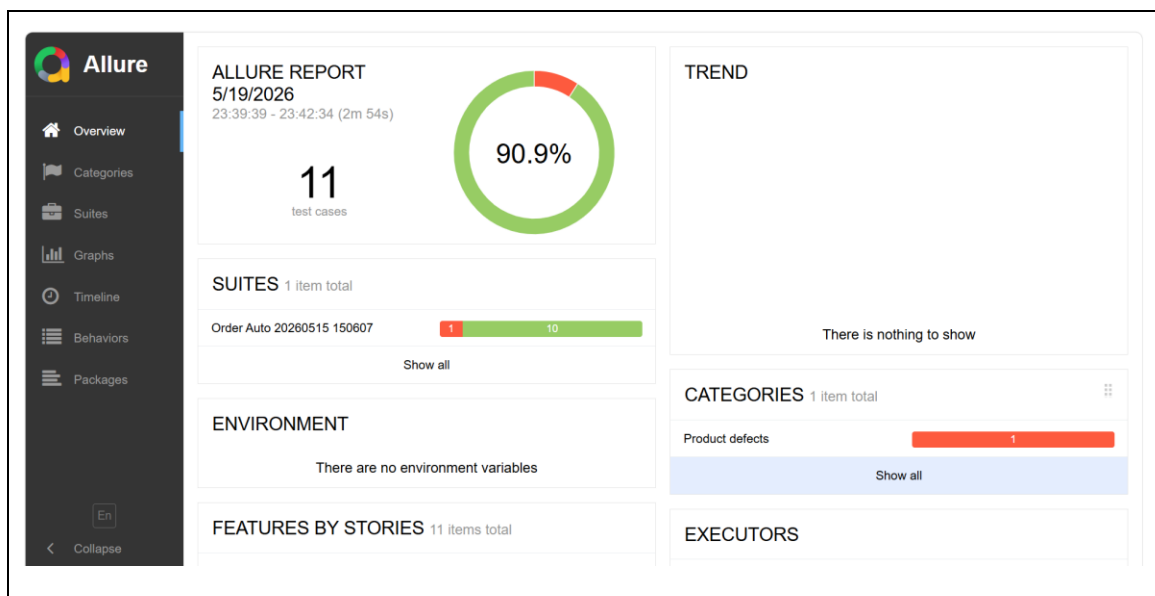


Hình 4.48 Báo cáo Allure kết quả thực thi chức năng Tìm kiếm

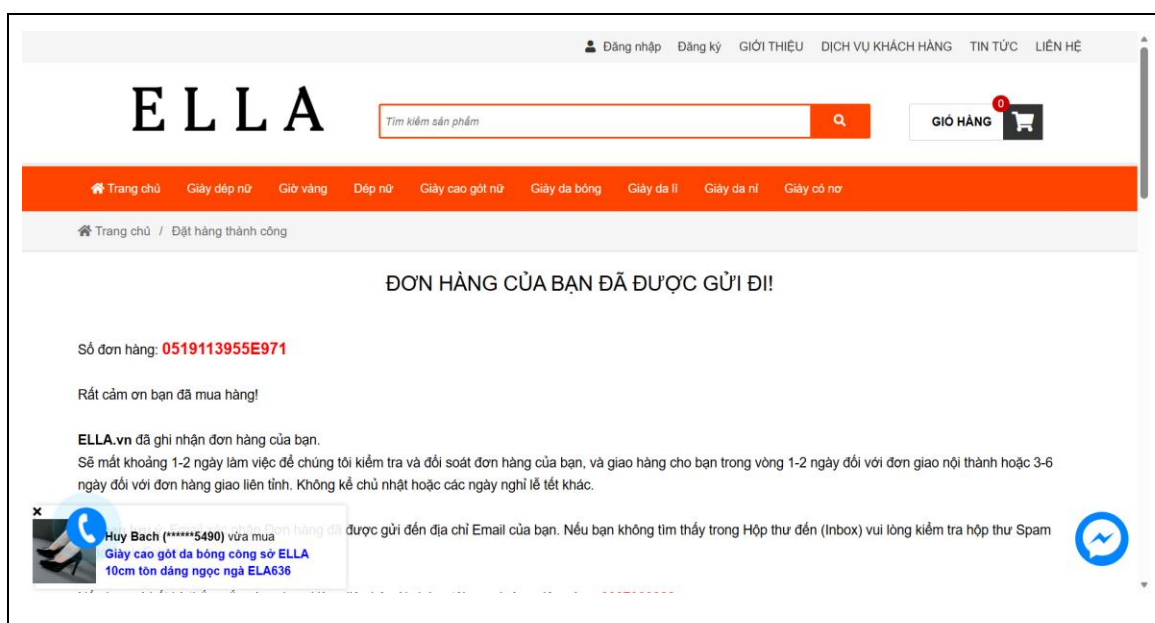
4.6.4 Báo cáo kết quả kiểm thử chức năng Đặt hàng

```
2026-05-10 22:19:27,631 - INFO - Browser opened
2026-05-10 22:19:27,635 - INFO - [STEP] Opening page: https://ella.vn/
2026-05-10 22:19:31,373 - INFO - [STEP] Click element <selenium.webdriver.remote.webelement.WebElement (session="9f2d
2026-05-10 22:19:31,379 - INFO - [STEP] Wait until element <selenium.webdriver.remote.webelement.WebElement (session=
2026-05-10 22:19:32,523 - INFO - [STEP] Click element xpath://div[@class='detail-muangay-button detail-muangay-center
2026-05-10 22:19:32,527 - INFO - [STEP] Wait until element xpath://div[@class='detail-muangay-button detail-muangay-c
2026-05-10 22:19:34,706 - INFO - [STEP] Opening page: css:a[title='Giỏ hàng']
2026-05-10 22:19:35,592 - INFO - [STEP] Input text "Linh" to element id:t_ten
2026-05-10 22:19:35,596 - INFO - [STEP] Wait until element id:t_ten is visible with timeout 10s
2026-05-10 22:19:35,847 - INFO - [STEP] Input text "0123456789" to element id:t_dienthoai
2026-05-10 22:19:35,851 - INFO - [STEP] Wait until element id:t_dienthoai is visible with timeout 10s
2026-05-10 22:19:36,114 - INFO - [STEP] Input text "linhnguyen@gmail" to element id:t_email
2026-05-10 22:19:36,118 - INFO - [STEP] Wait until element id:t_email is visible with timeout 10s
2026-05-10 22:19:36,296 - INFO - [STEP] Input text "123 Main St, City" to element id:t_diachi
2026-05-10 22:19:36,301 - INFO - [STEP] Wait until element id:t_diachi is visible with timeout 10s
2026-05-10 22:19:36,506 - INFO - [STEP] Input text "Giao giờ hành chính" to element id:t_ghichu
2026-05-10 22:19:36,509 - INFO - [STEP] Wait until element id:t_ghichu is visible with timeout 10s
2026-05-10 22:19:36,762 - INFO - [STEP] Click element xpath://button[@id='sb_submit_cart']
2026-05-10 22:19:36,766 - INFO - [STEP] Wait until element xpath://button[@id='sb_submit_cart'] is visible with timeo
2026-05-10 22:19:39,301 - INFO - [STEP] Text: ĐƠN HÀNG CỦA BẠN ĐÃ ĐƯỢC GỬI ĐI!
2026-05-10 22:19:39,304 - INFO - [STEP] Expected: ĐƠN HÀNG CỦA BẠN ĐÃ ĐƯỢC GỬI ĐI!
2026-05-10 22:19:39,332 - INFO - [STEP] Current URL: https://ella.vn/hoan-tat
2026-05-10 22:19:39,344 - INFO - Close browser
2026-05-10 22:19:42,378 - INFO - =====
2026-05-10 22:19:42,379 - INFO - Browser opened
2026-05-10 22:19:42,384 - INFO - [STEP] Opening page: https://ella.vn/
2026-05-10 22:19:45,990 - INFO - [STEP] Click element <selenium.webdriver.remote.webelement.WebElement (session="ec5b
2026-05-10 22:19:45,999 - INFO - [STEP] Wait until element <selenium.webdriver.remote.webelement.WebElement (session=
2026-05-10 22:19:47,054 - INFO - [STEP] Click element xpath://div[@class='detail-muangay-button detail-muangay-center
2026-05-10 22:19:47,057 - INFO - [STEP] Wait until element xpath://div[@class='detail-muangay-button detail-muangay-c
2026-05-10 22:19:49,221 - INFO - [STEP] Opening page: css:a[title='Giỏ hàng']
2026-05-10 22:19:50,034 - INFO - [STEP] Input text "Linh" to element id:t_ten
2026-05-10 22:19:50,039 - INFO - [STEP] Wait until element id:t_ten is visible with timeout 10s
```

Hình 4.49 Log ghi lại quá trình thực thi chức năng Đặt hàng



Hình 4.50 Báo cáo Allure kết quả thực thi chức năng Đặt hàng

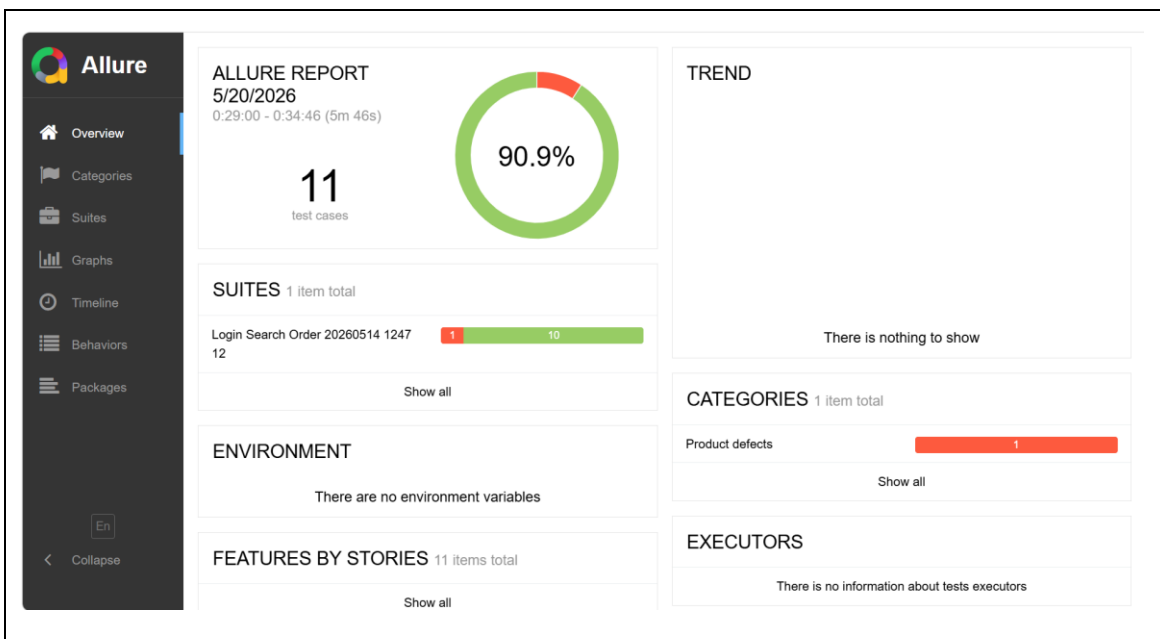


Hình 4.51 Ảnh chụp màn hình lỗi chức năng Đặt hàng (test 3)

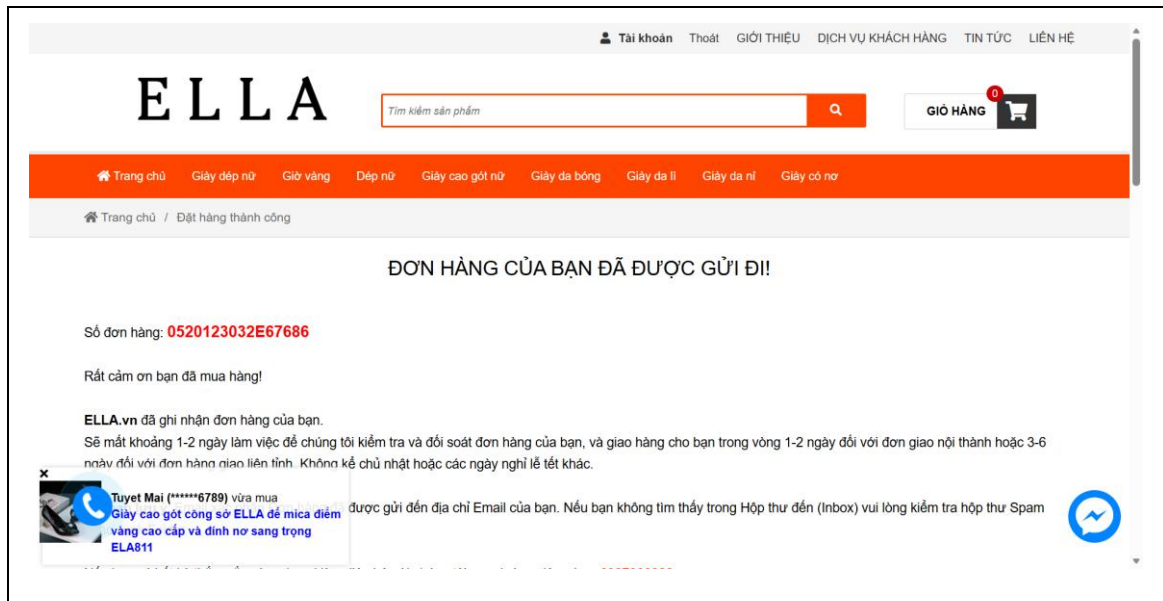
4.6.5 Báo cáo kết quả kiểm thử chức năng Mua hàng hoàn chỉnh

```
2026-05-20 00:29:42,192 - INFO - Browser opened
2026-05-20 00:29:42,211 - INFO - [STEP] Opening page: https://ella.vn/
2026-05-20 00:29:45,386 - INFO - [STEP] Click element xpath://div[@class='lf']/span[contains(text(),'Đăng nhập')]
2026-05-20 00:29:45,394 - INFO - [STEP] Wait until element xpath://div[@class='lf']/span[contains(text(),'Đăng nhập')] is visible with timeout 10s
2026-05-20 00:29:45,937 - INFO - [STEP] Input text "tuyetmai99@gmail.com" to element css:input[placeholder='Email']
2026-05-20 00:29:45,951 - INFO - [STEP] Wait until element css:input[placeholder='Email'] is visible with timeout 10s
2026-05-20 00:29:46,949 - INFO - [STEP] Input text "tuyetmai99" to element css:input[placeholder='Password']
2026-05-20 00:29:46,959 - INFO - [STEP] Wait until element css:input[placeholder='Password'] is visible with timeout 10s
2026-05-20 00:29:47,546 - INFO - [STEP] Click element xpath://button[contains(text(),'Đăng nhập')]
2026-05-20 00:29:47,554 - INFO - [STEP] Wait until element xpath://button[contains(text(),'Đăng nhập')] is visible with timeout 10s
2026-05-20 00:29:51,819 - INFO - [STEP] Click element <selenium.webdriver.remote.webelement.WebElement (session="7584d26cf28ebdae0b51c03dba
2026-05-20 00:29:51,827 - INFO - [STEP] Wait until element <selenium.webdriver.remote.webelement.WebElement (session="7584d26cf28ebdae0b51c
2026-05-20 00:29:52,676 - INFO - [STEP] Element located by: xpath://div[@class='lf']/span[contains(text(),'Tài khoản')] is visible on the
2026-05-20 00:29:52,682 - INFO - [STEP] Opening page: https://ella.vn/
2026-05-20 00:29:53,305 - INFO - [STEP] Click element css:input[name='q']
2026-05-20 00:29:53,309 - INFO - [STEP] Wait until element css:input[name='q'] is visible with timeout 10s
2026-05-20 00:29:53,548 - INFO - [STEP] Input text "ELA811" to element css:input[name='q']
2026-05-20 00:29:53,557 - INFO - [STEP] Wait until element css:input[name='q'] is visible with timeout 10s
2026-05-20 00:29:53,996 - INFO - [STEP] Click element css:button[type='submit']
2026-05-20 00:29:54,004 - INFO - [STEP] Wait until element css:button[type='submit'] is visible with timeout 10s
2026-05-20 00:29:56,326 - INFO - [STEP] Page contains text: ELA811
2026-05-20 00:29:56,723 - INFO - [STEP] Click element <selenium.webdriver.remote.webelement.WebElement (session="7584d26cf28ebdae0b51c03dba
2026-05-20 00:29:56,739 - INFO - [STEP] Wait until element <selenium.webdriver.remote.webelement.WebElement (session="7584d26cf28ebdae0b51c
2026-05-20 00:29:59,080 - INFO - [STEP] Click element xpath://div[@class='detail-muangay-button detail-muangay-center detail-muangay-topcen
2026-05-20 00:29:59,087 - INFO - [STEP] Wait until element xpath://div[@class='detail-muangay-button detail-muangay-center detail-muangay-topcen
2026-05-20 00:30:02,037 - INFO - [STEP] Element located by: xpath://*[normalize-space()='Danh sách Sản phẩm'] is present on the page.
2026-05-20 00:30:02,045 - INFO - [STEP] Opening page: css:a[title='Giỏ hàng']
2026-05-20 00:30:02,149 - INFO - [STEP] Input text "Tuyệt Mai" to element id:t_ten
2026-05-20 00:30:02,156 - INFO - [STEP] Wait until element id:t_ten is visible with timeout 10s
2026-05-20 00:30:02,546 - INFO - [STEP] Input text "01234" to element id:t_dienthoai
2026-05-20 00:30:02,556 - INFO - [STEP] Wait until element id:t_dienthoai is visible with timeout 10s
2026-05-20 00:30:02,949 - INFO - [STEP] Input text "tuyetmai99@gmail" to element id:t_email
2026-05-20 00:30:02,952 - INFO - [STEP] Wait until element id:t_email is visible with timeout 10s
```

Hình 4.52 Log ghi lại quá trình thực thi chức năng Đặt hàng hoàn chỉnh

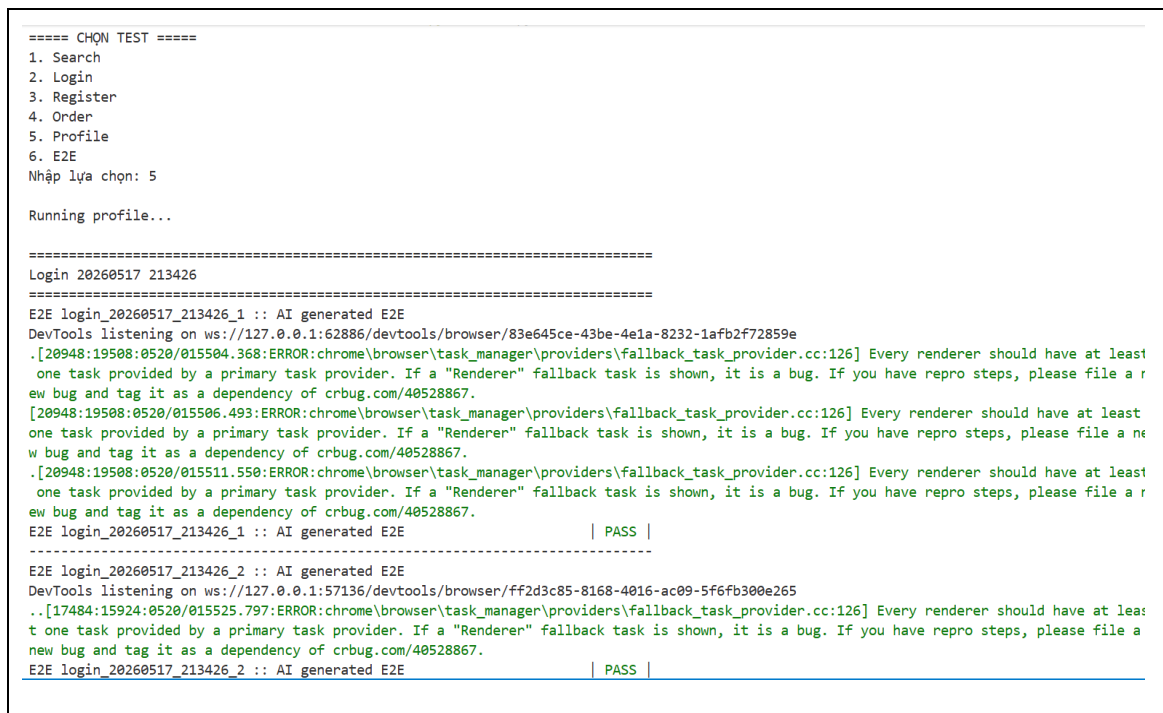


Hình 4.53 Báo cáo Allure kết quả thực thi chức năng Đặt hàng hoàn chỉnh



Hình 4.54 Ảnh chụp màn hình lỗi chức năng Đặt hàng (E2E test 3)

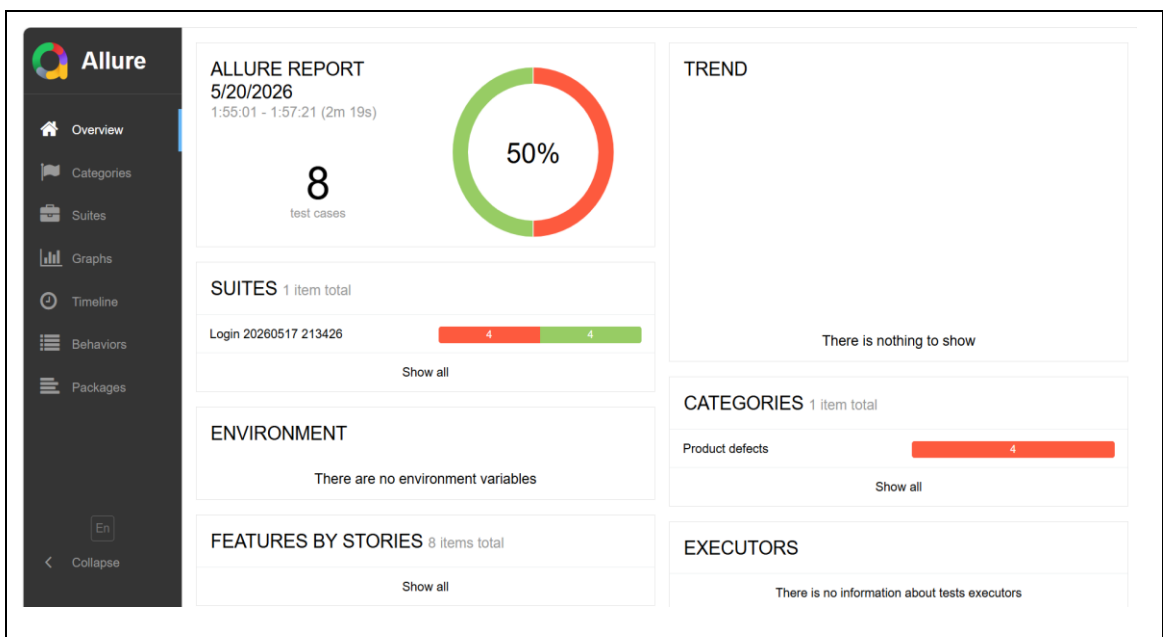
4.6.6 Báo cáo kết quả kiểm thử chức năng Cập nhật hồ sơ



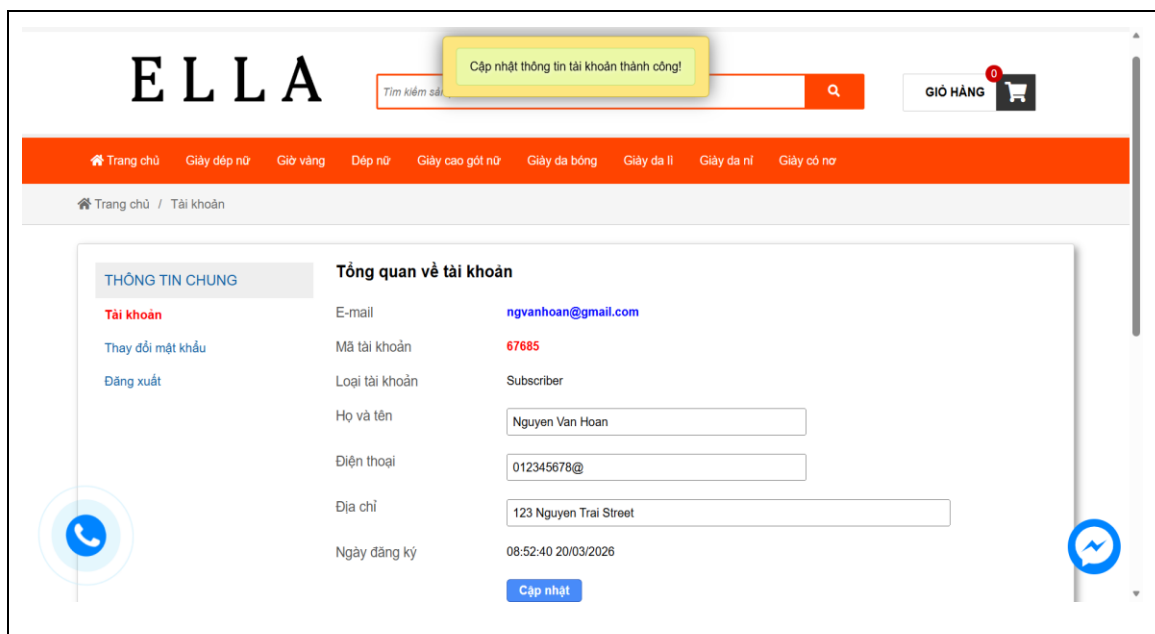
Hình 4.55 Quá trình thực thi kiểm thử chức năng Cập nhật hồ sơ

```
2026-05-20 01:55:03,599 - INFO - =====
2026-05-20 01:55:03,601 - INFO - Browser opened
2026-05-20 01:55:03,610 - INFO - [STEP] Opening page: https://ella.vn/
2026-05-20 01:55:06,958 - INFO - [STEP] Click element xpath://div[@class='lf']/span[contains(text(),'Đăng nhập')]
2026-05-20 01:55:06,961 - INFO - [STEP] Wait until element xpath://div[@class='lf']/span[contains(text(),'Đăng nhập')] is visible with tim
2026-05-20 01:55:07,190 - INFO - [STEP] Input text "ngvanhoan@gmail.com" to element css:input[placeholder='Email']
2026-05-20 01:55:07,195 - INFO - [STEP] Wait until element css:input[placeholder='Email'] is visible with timeout 10s
2026-05-20 01:55:08,071 - INFO - [STEP] Input text "ngvanhoan" to element css:input[placeholder='Password']
2026-05-20 01:55:08,074 - INFO - [STEP] Wait until element css:input[placeholder='Password'] is visible with timeout 10s
2026-05-20 01:55:08,282 - INFO - [STEP] Click element xpath://button[contains(text(),'Đăng nhập')]
2026-05-20 01:55:08,286 - INFO - [STEP] Wait until element xpath://button[contains(text(),'Đăng nhập')] is visible with timeout 10s
2026-05-20 01:55:11,948 - INFO - [STEP] Click element <selenium.webdriver.remote.webelement.WebElement (session="7f5519ecf9ff2eb94d8ec4cec5
2026-05-20 01:55:11,952 - INFO - [STEP] Wait until element <selenium.webdriver.remote.webelement.WebElement (session="7f5519ecf9ff2eb94d8ec
2026-05-20 01:55:12,697 - INFO - [STEP] Element located by: xpath://div[@class='lf']/span[contains(text(),'Tài khoản')] is visible on the
2026-05-20 01:55:12,701 - INFO - [STEP] Click element xpath://div[@class='lf']/span[contains(text(),'Tài khoản')]
2026-05-20 01:55:12,708 - INFO - [STEP] Wait until element xpath://div[@class='lf']/span[contains(text(),'Tài khoản')] is visible with tim
2026-05-20 01:55:14,452 - INFO - [STEP] Input text "Nguyen Van Hoan" to element xpath://input[@name='t_hoten']
2026-05-20 01:55:14,457 - INFO - [STEP] Wait until element xpath://input[@name='t_hoten'] is visible with timeout 10s
2026-05-20 01:55:14,777 - INFO - [STEP] Input text "0123456789" to element xpath://input[@name='t_dienthoai']
2026-05-20 01:55:14,780 - INFO - [STEP] Wait until element xpath://input[@name='t_dienthoai'] is visible with timeout 10s
2026-05-20 01:55:14,987 - INFO - [STEP] Input text "123 Nguyen Trai Street" to element xpath://input[@name='t_diachi']
2026-05-20 01:55:14,991 - INFO - [STEP] Wait until element xpath://input[@name='t_diachi'] is visible with timeout 10s
2026-05-20 01:55:15,210 - INFO - [STEP] Click element xpath://input[@value='Cập nhật']
2026-05-20 01:55:15,215 - INFO - [STEP] Wait until element xpath://input[@value='Cập nhật'] is visible with timeout 10s
2026-05-20 01:55:16,220 - INFO - [STEP] Text: Cập nhật thông tin tài khoản thành công!
2026-05-20 01:55:16,223 - INFO - [STEP] Expected: Cập nhật thông tin tài khoản thành công!
2026-05-20 01:55:16,232 - INFO - Close browser
```

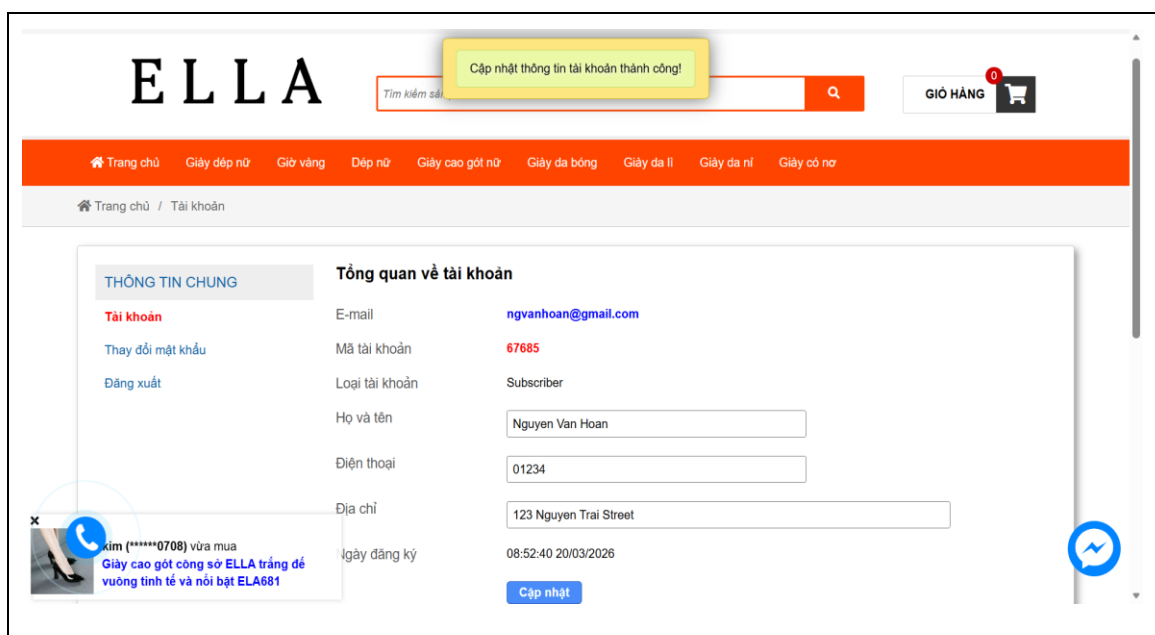
Hình 4.56 Log ghi lại quá trình thực thi chức năng Cập nhật hồ sơ



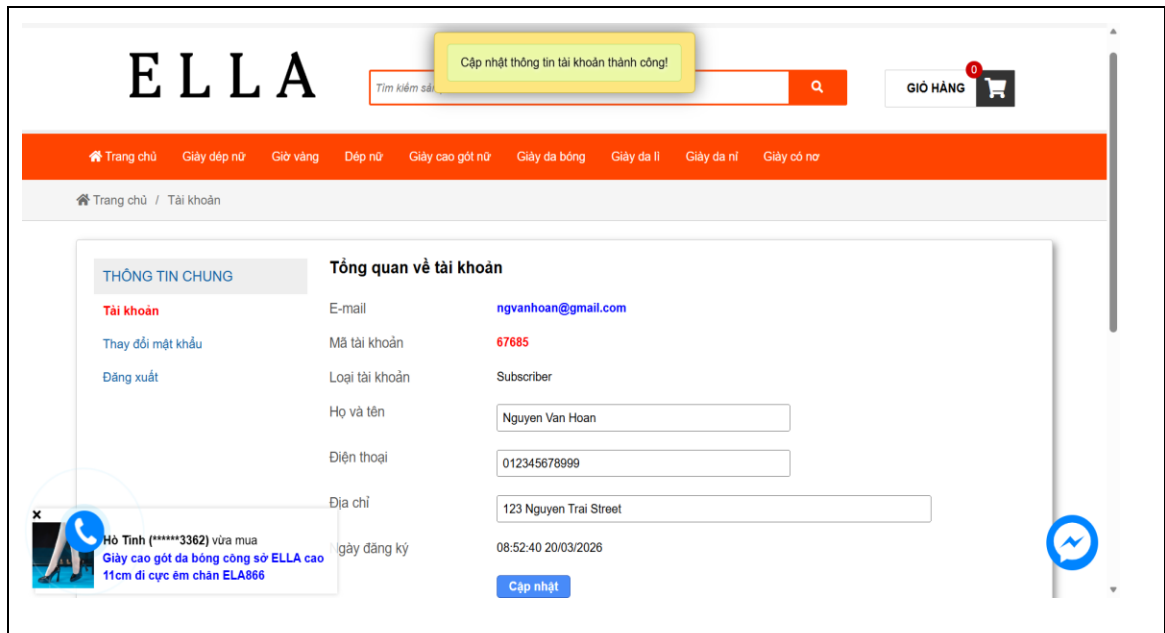
Hình 4.57 Báo cáo Allure kết quả thực thi chức năng Cập nhật hồ sơ



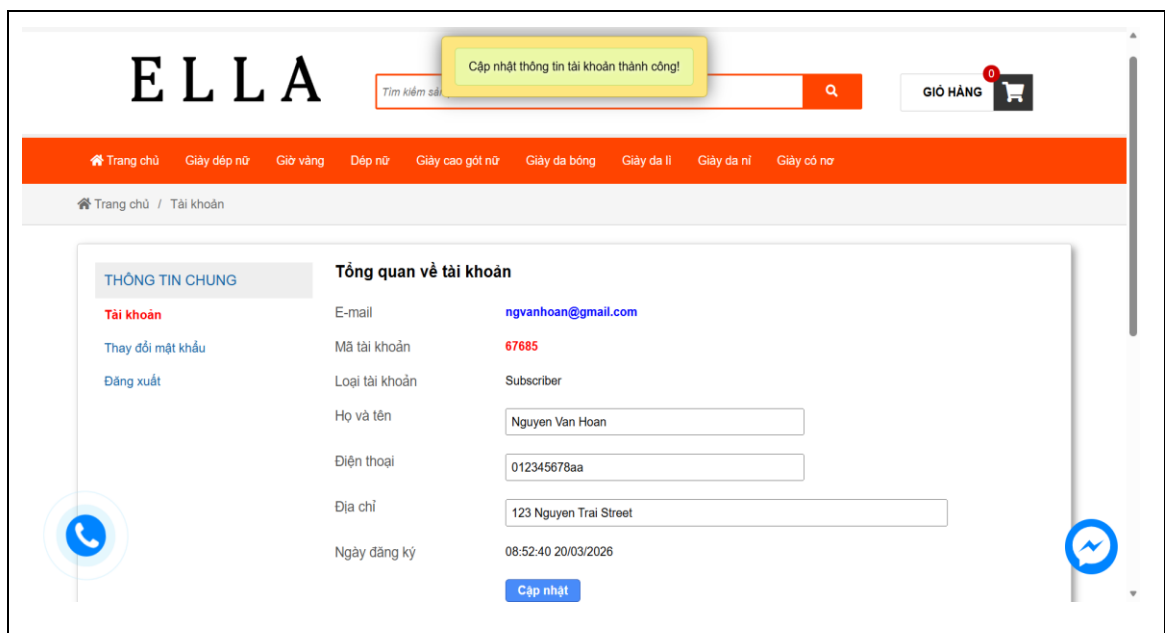
Hình 4.58 Ảnh màn hình lỗi chức năng Cập nhật hồ sơ (test 5)



Hình 4.59 Ảnh màn hình lỗi chức năng Cập nhật hồ sơ (test 6)



Hình 4.60 Ảnh màn hình lỗi chức năng Cập nhật hồ sơ (test 7)



Hình 4.61 Ảnh màn hình lỗi chức năng Cập nhật hồ sơ (test 8)

4.7 Đóng gói và hướng dẫn sử dụng framework

Sau khi xây dựng các chức năng chính của framework, hệ thống được tổ chức và đóng gói lại theo cấu trúc thư mục rõ ràng nhằm giúp người dùng có thể dễ dàng cài đặt, sử dụng và mở rộng. Việc đóng gói framework không chỉ bao gồm mã nguồn chương trình mà còn bao gồm tài liệu hướng dẫn sử dụng, vị trí đặt dữ liệu đầu vào,

vị trí lưu kết quả đầu ra, cách thực thi các chức năng sinh keyword, sinh luồng kiểm thử E2E và cách xem báo cáo kết quả kiểm thử.

Mục tiêu của việc đóng gói là giúp framework có thể được bàn giao cho người dùng khác, ví dụ như tester hoặc người phát triển tiếp theo, mà không phụ thuộc hoàn toàn vào người xây dựng ban đầu. Người dùng chỉ cần đọc tài liệu hướng dẫn, chuẩn bị dữ liệu đầu vào theo đúng định dạng, chạy các lệnh tương ứng và kiểm tra kết quả đầu ra được sinh ra bởi hệ thống.

4.7.1 Hướng dẫn chuẩn bị dữ liệu đầu vào

Đối với chức năng sinh keyword bằng AI, người dùng cần chuẩn bị file mô tả chức năng và đặt vào thư mục `generate_keywords_use_AI/input/`. Nội dung file mô tả cần thể hiện rõ tên chức năng, các bước thao tác chính, dữ liệu đầu vào và kết quả mong đợi.

Ví dụ, với chức năng đăng nhập, người dùng có thể tạo file `login.txt` với nội dung mô tả các bước như mở trang đăng nhập, nhập email, nhập mật khẩu, nhấn nút đăng nhập và kiểm tra kết quả đăng nhập.

Đối với chức năng sinh luồng kiểm thử E2E, người dùng cũng đặt file mô tả kịch bản nghiệp vụ vào thư mục `generate_keywords_use_AI/input/`. Nội dung file cần mô tả thứ tự các nghiệp vụ trong luồng, ví dụ: đăng nhập, tìm kiếm sản phẩm, thêm vào giỏ hàng và đặt hàng.

4.7.2 Hướng dẫn sinh keyword và sinh luồng E2E

Sau khi chuẩn bị dữ liệu đầu vào, người dùng có thể chạy script tương ứng để sinh keyword hoặc sinh luồng kiểm thử. Đối với chức năng sinh keyword, hệ thống sử dụng file mô tả chức năng, kết hợp với prompt đã xây dựng để gửi yêu cầu đến mô hình AI. Kết quả trả về được xử lý, kiểm tra trùng lặp và inject vào các tầng keyword phù hợp như UI, Business hoặc Verify.

Lệnh thực thi chức năng sinh keyword:

```
python generate_keywords_use_AI/generate_keyword.py <feature>
```

Đối với chức năng sinh luồng kiểm thử E2E, hệ thống đọc mô tả nghiệp vụ đầu vào, tải danh sách keyword hiện có trong framework, sau đó yêu cầu AI sinh ra flow kiểm thử phù hợp. Kết quả flow được lưu tại thư mục output và được hệ thống chuyển thành file .robot trong thư mục tests/e2e/. Lệnh thực thi sinh luồng E2E:

```
python generate_keywords_use_AI/generate_e2e.py <path in>
```

4.7.3 Hướng dẫn thực thi kiểm thử và xem báo cáo

Sau khi keyword hoặc flow kiểm thử được sinh ra, tester có thể kiểm tra và bổ sung logic cần thiết trong các file Robot Framework. Để chạy framework, người dùng thực thi file run.py. File này cung cấp menu lựa chọn chức năng kiểm thử, giúp người dùng dễ dàng chọn test case hoặc luồng E2E cần chạy.

Lệnh thực thi kiểm thử: `python run.py`

Sau khi quá trình kiểm thử hoàn tất, hệ thống sinh báo cáo kết quả tại thư mục report/. Báo cáo bao gồm log thực thi, báo cáo Robot Framework và báo cáo Allure. Ngoài ra, nếu test case bị lỗi, hệ thống sẽ lưu ảnh chụp màn hình tại thư mục screenshots/ để hỗ trợ quá trình phân tích lỗi.

Bên cạnh nội dung trình bày trong báo cáo, framework còn được bổ sung file README.md tại thư mục gốc của project. File README.md đóng vai trò là tài liệu hướng dẫn sử dụng nhanh cho người dùng khi nhận mã nguồn framework. Nội dung README.md bao gồm giới thiệu framework, công nghệ sử dụng, cấu trúc thư mục, yêu cầu cài đặt, cách chuẩn bị đầu vào, cách chạy các chức năng AI, cách thực thi kiểm thử và cách xem báo cáo kết quả.

Việc bổ sung README.md giúp framework có tính hoàn thiện hơn, dễ bàn giao hơn và hỗ trợ người dùng khác có thể tiếp cận, cài đặt và sử dụng framework mà không cần phụ thuộc hoàn toàn vào người phát triển ban đầu.

KẾT LUẬN

1. Kết quả đạt được của đề tài

- Xây dựng được framework kiểm thử tự động ứng dụng Web bằng Robot Framework kết hợp SeleniumLibrary.
- Thiết kế framework theo kiến trúc phân tầng: Business Layer, UI Action Layer, Verification Layer.
- Tổ chức framework theo hướng dễ bảo trì và mở rộng.
- Xây dựng được cơ chế sử dụng AI thông qua Ollama Local API để hỗ trợ sinh keyword kiểm thử từ mô tả chức năng đầu vào.
- Xây dựng được cơ chế xử lý output do AI sinh ra.
- Xây dựng được cơ chế sinh luồng kiểm thử, luồng End-to-End kiểm thử bằng AI dựa trên keyword repository hiện có trong framework.
- Tích hợp các thành phần hỗ trợ quá trình kiểm thử tự động: ghi log, sinh báo cáo Allure, chụp màn hình khi lỗi,
- Thực nghiệm framework trên các chức năng chính của ứng dụng Web.
- Cuốn báo cáo Đồ án tốt nghiệp, slide thuyết trình đảm bảo đầy đủ nội dung, cấu trúc, định dạng theo quy định.
- Repository mã nguồn đã đóng gói, có README.md hướng dẫn sử dụng.

2. Hạn chế của đề tài

Mặc dù đề tài đã đạt được các mục tiêu chính đặt ra, trong phạm vi thời gian và điều kiện triển khai của đồ án, hệ thống vẫn còn một số giới hạn nhất định như sau:

- AI mới hỗ trợ khởi tạo keyword và luồng kiểm thử, chưa thể tự động hoàn thiện toàn bộ test script.
- Cơ chế kiểm tra trùng lặp keyword mới dừng ở mức cơ bản, chưa xử lý tốt các keyword có ý nghĩa tương đồng phức tạp.

- Output do AI sinh ra vẫn cần kiểm thử viên kiểm tra và chỉnh sửa trước khi sử dụng.
- Framework chưa tích hợp các kỹ thuật nâng cao như RAG, semantic matching, tự động sinh locator, tự động sinh test data và CI/CD.

3. Hướng phát triển của đề tài

Trong tương lai, đề tài có thể tiếp tục phát triển theo các hướng sau:

- Nâng cấp cơ chế kiểm tra trùng lặp keyword bằng semantic matching, embedding hoặc vector database.
- Tích hợp RAG để AI khai thác keyword repository và ngữ cảnh framework tốt hơn.
- Nghiên cứu tự động sinh locator và dữ liệu kiểm thử bằng AI.
- Nâng cao khả năng sinh flow End-to-End cho các nghiệp vụ phức tạp hơn.
- Tích hợp framework với CI/CD để hỗ trợ kiểm thử tự động liên tục.
- Thử nghiệm thêm các mô hình AI khác nhau để đánh giá và lựa chọn mô hình phù hợp hơn.

TÀI LIỆU THAM KHẢO

- [1] Trường Đại học Sư phạm Kỹ thuật Hưng Yên, Giáo trình Kiểm thử phần mềm tự động, Hưng Yên, 2025.
- [2] Trường Đại học Sư phạm Kỹ thuật Hưng Yên, Giáo trình kiểm thử ứng dụng Web và di động, Hưng Yên, 2025.
- [3] Trường đại học Sư phạm Kỹ thuật Hưng Yên, Kiểm thử phần mềm, Hưng Yên, 2024.
- [4] A. W. Services, “What is Prompt Engineering?,” 7 March 2026 . [Online]. Available: <https://aws.amazon.com/what-is/prompt-engineering/>. [Accessed: March 10, 2026].
- [5] Robot Framework Foundation, “Robot Framework User Guide,” Robot Framework Foundation. [Online]. Available: <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html>. [Accessed: March 10, 2026].
- [6] A. Chatterjee, “What is Keyword-Driven Testing?,” testRigor, Sep. 27, 2024. [Online]. Available: <https://testrigor.com/blog/keyword-driven-testing/>. [Accessed: March 10, 2026].
- [7] Amazon Web Services, “What is LLM (Large Language Model)?,” AWS. [Online]. Available: <https://aws.amazon.com/what-is/large-language-model/>. [Accessed: March 10, 2026].
- [8] Ariffud M., "What is Ollama? Understanding how it works, main features and models," Hostinger, Dec. 22, 2025. [Online]. Available: <https://www.hostinger.com/tutorials/what-is-ollama>. [Accessed: March 10, 2026].